SHIPHARNESS · TEAMTRACK

bare arm harness arm

Module 1: Harness Engineering for Coding Agents

Live Demo Guide

Every command, every screen and every result of the five live demos, step by step, with what to say in class.

2026-10-18

Console screens 1 to 5 · Terminal 1 and Terminal 2

Contents

INTRODUCTION Why this guide exists 3

PART 0 Before you start 4
- What you need 4
- What PURPOSE=pilot means 11

CHAPTER 1 Section 1: Instructions 13
- Result summary 35
- Time and cost 35
- What to say in class 35

CHAPTER 2 State and sessions 37
- Result summary 67
- Time and cost 68
- What to say in class 68

CHAPTER 3 Section 3: Scope and sandbox 69
- Result summary 95
- Time and cost 95
- What to say in class 96

CHAPTER 4 Section 4: Verification 97
- Result summary 128
- Time and cost 128
- What to say in class 129
- Sources 129

CHAPTER 5 Final project: the whole harness, live A/B 130
- Result summary 161
- Time and cost 162
- What to say in class 162

APPENDIX Cheat sheet, troubleshooting, glossary and fallback 163
- Command cheat sheet 163
- Troubleshooting 164
- Glossary 165
- Fallback: replay a recorded run 166

INTRODUCTION

Why this guide exists

A coding agent can write a feature, run the tests, and say "done", while the work is wrong in ways the tests never check. It copies a query with no tenant filter from old code. It loses its plan when a session stops and redoes a migration. It edits files the ticket never mentioned. It claims the work is finished when the hidden tests fail. More prompting does not fix this. What fixes it is the harness: the files and code around the agent that decide what it sees and what it may do.

This guide walks you through the five live demos of Module 1, one command and one screen at a time. In each demo the agent works on TeamTrack, a small multi-tenant issue tracker. The referee (the program that scores each run after the agent stops) then answers one question fixed before any run. You watch it happen on the console, a browser page with five screens.

Four demos show one harness layer each: instructions, state and sessions, scope and sandbox, and verification. Section 3: Scope and sandbox and the final project are A/B runs: the same ticket with the harness off (the bare arm) and on (the harness arm), scored the same way. The results are measured, not promised: some layers made a clear difference on these tickets, and some made none. Each chapter shows what this run found, and what to say about it in class.

PART 0

Before you start

Every demo in this guide uses the same set-up: two Ubuntu terminals in your copy of the course repository, and the console open in a browser. This part gets you there and checks that the machine is ready, with no model calls and no cost. Do it once before class, and again on the morning of the class (Steps 3 to 7 take a few minutes once everything is installed).

> **Terms on this page**
>
> - **Console**: the browser page that shows a run live, on five screens.
> - **Referee**: the program that scores each run after the agent stops.
> - **Class date**: the class day in `.env`, which names the LangSmith project and keys the session cap.
> - **Purpose**: the make variable that picks the LangSmith project and the cost session.

What you need

Item	What it is for	Where to set it up
Windows 11 with WSL2 and Ubuntu	Every course command runs in the Ubuntu terminal	`setup/windows.md`
Docker Engine inside Ubuntu	The agent and the referee run in containers	`setup/windows.md`, step 5
git, uv and make	Clone, Python packages, the `make` commands	`setup/windows.md`, steps 6 to 8
A clone of the course repository	The code, the app, and the checkpoint tags	`setup/windows.md`, step 9
Two keys in `.env`	OpenAI for the agent, LangSmith for the traces	`setup/windows.md`, step 10

On macOS or Linux, follow `setup/macos.md` or `setup/linux.md` instead; every command in this guide is the same, and "Ubuntu terminal" means your normal terminal.

> ⚠ **WARNING**
>
> Your two keys live only in `.env` at the repository root. Never paste them into a chat, a ticket, a slide, or a command line, and never share a screenshot of `.env`. The tools read the file themselves and never print a key. If a key leaks, delete it on its provider's page and create a new one.

Step 1 Open Terminal 1

WHERE: Windows, the Start menu

TYPE: Click Start, type `Ubuntu`, and open the Ubuntu app. From PowerShell you can type this instead:

TYPE THIS ⧉ COPY

```
wsl -d Ubuntu
```

Then go to your copy of the repository:

TYPE THIS ⧉ COPY

```
cd ~/shipharness
```

YOU SHOULD SEE: an Ubuntu prompt that ends in `~/shipharness$`.

LOOK AT: the folder in the prompt. It starts with `~`, not `/mnt/c`.

WHAT IT MEANS: this is Terminal 1. In this guide it runs the console and nothing else, so it stays open for the whole class.

✓ **CHECKPOINT**

`pwd` prints a path in your Linux home folder that ends in `/shipharness`.

IF IT LOOKS DIFFERENT

`No such file or directory` → your copy has another name or place; `cd` into the folder you cloned.
The path starts with `/mnt/c` → the copy is on the Windows drive and runs slowly; clone it again under `~` (`setup/windows.md`, step 9).
`wsl` is not found in PowerShell → WSL is not installed (`setup/windows.md`, step 4).

◆ **INSTRUCTOR NOTE**

Put Terminal 1 on the left half of the shared screen and the browser on the right. Say: "This terminal only serves the console. The runs start in the other one."

Step 2 Open Terminal 2

WHERE: Windows, a second Ubuntu window

TYPE: Open the Ubuntu app a second time (or a second tab in Windows Terminal), then:

TYPE THIS ⧉ COPY

```
cd ~/shipharness
```

YOU SHOULD SEE: a second prompt that ends in `~/shipharness$`.

LOOK AT: the window title or tab, so you can tell the two terminals apart.

WHAT IT MEANS: this is Terminal 2. Every check and every demo run starts here.

✓ CHECKPOINT
Two Ubuntu windows are open, both in the repository folder.

IF IT LOOKS DIFFERENT
The prompt shows another folder → run the `cd` command again in this window.

◆ INSTRUCTOR NOTE
Name the two terminals out loud once: "Terminal 1 serves the console, Terminal 2 starts the runs." The chapters use these names in every step. The class plan in `docs/instructor-guide.md` also starts the final A/B in a third Ubuntu window, opened the same way, right after `make show-4`. Every step and every time in this guide uses Terminal 2 alone, with one demo running at a time.

Step 3 Check that your copy matches the class release

WHERE: Terminal 2

TYPE:

TYPE THIS ⧉ COPY

```
uv run shipharness verify --tag class-v1
```

YOU SHOULD SEE: one line that starts with `your copy matches class-v1:`, followed by the number of paths and checkpoint trees it compared. On the course machine, for a fresh clone of the release:

> Your output will differ: run times, ids, and the agent's wording change on every run.

OUTPUT

```
your copy matches class-v1: 27 paths, 8 checkpoint trees
```

LOOK AT: the first words: `your copy matches`.

WHAT IT MEANS: the command compares every course file and every checkpoint tag in your copy with the record stored in the `class-v1` tag. It makes no model calls. A match means your demos start from the same code as the course's runs.

✓ CHECKPOINT
The line starts with `your copy matches class-v1`, and the command exits without an error.

IF IT LOOKS DIFFERENT

`x tag class-v1 not found` → run `git fetch --tags`, then run the command again.
One line per changed file → you edited a course file; `git status` names it, and `git restore` with the file name puts it back.
`is not a git clone` → you are not in the repository folder; `cd ~/shipharness`.

◆ INSTRUCTOR NOTE

Run this on the class machine the morning of the class. A copy that does not match can give a different result from the one this guide shows.

Step 4 Run make doctor

WHERE: Terminal 2

TYPE:

TYPE THIS ⧉ COPY

```
make doctor PURPOSE=pilot
```

YOU SHOULD SEE: one line per check, each starting with `+ ok`: python, git, uv, the container runtime, both keys, the two models, the prices, and `.env not in git`. It took 6 s on the course machine. The last lines there:

OUTPUT

```
  + ok          OPENAI_API_KEY         accepted by OpenAI
  + ok          agent model            gpt-6-luna accepted
  + ok          reviewer model         gpt-5.6-luna accepted
  + ok          LANGSMITH_API_KEY      accepted by LangSmith
  + ok          prices                 gpt-6-luna, gpt-5.6-luna
  + ok          .env not in git        not tracked
  + ok          SHIPHARNESS_CLASS_DATE not needed for PURPOSE=pilot
  result: + ready
Cost: $0.000000 (no model calls)
```

LOOK AT: the key lines. They end in `accepted by OpenAI` and `accepted by LangSmith`, and no part of a key is ever shown.

WHAT IT MEANS: one command checks the whole machine. The OpenAI checks ask whether the key works and whether it can use both course models; they use no tokens. `PURPOSE=pilot` skips the class date check (see "What PURPOSE=pilot means" below).

✓ CHECKPOINT

The last line reads `result: + ready`, and there is no `make: ***` line after it.

IF IT LOOKS DIFFERENT

`x missing OPENAI_API_KEY` (or `LANGSMITH_API_KEY`) → put the key in `.env` next to the `Makefile`.
`x missing agent model` (or `reviewer model`) → use the key from the program's email, or your own key on the Build tier.
`! warn container runtime` → start Docker with `sudo systemctl start docker`.

◆ INSTRUCTOR NOTE

Every `x` line has a `next step` line under it. Read that line out loud if a learner's check fails: the fix is printed there.

Step 5 Run make warm

WHERE: Terminal 2

TYPE:

TYPE THIS ⧉ COPY

```
make warm PURPOSE=pilot
```

YOU SHOULD SEE: one line per step with its time, then `warm: done` and a cost line. A first warm-up on a fresh clone builds both images and computes 10 baselines; on the course machine it took 431 s. Its first and last lines:

OUTPUT

```
images: built (shipharness-agent:1: no record of its source; shipharness-referee:1: no record of its source) (236.8 s)
page-in shipharness-agent:1 (0.6 s)
page-in shipharness-referee:1 (0.4 s)
plan: 10 baselines to compute (0.5 s)
referee baseline demo-1 (Feature 1: Org-wide issue search): computed (67.4 s)
gate baseline final (Feature 7: Labels): computed (43.0 s)
warm: done (429.0 s)
Cost: $0.000000 (this command: no calls; session $0.000000 of $2.50; build $0.000000 of $15.00 ($15.000000 left))
```

The excerpt skips the build log and the middle lines: one `referee baseline` line per class task and one `gate baseline` line per gated task. A second `make warm` builds nothing, every baseline line ends in `cached`, and it takes about 2 s.

LOOK AT: the last line before the cost line: `warm: done`.

WHAT IT MEANS: `make warm` rebuilds an image only if its source changed, starts each image once, and saves the referee's and the gate's results on the unchanged app (the baselines), so the first demo starts at once. It makes no model calls and is safe to run again.

✓ CHECKPOINT

The last line before the cost line is `warm: done`.

IF IT LOOKS DIFFERENT

`x images: build failed` → start Docker, then run `make images` to see the build error.
The last line is `warm: incomplete` → fix the step marked `x` and run `make warm PURPOSE=pilot` again; finished baselines stay cached.

◆ INSTRUCTOR NOTE

Run `make warm` on the class machine at least 30 minutes before class, so a first warm-up never runs during the session.

Step 6 Start the console

WHERE: Terminal 1

TYPE:

TYPE THIS ⧉ COPY

```
make console
```

YOU SHOULD SEE: three lines, then nothing more: the `uv run shipharness console --port 8765` command, the address line `Console: http://127.0.0.1:8765/`, and the line `Only warnings and errors print here; make console VERBOSE=1 (--verbose) logs each request.`

LOOK AT: the address on the `Console:` line: `http://127.0.0.1:8765/`.

WHAT IT MEANS: the console is a small web server that reads the run folders on disk. It is quiet by default: only the address, warnings and errors print here. Leave it running for the whole class. `make console VERBOSE=1` also prints one line per browser request.

✓ CHECKPOINT

The `Console:` line shows `http://127.0.0.1:8765/`, and the prompt does not come back.

IF IT LOOKS DIFFERENT

`address already in use` → another program holds port 8765: run `make console CONSOLE_PORT=8766` and use port 8766 in the next step.
The prompt comes back with an error → read the last line; if it names a missing package, run `make setup` once.

◆ INSTRUCTOR NOTE

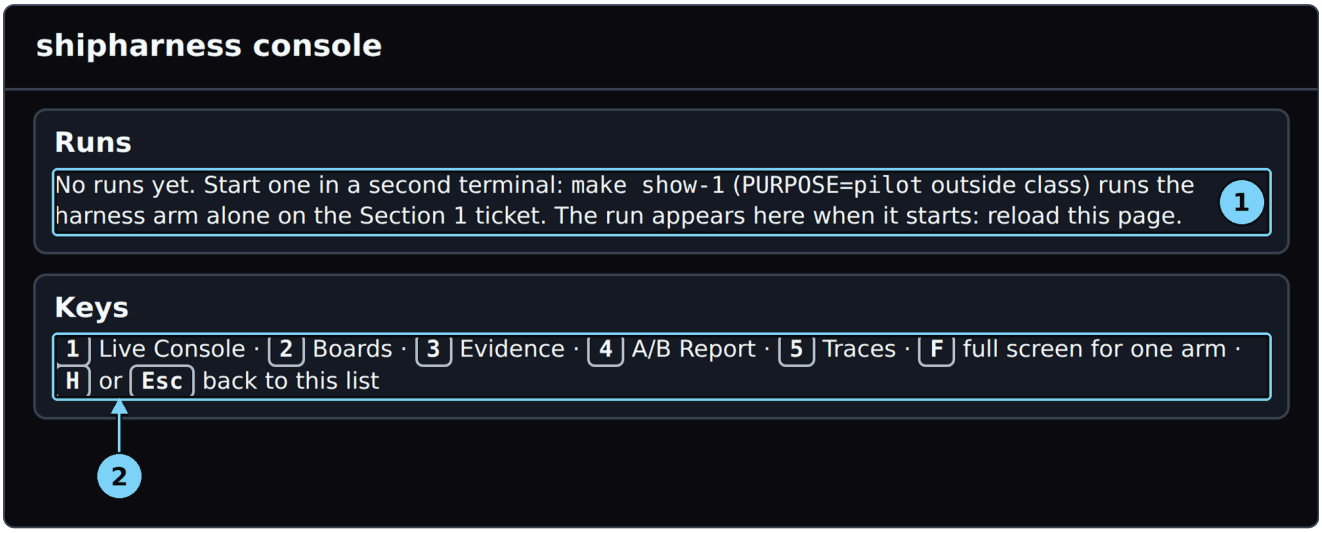
Never close Terminal 1 during class. If the console header shows `reconnecting`, the server stopped: run `make console` again and reload the browser.

Step 7 Open the console in the browser

WHERE: your Windows browser

TYPE: Open a new tab and go to `http://127.0.0.1:8765/`.

YOU SHOULD SEE: the "Runs" panel and the "Keys" panel. Before the first run, the list is empty:



A fresh console: no runs yet, and the keys every chapter uses.

1. No runs yet: start one in Terminal 2
2. Console keys

Once runs exist, each one is a card with its run id, section, feature, type ("harness in action" for a single-arm run, "A/B" for two arms), status ("finished" once the referee is done), and start time, newest first.

LOOK AT: the "Keys" panel under the run list: it lists the keys 1 to 5 and the screen each one opens, then `F`, and `H` or `Esc`.

WHAT IT MEANS: the console shows every run under `.shipharness/runs/` in your copy. A new run appears when you reload the page; the list does not refresh by itself.

✓ CHECKPOINT

The page loads and shows the "Runs" panel; on a fresh copy it starts with "No runs yet".

IF IT LOOKS DIFFERENT

The page does not load → check that Terminal 1 still runs `make console`, then try `http://localhost:8765/`.
Text or panels look cut off → press `Ctrl+0` to reset the browser zoom, then `Ctrl+F5` to reload.
The page looks like an older design → press `Ctrl+F5` for a hard reload.

◆ INSTRUCTOR NOTE

Zoom the browser so the back row can read the pills; the console is fluid and reflows at any zoom or Windows display scaling. `Ctrl+0` returns to the normal size.

Step 8 Learn the console keys

WHERE: browser, the console

TYPE: Nothing yet: a fresh copy has no runs, so there is no run card to open. Read the table now. After your first run (Chapter 1, Step 5), click the run id on its card, press `1`, `2`, `3`, `4` and `5` in turn, press `F`, then press `H` to go back to the run list.

YOU SHOULD SEE: once a run is open, a different screen for each number key:

Key	Screen	What it shows
`1`	Live	Each arm's turns, cost, harness decisions, the gate, and the verdict
`2`	Boards	Each arm's running app as a user sees it, every 5 s
`3`	Evidence	Files read, the diff, the referee's checks, the test filters
`4`	A/B report	The runs measured before class, one dot per run, and this run
`5`	Traces	A live tail of events and the LangSmith trace links

LOOK AT: the tabs at the top: the current screen's tab is outlined. `F` cycles one arm to the full width (both, bare, harness). "← All runs", `H` or Esc returns to the run list.

WHAT IT MEANS: every chapter of this guide names the screen by its key, such as "Press 3".

> ✓ **CHECKPOINT**
> You know which key opens which screen. After Chapter 1, each number key changes the screen, and `H` brings back the run list.

> **IF IT LOOKS DIFFERENT**
> The keys do nothing → click once on the page so it has the keyboard focus, then press the key again.
> The run list still says "No runs yet" → expected before Chapter 1; come back to this step after your first run.

> ◆ **INSTRUCTOR NOTE**
> Practise the keys before class with the run from your practice of Chapter 1. On screen 4, a run of type "harness in action" has one arm, so the screen has no two arms to compare.

What PURPOSE=pilot means

Every live command and every check takes a `PURPOSE`. It picks two things: the LangSmith project the traces go to, and the cost session that the session cap counts against. The run itself is the same: the same agent, the same harness, the same referee, and the same caps per run.

[illegible]	LangSmith project	Cost session
`class` (the default)	`shipharness-class-` plus the class date	One per class date
`pilot`	`shipharness-pilot`	One, named `pilot`
`assignment`	`shipharness-assignments`	One, named `assignment`

This guide adds `PURPOSE=pilot` so that practice runs never use the class project or the class day's $2.50 session cap. On class day, set `SHIPHARNESS_CLASS_DATE` in `.env` to the class day and run the same commands without `PURPOSE=pilot`: the traces then go to the class project.

CHAPTER 1

Section 1: Instructions

A coding agent is asked to add an org-wide issue search to a multi-tenant tracker. It finds an old list handler with no org filter and copies it, and the tests still pass. One customer can then see another customer's issues. In this chapter you run `make show-1`: the harness arm alone, live, on Feature 1: Org-wide issue search, with the instructions layer on (a short `AGENTS.md` map, focused documents, and one skill). You watch which files the agent opens. Then you read the referee's answer to one question: did the change add a new violation of House rule 1: Tenant isolation (another org's resource answers 404, never 403)?

Every number, quote and screenshot in this chapter comes from one live run of `make show-1 PURPOSE=pilot` on 2026-10-18, run `ab-20261018-162244-860638`, with nothing else running on the machine.

> Your output will differ: run times, ids, and the agent's wording change on every run.

Step 1 Check the console and the run list

WHERE: browser, the console's run list

TYPE: Click the console tab in your browser. If it shows a run, click "← All runs" or press H.

YOU SHOULD SEE: the page "shipharness console", with the "Runs" panel ("Newest first.") and the "Keys" panel.

LOOK AT: the top card in "Runs". Note its start time, so you can tell the new run apart in Step 4.

WHAT IT MEANS: Terminal 1 still runs `make console` (Part 0). The console reads the run folders on disk, so a new run appears as soon as it writes its first event.

✓ CHECKPOINT
The run list loads, and the "Keys" panel lists 1 Live Console to 5 Traces.

IF IT LOOKS DIFFERENT

- The page does not load → Terminal 1 stopped. Run `make console` again there, then reload.
- `address already in use` in Terminal 1 → another program holds port 8765. Run `make console CONSOLE_PORT=8766` and open that port.

◆ INSTRUCTOR NOTE
Put this tab and Terminal 2 side by side on the shared screen. Say: "One terminal starts the run; the browser shows it."

Step 2 Start the run

WHERE: Terminal 2, in the repository folder

TYPE:

TYPE THIS ⧉ COPY

```
make show-1 PURPOSE=pilot
```

YOU SHOULD SEE: make prints the command it runs, then nothing more for a few minutes:

OUTPUT

```
uv run shipharness ab --section demo-1 --feature org-search --arms harness   --purpose pilot \
  --profile class
```

LOOK AT: `--arms harness`. Only the harness arm runs.

WHAT IT MEANS: `make show-1` starts one agent in a container on the Feature 1 ticket. The map, the documents and the skill are placed in its workspace. When the agent stops, the referee scores what it shipped, House rule 1 first, then everything else. `PURPOSE=pilot` sends the traces to the practice project in LangSmith (the tracing service). In class, set `SHIPHARNESS_CLASS_DATE` in `.env` and run `make show-1` alone.

✓ CHECKPOINT

The `uv run shipharness ab` line appears and the prompt does not come back.

IF IT LOOKS DIFFERENT

- `x OpenAI refused the key`, no arm started → fix `OPENAI_API_KEY` in `.env`, then run `make doctor`.
- `x SHIPHARNESS_CLASS_DATE is not set` → add the class day to `.env`, or add `PURPOSE=pilot`.
- The prompt comes back at once with `docker run failed` → start Docker (`sudo systemctl start docker`), then run `make warm`.

◆ INSTRUCTOR NOTE

Launch on the Section 1 start slide, one minute in. This run took 4 min 45 s (two runs that ran alone: 3:37 to 4:45), so it ends long before the DEMO slide. One run costs at most $0.24: the per-arm cap plus the per-review cap, both in `.env.example`.

Step 3 Wait while it runs

WHERE: Terminal 2

TYPE: Nothing. Leave Terminal 2 alone.

YOU SHOULD SEE: no new lines until the end. In this run the command ended 4 min 45 s after launch.

LOOK AT: the browser instead (next step).

WHAT IT MEANS: the command prints the run id only at the end, so you find the run through the console's run list.

✓ CHECKPOINT
No error line in Terminal 2.

IF IT LOOKS DIFFERENT
- Nothing seems to happen for several minutes → run `docker ps` in another terminal. The agent and referee containers stay listed while the run works.

◆ INSTRUCTOR NOTE
At section minute 15, glance at Terminal 2 for the `Cost:` line. If it is not there yet, carry on teaching.

Step 4 Reload the run list

WHERE: browser, run list

TYPE: Reload the page (Ctrl+R).

YOU SHOULD SEE: a new card at the top of "Runs", with the type "harness in action" and the status "running".

The show-1 card after the run finished. It is at the bottom here because five later runs started after it.

1. Run id: open the run
2. Type: harness in action or A/B
3. Status
4. Start time

LOOK AT: the type and the status on the new card.

WHAT IT MEANS: "harness in action" means one arm, not an A/B. Its result stays out of every bare-versus-harness comparison.

✓ CHECKPOINT

The top card says "Section 1: Instructions", "Feature 1: Org-wide issue search", "harness in action" and "running", with today's start time.

IF IT LOOKS DIFFERENT

- No new card → wait 10 seconds and reload again; the card appears with the run's first event.
- The top card is an older run → compare the start time with Step 1, and check Terminal 2 for an error.

◆ INSTRUCTOR NOTE

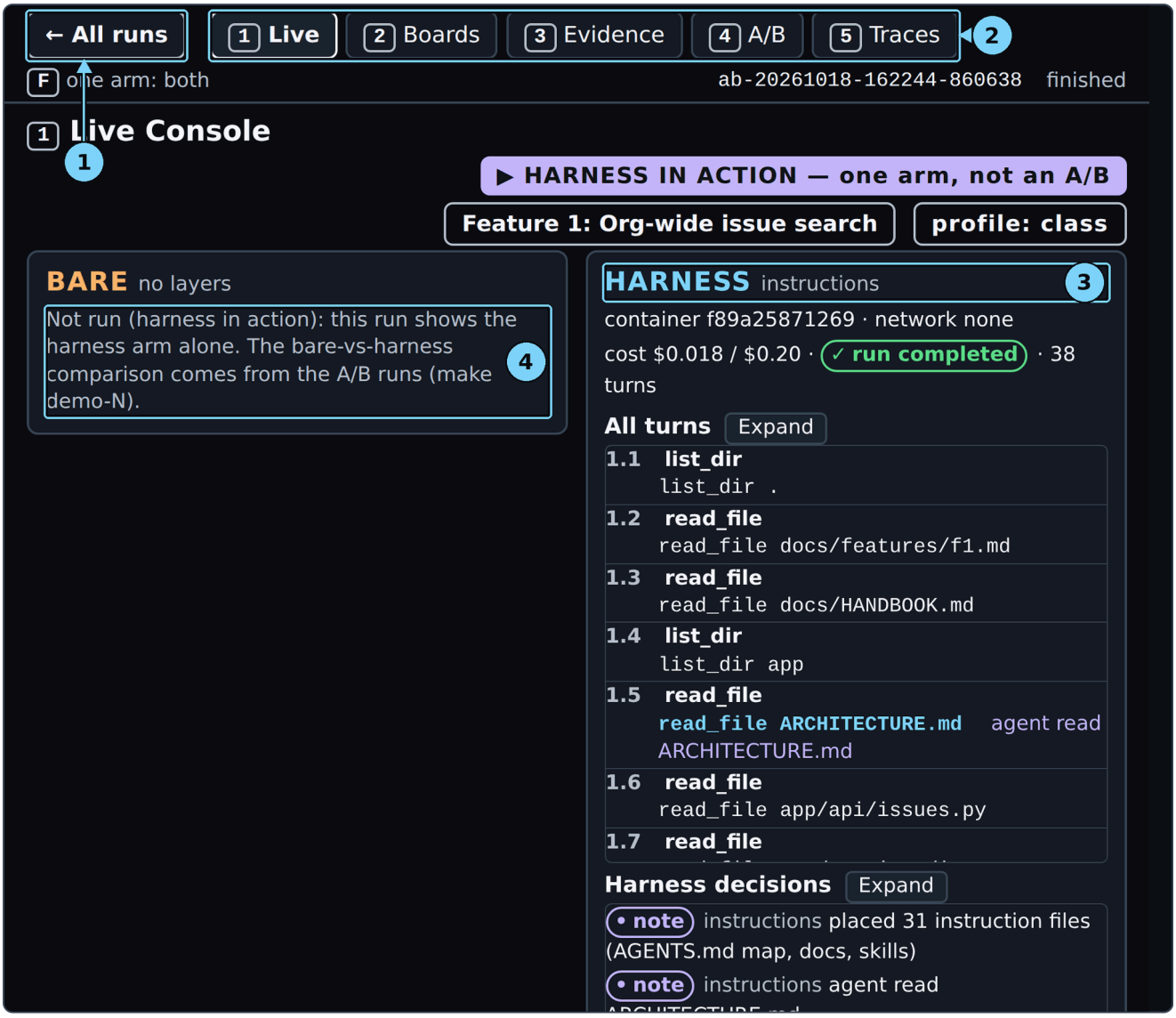
Say: "This list does not refresh by itself. Reload after every launch."

Step 5 Open the run

WHERE: browser, run list

TYPE: Click the run id on the top card, then press 1.

YOU SHOULD SEE: screen 1, "Live Console", with the badges "HARNESS IN ACTION — one arm, not an A/B", "Feature 1: Org-wide issue search" and "profile: class".



Screen 1 of this run. The left panel is the bare arm, the right panel the harness arm.

1. ← All runs: back to the run list
2. Screen tabs 1 to 5
3. Harness arm line: its layers
4. Bare arm: Not run (harness in action)

LOOK AT: the left panel, BARE: "Not run (harness in action): this run shows the harness arm alone. The bare-vs-harness comparison comes from the A/B runs (make demo-N)."

WHAT IT MEANS: in a single-arm run no bare agent starts. Every screen says "Not run (harness in action)" for the bare arm instead of an empty result. The comparison with the bare arm comes from the runs before class, on screen 4.

✓ CHECKPOINT

The right panel, HARNESS instructions, shows a container line ("container … · network none") and a cost line ("cost … / $0.20").

IF IT LOOKS DIFFERENT

- "no container yet" → the container is still starting; wait a few seconds.
- The keys do nothing → click once on the page so it has focus, then press 1.

◆ INSTRUCTOR NOTE

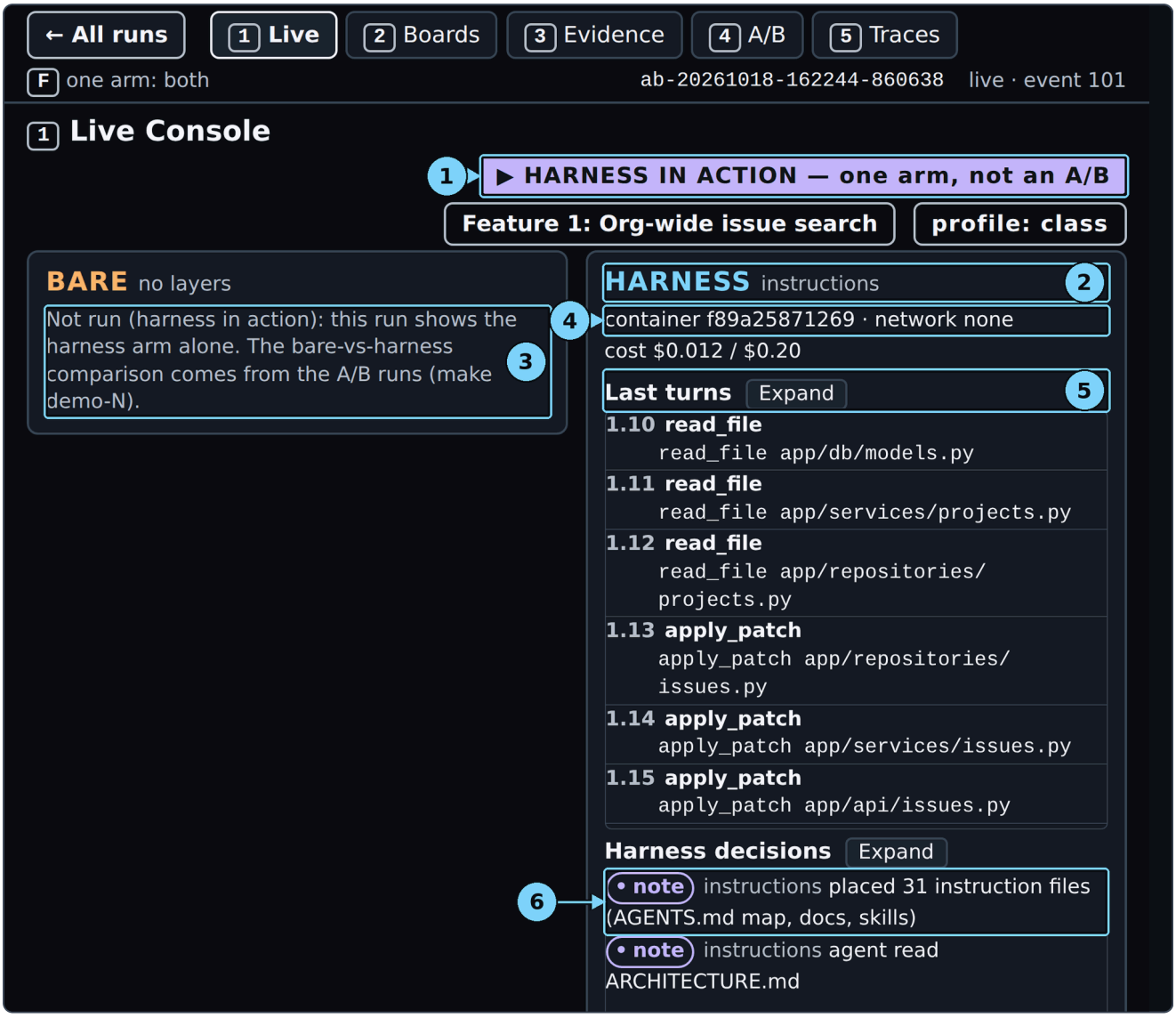
Point at the bare panel: "No bare arm today. Before class we compared both arms on this ticket and found no measurable difference. Today shows the map at work."

Step 6 Read the first harness notes

WHERE: browser, screen 1, right panel

TYPE: Press 1, if you are on another screen.

YOU SHOULD SEE: under "Harness decisions": "• note instructions placed 31 instruction files (AGENTS.md map, docs, skills)".



Early in the run (the run's log replayed to event 101): the agent has read the documents and starts writing code.

1. HARNESS IN ACTION badge, or the feature
2. Harness arm line: its layers
3. Bare arm: Not run (harness in action)
4. Container line
5. Last turns
6. Harness decisions: the layer notes

LOOK AT: the note "placed 31 instruction files".

WHAT IT MEANS: before the agent's first turn, the instructions layer copied the map, the documents and the skill into the workspace. The map goes into the agent's context in every session. The documents and the skill body stay on disk until the agent opens them.

✓ CHECKPOINT
"placed 31 instruction files" is the first harness decision.

IF IT LOOKS DIFFERENT
- "Harness decisions: None yet." → the first note comes about a second after the container starts; wait.

◆ INSTRUCTOR NOTE
Say: "The map is about 100 lines and points outward. The bare arm has every rule too, in one long handbook."

Step 7 Watch which files the agent opens

WHERE: browser, screen 1, right panel

TYPE: Click "Expand" next to "Last turns" ("All turns" once the run has ended). Scroll inside the list.

YOU SHOULD SEE: one row per tool call. In this run: `1.2 read_file docs/features/f1.md`, `1.3 read_file docs/HANDBOOK.md`, `1.4 list_dir app`, then `1.5 read_file ARCHITECTURE.md`, highlighted, with "agent read ARCHITECTURE.md" next to it. The code reads follow: `app/api/issues.py`, `app/services/issues.py`, and more. At turn 1.13 the agent starts writing code with `apply_patch`.

LOOK AT: the highlighted row 1.5 and the matching second note under "Harness decisions": "agent read ARCHITECTURE.md".

WHAT IT MEANS: a highlighted row is a read of a file the layer placed. The layer adds a note the first time the agent reads each such file. A note never blocks the agent. In this run the agent read the architecture document the map points to, and did not open the skill. A file read through a shell command, such as `cat`, does not appear.

✓ CHECKPOINT
At least one "agent read …" note under "Harness decisions".

IF IT LOOKS DIFFERENT
- Only the "placed" note → the agent read nothing the layer placed. That is a result: say so.
- The list does not scroll → click inside it first, or click "Expand".

◆ INSTRUCTOR NOTE
Say: "Watch which files it opens before it writes code." In this run it read the architecture document, not the skill. If a run opens the skill, the note reads "agent read skill add-endpoint (tier 2)". If it does not, say that the map offered the skill and the agent did not need it.

Step 8 Read the gate area

WHERE: browser, screen 1, right panel

TYPE: Scroll the right panel down to "Gate".

YOU SHOULD SEE: "No gate." and then the referee's checks as they run: "House rule 1 probes 53/53", "House rule 2 probes 12/12", "House rule 3 operations 26/26", "House rule 4 probes 6/6", "House rule 5 probes 12/12", "House rule 6 import-linter", "House rule 7 alembic upgrade head", "Existing tests (regression)", "hidden tests", "Files outside the allowed paths (diff_scope)", "Flaky tests (flakiness), run 5/5", "Session continuity (continuity)", "board".

LOOK AT: "No gate." and the first label, "House rule 1 probes 53/53".

WHAT IT MEANS: Section 1 has no verification layer, so nothing checks the agent's "done" before it stops. The labels after "No gate." are the referee's checks after the run. House rule 1 runs first, because it decides the primary verdict.

✓ CHECKPOINT
"No gate." is shown and, after the run ends, "House rule 1 probes" comes first.

IF IT LOOKS DIFFERENT
- A "Gate refused 'done'" line → you opened a Section 4: Verification run. Press H and open the Section 1 card.

◆ INSTRUCTOR NOTE
Say: "No gate here. The agent decides when it is done. Section 4 adds the gate."

Step 9 Read "Agent said"

WHERE: browser, screen 1, right panel

TYPE: Look below "Gate".

YOU SHOULD SEE: "Agent said: ✓ done (build) · ✓ done (test phase)". The cost line above reads "cost $0.018 / $0.20 · ✓ run completed · 38 turns".

LOOK AT: the "Agent said:" pills, one per phase.

WHAT IT MEANS: this is the agent's own claim, per phase. After it said done in the build phase, the agent had a short test phase in which it may only add tests, and it said done again; then the run ended. A run that claims in the build phase only reads just "Agent said: ✓ done". The claim is not the score; the referee decides.

✓ CHECKPOINT
"Agent said:" shows "done", "not done" or "no claim", with "(build)" and "(test phase)" when the agent also claimed in the test phase.

IF IT LOOKS DIFFERENT

- "– no claim" → the agent stopped without saying done (turn limit or time limit). The referee still scores what it shipped.

◆ INSTRUCTOR NOTE

If it says "no claim": "The agent never said done. The referee scores the code anyway. A missing claim is not a House rule 1 failure."

Step 10 Read the primary verdict

WHERE: browser, screen 1, right panel

TYPE: Look at "Referee", next to "Agent said".

YOU SHOULD SEE: "✓ primary verdict ready: PASS", then "House rule 1: Tenant isolation, new violations 0".

LOOK AT: the referee pill.

WHAT IT MEANS: the referee called every endpoint as an Acme user with Globex ids and expected 404. PASS means no new endpoint shows another org's data. The legacy `/v1/issues` leak is in the starting code, so it does not count as new.

✓ CHECKPOINT

The pill says "primary verdict ready", with PASS or FAIL.

IF IT LOOKS DIFFERENT

- "⏳ referee pending" → the agent is still running or the referee is starting; wait.
- "primary verdict ready: FAIL" → a real result. Go to screen 3 to see which check failed.

◆ INSTRUCTOR NOTE

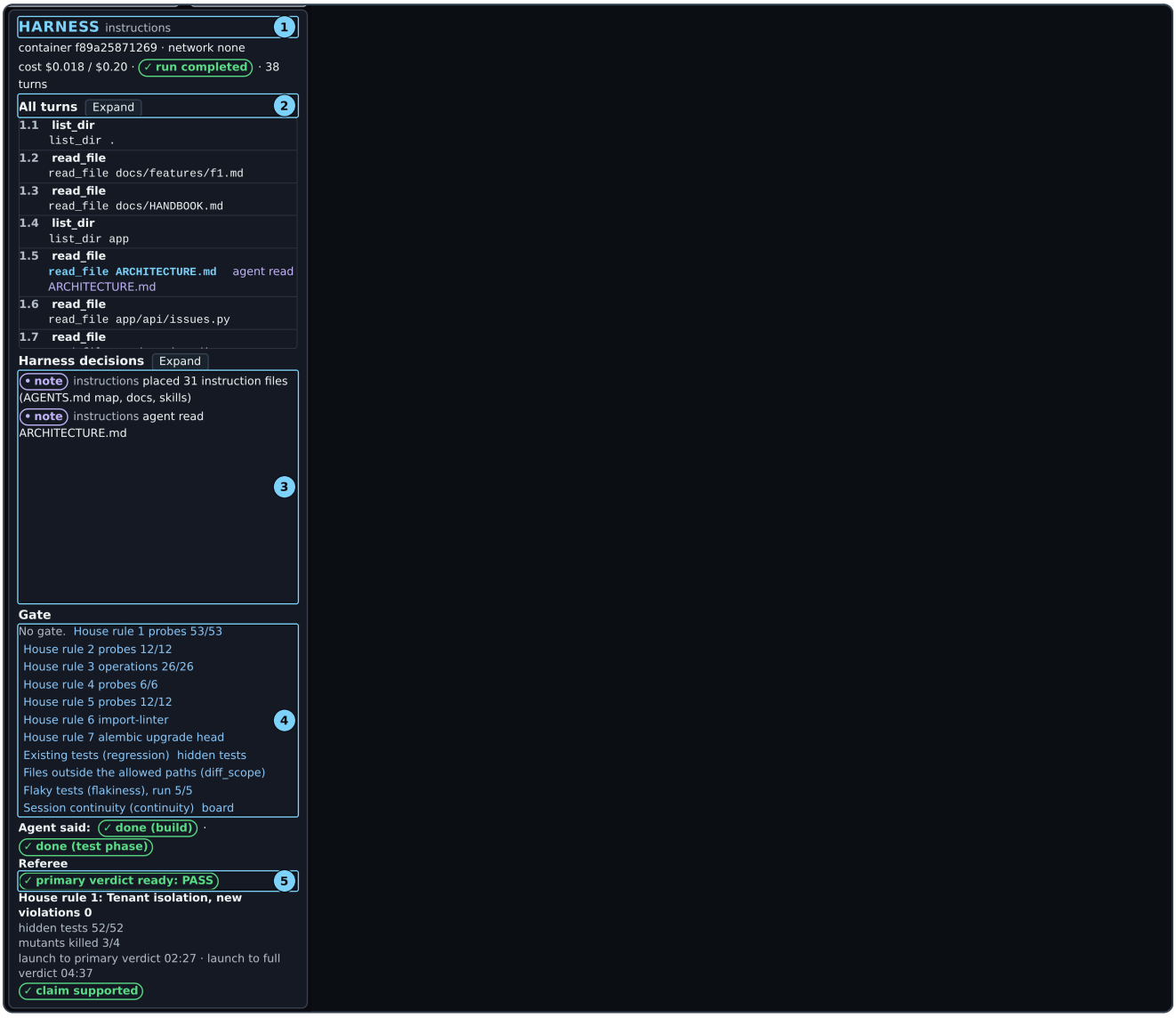
On PASS: "No new tenant leak. One run shows the layer ran, not that it helped." On FAIL: "A leak shipped with the map in place. That is a result." Show screen 3 next. Do not rerun to get a PASS.

Step 11 Read the full verdict

WHERE: browser, screen 1, right panel

TYPE: Wait until "secondary checks running" is replaced by the full verdict.

YOU SHOULD SEE: "hidden tests 52/52", "mutants killed 3/4", "launch to primary verdict 02:27 · launch to full verdict 04:37", and the pill "✓ claim supported".



The harness panel at the end: two notes, no gate, the claim and the referee.

1. Harness arm line: its layers
2. Turns list
3. Harness decisions
4. Gate list
5. Referee pill (harness)

LOOK AT: the hidden tests count and "claim supported".

WHAT IT MEANS: hidden tests are tests the agent never sees. A mutant is a copy of the code with one small bug planted on purpose; a good test catches (kills) it. "Claim supported" means the agent's "done" matches the referee's result. The console counts its times from the run's first event, which came 8 seconds after you pressed Enter, so the times from the moment you launch are 2 min 35 s and 4 min 45 s.

✓ CHECKPOINT

Both times are shown, and "claim supported" or "claim not supported" appears.

IF IT LOOKS DIFFERENT

- "claim not supported" → the agent said done and the referee disagrees. Say so; that is the Section 4 story.

◆ INSTRUCTOR NOTE

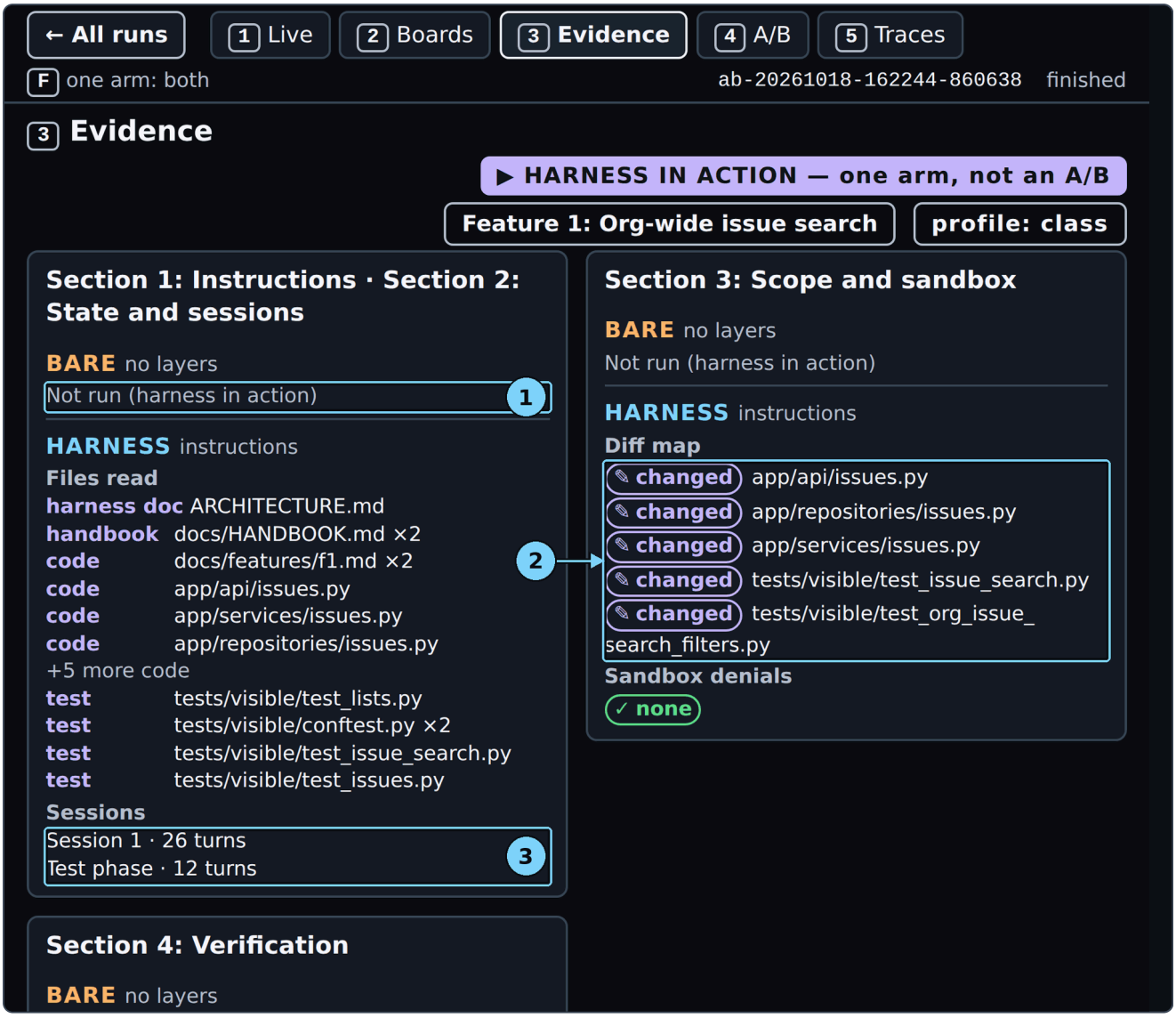
Say: "This is one harness arm on one run. It tells us the layer ran; it cannot tell us the layer helped. That needs the A/B."

Step 12 Open screen 3, files read and sessions

WHERE: browser, screen 3 (press 3), left column

TYPE: Press 3.

YOU SHOULD SEE: the column "Section 1: Instructions · Section 2: State and sessions". Under BARE: "Not run (harness in action)". Under HARNESS: "Files read" by group, then "Sessions": "Session 1 · 26 turns" and "Test phase · 12 turns".



Screen 3: what the agent read, its sessions, and what it changed.

1. Bare arm: Not run (harness in action)
2. Diff map (harness)
3. Sessions, including the test phase

LOOK AT: the first rows of "Files read": "harness doc ARCHITECTURE.md" and "handbook docs/HANDBOOK.md ×2". There is no skill row.

WHAT IT MEANS: "Files read" lists only `read_file` calls, grouped as harness doc, handbook, code and test. This agent read the architecture document from the map and also the long handbook that both arms have. The test phase is the short second session after the build.

✓ CHECKPOINT

At least one "harness doc" or "skill" row under "Files read".

IF IT LOOKS DIFFERENT

- "Nothing read yet." → the run has not started its first turn; go back to screen 1.

◆ INSTRUCTOR NOTE

Say: "This list explains a result; it does not score it. The score is what the referee finds in the shipped code."

Step 13 Screen 3, what the agent changed

WHERE: browser, screen 3, middle column "Section 3: Scope and sandbox"

TYPE: Look at the middle column.

YOU SHOULD SEE: "Diff map" with five "✎ changed" rows: `app/api/issues.py`, `app/repositories/issues.py`, `app/services/issues.py`, `tests/visible/test_issue_search.py`, `tests/visible/test_org_issue_search_filters.py`. Below: "Sandbox denials ✓ none".

LOOK AT: that every changed file is an issues API, service or repository file, or a test.

WHAT IT MEANS: the agent changed only the files the ticket needs. Section 1 has no scope layer; the referee checks the paths after the run.

✓ CHECKPOINT

"Sandbox denials ✓ none".

IF IT LOOKS DIFFERENT

- A "⊘ denied" row → the sandbox refused a command, for example a network call. Read the reason code on the row.

◆ INSTRUCTOR NOTE

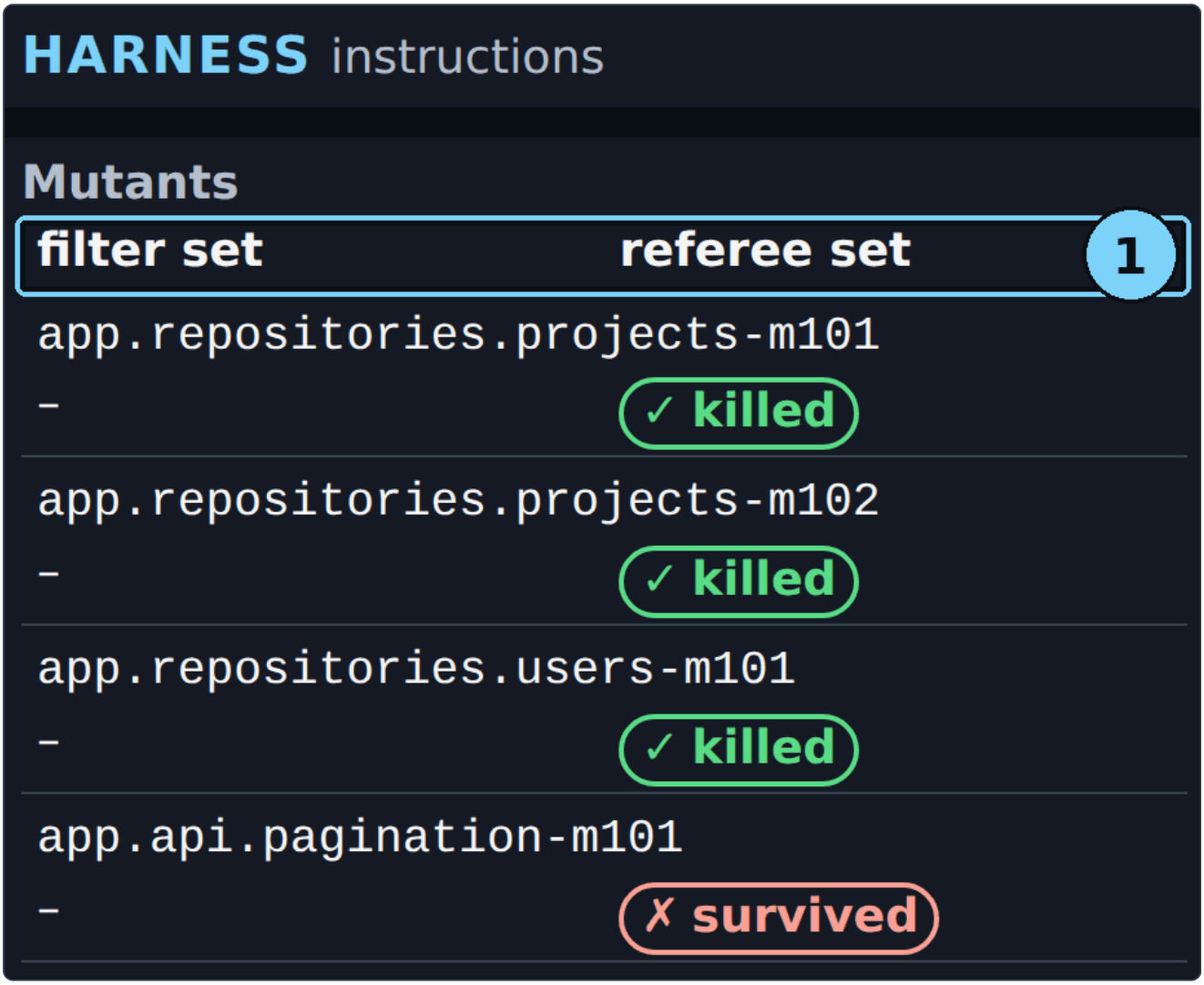
Keep this short; Section 3 is about this column.

Step 14 Screen 3, the referee checks and the reviewer

WHERE: browser, screen 3, right column "Section 4: Verification"

TYPE: Look at the right column, then scroll down to "Mutants".

YOU SHOULD SEE: one row per referee check, each with "✓ 0 new": House rule 1 to House rule 7, "Existing tests (regression)", "hidden tests", "Files outside the allowed paths (diff_scope)", "Flaky tests (flakiness)", "Session continuity (continuity)", "board check". Then "✓ reviewer accept tenant risk none · The new GET /issues endpoint implements org-scoped, cursor-paginated issue search with the required filters and validation. ...". Then the mutants table: three "✓ killed", and `app.api.pagination-m101` "✗ survived".



The mutants table. Section 1 has no test filters, so the filter set column shows a dash.

1 Mutants (harness): filter set and referee set

LOOK AT: the House rule 1 row, then the reviewer line.

WHAT IT MEANS: the seven house rules are House rule 1: Tenant isolation, House rule 2: Roles, House rule 3: API conventions, House rule 4: Safe retries, House rule 5: Audit log, House rule 6: Layering and House rule 7: Migrations. The referee is a program: it probes, runs tests and counts. The reviewer is a model that reads the change blinded (it does not know which arm wrote it) and gives its own verdict and reason. Here both agree. One mutant in the pagination code survived: the referee's test run did not catch that planted bug.

✓ CHECKPOINT

The House rule 1 row reads "✓ 0 new".

IF IT LOOKS DIFFERENT

- A row with "✗ … new" → that check found new violations. Hover the row for the full name and the number of probes.
- No reviewer line → the review is still running, or it hit its cost cap.

◆ INSTRUCTOR NOTE

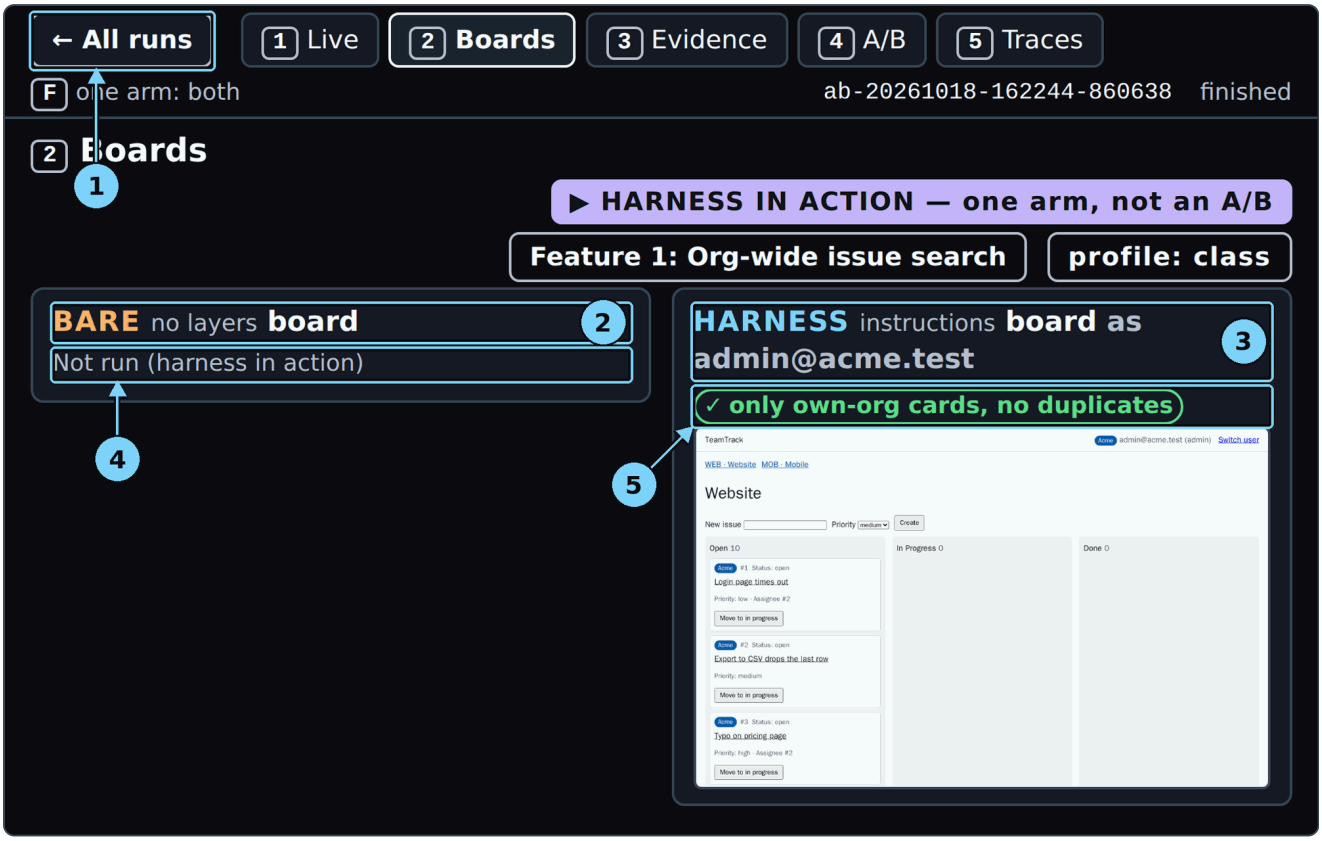
If the reviewer rejects: "The reviewer and the referee disagree. The referee's score stands; the reviewer's reason is a lead for code review." Point at "tenant risk". In this run it accepted, with tenant risk none.

Step 15 Open screen 2, the board

WHERE: browser, screen 2 (press 2)

TYPE: Press 2.

YOU SHOULD SEE: BARE "Not run (harness in action)". HARNESS "board as admin@acme.test", the pill "✓ only own-org cards, no duplicates", and the TeamTrack board with Acme cards only.



Screen 2: no card from another org on the board.

1. ← All runs: back to the run list
2. Bare board heading
3. Harness board heading: signed in as one user
4. Bare arm: Not run (harness in action)
5. Harness board marks

LOOK AT: the pill above the screenshot.

WHAT IT MEANS: the console takes a board screenshot as an Acme user every 5 seconds. A card from another org would carry a "⚠ other org" mark. Feature 1 changes no board pixels, so the board shows one frame.

✓ CHECKPOINT

"only own-org cards, no duplicates".

IF IT LOOKS DIFFERENT

- "No board screenshot yet" → the preview container has not taken one; wait 5 seconds.

◆ INSTRUCTOR NOTE

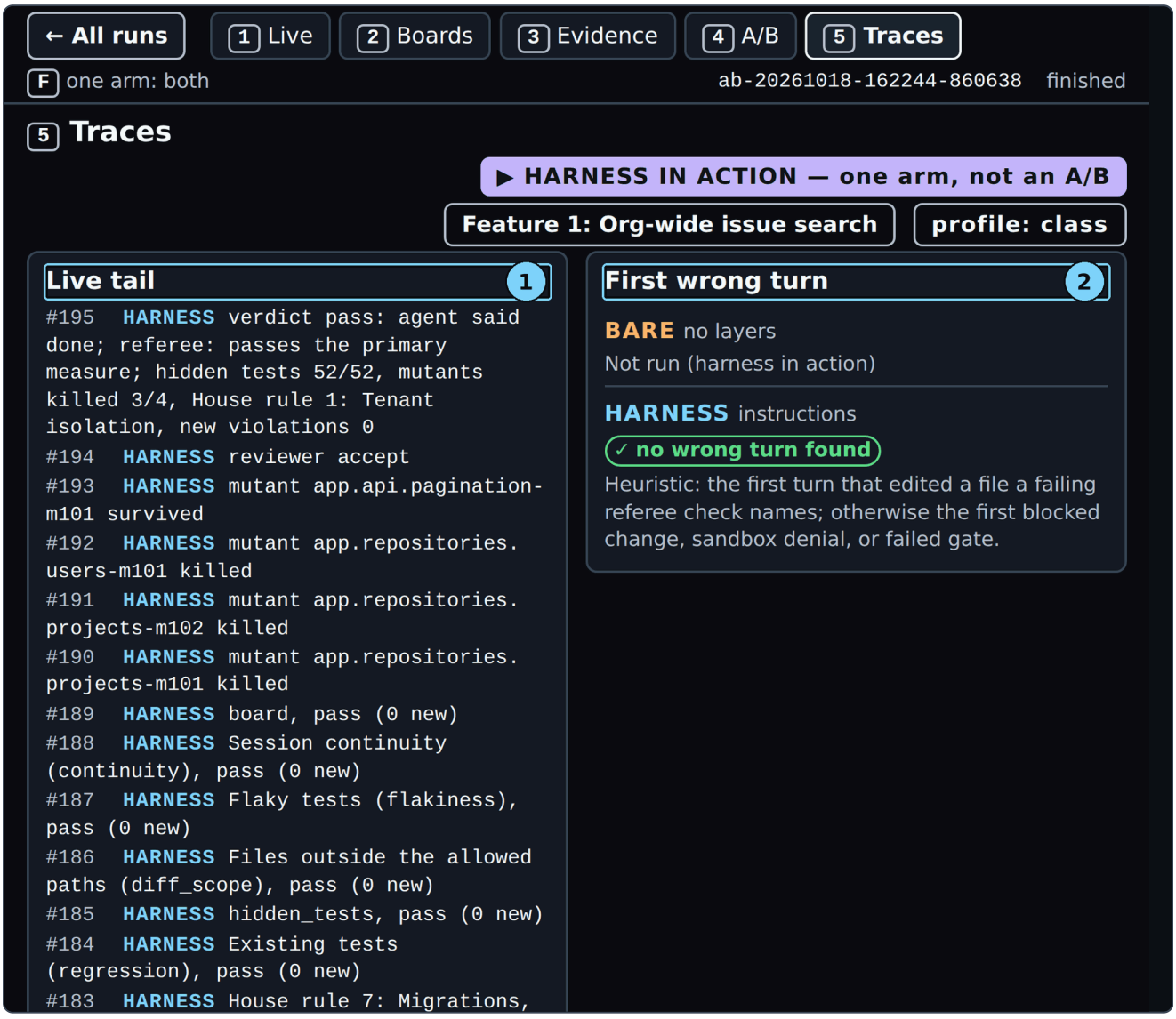
Say: "This feature changes no board pixels, so there is one frame. The board matters in later sections."

Step 16 Open screen 5, traces

WHERE: browser, screen 5 (press 5)

TYPE: Press 5.

YOU SHOULD SEE: "Live tail", whose newest line is "#195 HARNESS verdict pass: agent said done; referee: passes the primary measure; ...". "First wrong turn" shows "✓ no wrong turn found". "LangSmith" shows an agent link and a referee link.



Screen 5: where you look for the first wrong turn.

1. Live tail
2. First wrong turn

LOOK AT: "First wrong turn".

WHAT IT MEANS: the console marks the first turn that edited a file a failing check names. With no failing check, there is none. The LangSmith links open every model call of the agent and of the referee's review.

✓ CHECKPOINT

Two LangSmith links under HARNESS: "agent" and "referee".

IF IT LOOKS DIFFERENT

- "No trace link yet." → the links arrive at the start of the run and of the referee; reload the page.

◆ INSTRUCTOR NOTE

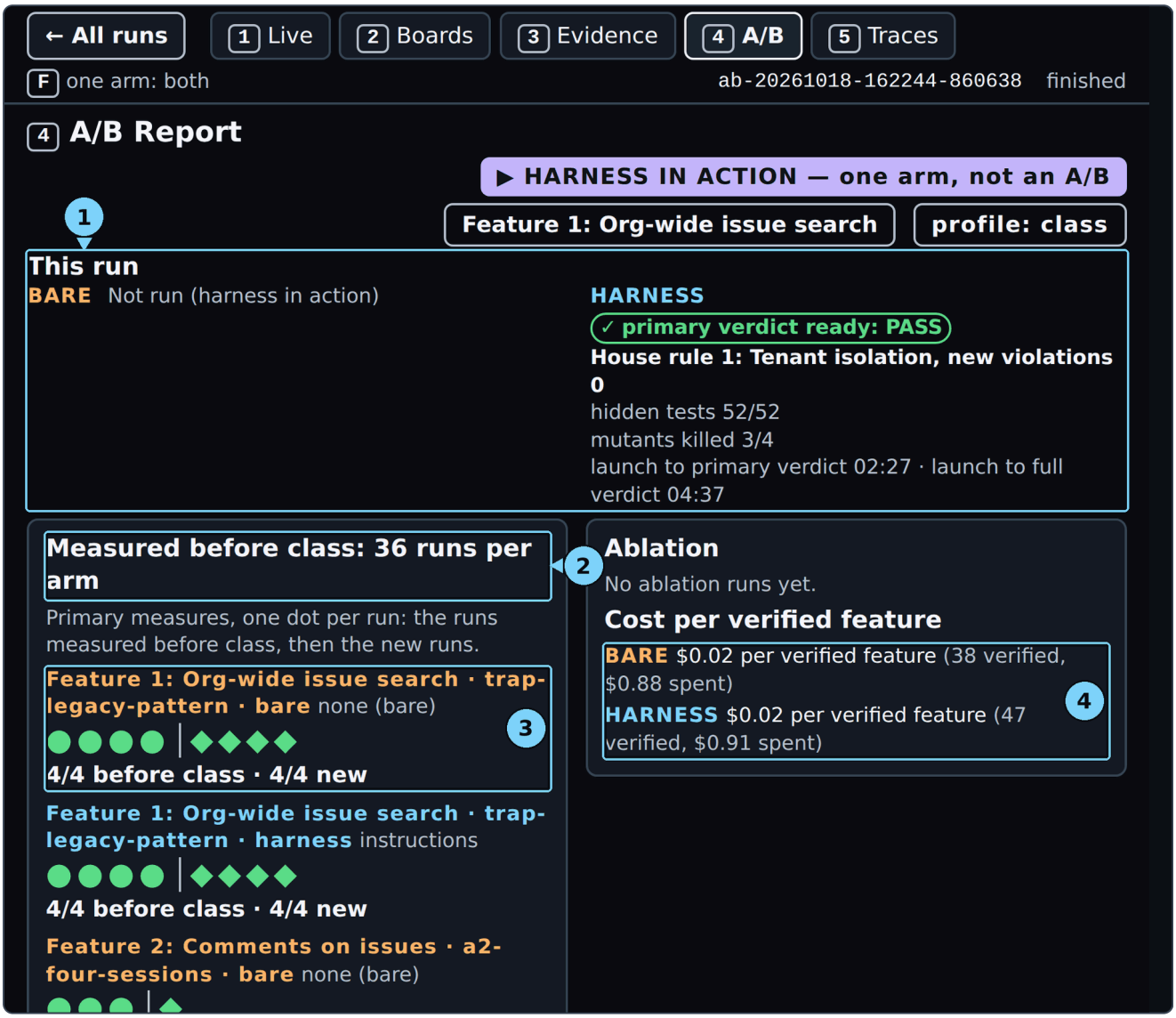
On a FAIL, this screen names the turn to open. Click the agent link and go to that turn.

Step 17 Open screen 4, the comparison from before class

WHERE: browser, screen 4 (press 4)

TYPE: Press 4. Scroll the left panel to see all features.

YOU SHOULD SEE: "This run" at the top: BARE "Not run (harness in action)" and HARNESS with the same referee lines as screen 1. Below, "Measured before class: 36 runs per arm", one dot per run. The two Feature 1 rows (`trap-legacy-pattern`, the ticket with the legacy handler) each show four filled dots and "4/4 before class".



Screen 4: the comparison comes from the runs before class, not from this run.

1. This run
2. Measured before class
3. Dots per run
4. Cost per verified feature

LOOK AT: the two Feature 1 rows: 4/4 for the bare arm and 4/4 for the harness arm.

WHAT IT MEANS: this run is not a dot. The dots are the runs before class. On this ticket both arms passed in every run: no measurable difference on this task with this model.

✓ CHECKPOINT

"This run" matches screen 1, and both Feature 1 rows are shown.

IF IT LOOKS DIFFERENT

- No dots → the console has no before-class records to read. Check that `docs/evidence/scores/` exists in the repository (`exports/` holds only your own new runs). The run is still valid; use the "What we measured" slide.

◆ INSTRUCTOR NOTE

Say: "No measurable difference on this task with this model. Here the assumption did not hold: the bare arm applied House rule 1 from the handbook."

Step 18 Read the final lines in Terminal 2

WHERE: Terminal 2

TYPE: Nothing. The command has ended and the prompt is back.

YOU SHOULD SEE: the A/B line, the harness arm's line and the `Cost:` line:

OUTPUT

```
A/B ab-20261018-162244-860638 (Feature 1: Org-wide issue search, layers=['instructions'],
profile=class, harness in action)
  harness  completed         agent said done · House rule 1: Tenant isolation, new violations: 0 ·
primary measure passed · hidden tests 52 of 52 · mutants killed 3 of 4 · reviewer accept · cost
$0.0178
Cost: $0.020302 (this command: gpt-5.6-luna $0.002464, gpt-6-luna $0.017838; session $0.020302 of
$2.50; build $0.020302 of $15.00 ($14.979698 left))
```

LOOK AT: the arm line: "completed", "agent said: done (build) · done (test phase)", "House rule 1: Tenant isolation, new violations: 0", "primary measure passed". Then the first figure of the `Cost:` line and the session figure.

WHAT IT MEANS: the arm line repeats screen 1 in plain words. The arm cost ($0.0178) is the agent alone. The `Cost:` line is the whole command, agent plus review: $0.020302. The session figure is the running total against the class cap of $2.50. The command exits with 0 when the arm completed.

✓ CHECKPOINT

The arm line says "completed" and the prompt is back.

IF IT LOOKS DIFFERENT

- `ABORTED: timeout` or `ABORTED: cap` → the time limit or a cost cap. Keep the run and read its verdict as usual; the exit code is 2.
- `ERROR` → the exit code is 1 and make prints an error line. Check Docker, then see the troubleshooting table.

> **◆ INSTRUCTOR NOTE**
> Read the session figure aloud: "$0.020302 of $2.50 so far."

Result summary

Item	This run
Agent said	done (build) · done (test phase)
House rule 1, new violations	0 (PASS)
Hidden tests	52 of 52
Mutants killed	3 of 4
Reviewer	accept, tenant risk none
Claim	supported
Turns	26 build, 12 test phase
Files changed	5

Time and cost

The run ran alone on the machine.

Moment	This run, from launch	Console shows	Range, two runs, 2026-10-18
Primary verdict	2 min 35 s	02:27	1:45 to 2:35
Full verdict	4 min 45 s	04:37	3:37 to 4:45
Command ended	4 min 45 s		
Cost of the command	$0.020302		$0.020 to $0.021

The console counts from the run's first event, 8 seconds after launch. The command cost $0.020302: $0.017838 for the agent and $0.002464 for the review. The cap is $0.24.

The range covers this run and a second run of the same command by a reviewer on a fresh clone, both on 2026-10-18 and each alone on the machine (`ab-20261018-175605-9471af`). That run has no recorded launch time, so its times count from its first event plus this run's 8 s. Running two demos at once makes both slower.

What to say in class

- "The map is short and points outward. Watch which files the agent opens before it writes code: here, the architecture document, not the skill."
- "No new tenant leak in this run. One run shows the layer ran, not that it helped."

- "The referee scores; the blinded reviewer gives an opinion. Here they agree."
- "Before class both arms passed on this ticket every time: no measurable difference on this task with this model."
- "What would have to be true of a model or a ticket for the map to earn its place?"

CHAPTER 2

State and sessions

When a coding agent's session ends, the next one starts with an empty context. If the first session left half a feature and no note, the second one has to guess. It may write a second migration for a table that already exists, or build again what was already there. In this chapter you run Feature 2: Comments on issues with the harness's state layer on. Session 1 gets only 12 turns, so it stops partway through the ticket. The harness writes the handoff at the stop. A fresh 38-turn session then reads those files first and carries on. Afterwards the referee answers one question: did session 2 break or redo session 1's work?

The numbers and screenshots in this chapter come from the run `ab-20261018-162745-ccba38`, started on 2026-10-18 at 16:27 UTC. No other run shared the machine with it.

> Your output will differ: run times, ids, and the agent's wording change on every run.

Step 1 Go back to the run list

WHERE: Terminal 1 and the browser, run list

TYPE: Click "← All runs", or press H.

YOU SHOULD SEE: the run list, with "Runs", "Newest first.", and one card for each run from the earlier chapters.

LOOK AT: Terminal 1. It is still running `make console`, with no error.

WHAT IT MEANS: the console serves every run in the run folder, whether live or finished. You start the new run from Terminal 2, and the console finds it without any extra step.

✓ CHECKPOINT
The run list loads at the address Terminal 1 printed (http://127.0.0.1:8765/ by default).

IF IT LOOKS DIFFERENT

- The page does not load → Terminal 1 stopped. Run `make console` again in Terminal 1.
- `address already in use` → another program holds the port. Run `make console CONSOLE_PORT=8766` and open that port.

◆ INSTRUCTOR NOTE
Say: "Section 1: Instructions ran one session. This section cuts the work into two sessions on purpose and watches what crosses the gap."

Step 2 Start the run

WHERE: Terminal 2

TYPE:

TYPE THIS ⧉ COPY

```
make show-2 SESSION_BUDGET=12,38 PURPOSE=pilot
```

YOU SHOULD SEE: the make target prints the command it runs. After that, Terminal 2 stays quiet for about seven minutes.

OUTPUT

```
uv run shipharness ab --section demo-2 --feature comments --arms harness  --session-budget=12,38 --
purpose pilot \
  --profile class
```

The `A/B ab-…` line and the result lines come only at the end (Step 21).

LOOK AT: `--arms harness`, `--session-budget=12,38`, and `--profile class` in the printed command.

WHAT IT MEANS: `SESSION_BUDGET=12,38` sets the session turn budget (the fixed number of turns a session gets before the runner stops it). Session 1 gets 12 turns and session 2 gets 38. Without the setting, session 1 gets the ticket's own 20 turns. The command runs the harness arm alone, with two harness layers: instructions (from Section 1) and state (this section). In class, drop `PURPOSE=pilot` and put the class day in `.env` instead (Part 0).

✓ CHECKPOINT

The printed command contains `--arms harness`, `--session-budget=12,38`, and `--profile class`.

IF IT LOOKS DIFFERENT

- `x SHIPHARNESS_CLASS_DATE is not set`, exit 2 → put the class day in `.env`, or add `PURPOSE=pilot` outside class.
- `x OpenAI refused the key`, exit 2 → fix the key in `.env`, then run `make doctor`.
- The arm ends `ERROR` with `docker run failed` → Docker is not running. Run `sudo systemctl start docker`, then `make warm`.

◆ INSTRUCTOR NOTE

Launch at section minute 1, on the start slide. This run took 7 min 12 s from launch to the last line, so it is finished long before the demo block. Say: "This run cuts the agent off after 12 turns. Let's see what the next session finds."

Step 3 Wait for the run to start

WHERE: Terminal 2

TYPE: nothing. Wait about 10 seconds.

YOU SHOULD SEE: Terminal 2 stays quiet. It prints the result only after the referee has finished (Step 21).

LOOK AT: Terminal 2 shows no line that starts with `x`.

WHAT IT MEANS: the runner is starting the container and writing the harness files. The console shows the run from its first event onwards.

✓ CHECKPOINT

Terminal 2 shows no error after 10 seconds.

IF IT LOOKS DIFFERENT

- The command returned to the prompt at once → read the last line. It names the missing key, the missing date, or the Docker problem (Step 2).

◆ INSTRUCTOR NOTE

Go on with the section's story slides. Come back to the console for the demo block, or show the stop live at about minute 1 if the room wants to see it.

Step 4 Reload the run list

WHERE: browser, run list

TYPE: Reload the page (Ctrl+R).

YOU SHOULD SEE: a new card at the top, with the run id and then "Section 2: State and sessions", "Feature 2: Comments on issues", "harness in action", the status ("running" while it runs, "finished" at the end), and the start time.

The run list after the run finished. The Section 2 card is fifth here, because four later runs started after it.

1. Run id: open the run
2. Type: harness in action or A/B
3. Status
4. Start time

LOOK AT: the type "harness in action" and the start time.

WHAT IT MEANS: "harness in action" means a single-arm run, with the harness arm alone. The comparison between the bare arm and the harness arm comes from runs measured before class (Step 19).

✓ CHECKPOINT

The newest card names Feature 2: Comments on issues and says "harness in action".

IF IT LOOKS DIFFERENT

- No new card → wait 10 seconds and reload again. The card appears with the run's first event.
- A card shows "unknown" in every field → the run has only just started. Reload again in a few seconds.

◆ INSTRUCTOR NOTE

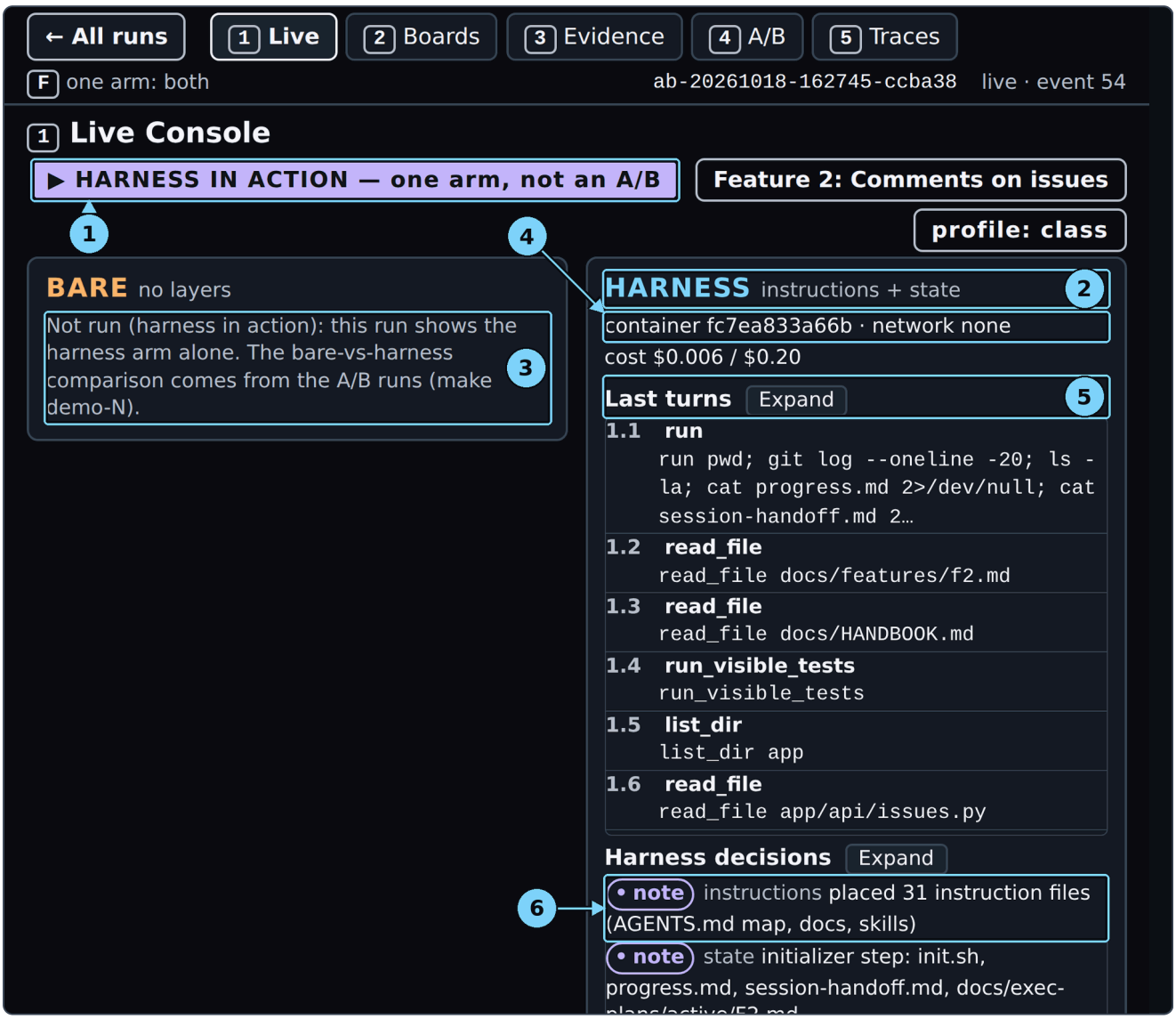
Read the card aloud: "Section 2, comments on issues, the harness arm alone."

Step 5 Open the run on screen 1

WHERE: browser, screen 1 (Live)

TYPE: Click the newest run id, then press 1.

YOU SHOULD SEE: two arm panels. The bare panel says "Not run (harness in action): this run shows the harness arm alone." The harness panel shows "instructions + state", the container line, the cost against the $0.20 cap, "Last turns", and "Harness decisions".



Screen 1 during session 1, showing the first turns and the two layer notes.

1. HARNESS IN ACTION badge, or the feature
2. Harness arm line: its layers
3. Bare arm: Not run (harness in action)
4. Container line
5. Last turns
6. Harness decisions: the layer notes

LOOK AT: "Harness decisions". The first line is the instructions layer's "placed 31 instruction files (AGENTS.md map, docs, skills)". The second is the state layer's "initializer step: init.sh, progress.md, session-handoff.md, docs/exec-plans/active/F2.md".

WHAT IT MEANS: the state layer writes four files and commits them before session 1 starts:

- `init.sh` starts the app and runs a basic test.
- `progress.md` records what was done and what comes next, and keeps a session log.
- `session-handoff.md` is the handoff to the next session.
- `docs/exec-plans/active/F2.md` is the exec plan, with the steps and decisions.

The initializer step is plain code, so it makes no model call.

✓ CHECKPOINT
"Harness decisions" contains the initializer step line.

IF IT LOOKS DIFFERENT
- No initializer step line → the run has no state layer. Check that you ran `make show-2`, not `make show-1`.
- "No turns yet." → the container is still starting. Wait a few seconds; the page updates itself.

◆ INSTRUCTOR NOTE
Point at the initializer line. Say: "These four files are the whole state layer. The next session knows only what is in them, or in git."

Step 6 Watch session 1 work

WHERE: browser, screen 1, harness panel, the turn list

TYPE: nothing. Watch the rows 1.1 to 1.12.

YOU SHOULD SEE:

- Turn 1.1 runs `pwd; git log --oneline -20; ls -la; cat progress.md; cat session-handoff.md; cat docs/exec-plans/active/F2.md; bash init.sh`.
- Turns 1.2 to 1.10 read the ticket's spec (`docs/features/f2.md`) and the house rules (`docs/HANDBOOK.md`), run the visible tests, and read the issue code.
- Turn 1.11 patches `progress.md`, `docs/exec-plans/active/F2.md`, and `session-handoff.md`.
- Turn 1.12 commits those three files.

LOOK AT: turn 1.1, then turns 1.11 and 1.12.

WHAT IT MEANS: turn 1.1 is the routine every session starts with, called Getting up to speed: `pwd`, the progress file and the git log, the handoff and the plan, then `bash init.sh`. With three turns left, the runner sends a wrap-up message. The agent uses its last turns to write down what it learned and to commit it. Session 1 changed no product code.

✓ CHECKPOINT
The turns count 1.1, 1.2, and so on, up to 1.12.

IF IT LOOKS DIFFERENT

- The turns go past 1.12 → `SESSION_BUDGET=12,38` was left out. Let it run; the stop comes at turn 20.
- Session 1 ends before 1.12 with `claim_done` → the agent said done in session 1. Read the run as one that never faced the stop.

◆ INSTRUCTOR NOTE

Say: "Twelve turns is not enough for this ticket. This session spends almost all of them reading, then writes down what it found. That is normal for a first session."

Step 7 See session 1 stop

WHERE: browser, screen 1, harness panel, the turn list and "Harness decisions"

TYPE: nothing. The stop came 1 min 10 s into the run.

YOU SHOULD SEE: a new harness decision, "session stop · state · session 1 handoff written to session-handoff.md". After turn 1.12, the turn list shows the marker "Session 1 stopped · ended by turn_budget; handoff session-handoff.md".

Session 1 stops: the state layer writes the handoff that session 2 reads first.

1. Turns list
2. Handoff written to session-handoff.md

LOOK AT: "ended by turn_budget" and "handoff written to session-handoff.md".

WHAT IT MEANS: the runner stopped session 1 because its 12 turns were used up. "turn_budget" is the session turn budget at work. In an A/B run, both arms hit the same stop, which is what lets the course compare them.

✓ **CHECKPOINT**

"Harness decisions" contains the "session stop" line for session 1.

IF IT LOOKS DIFFERENT

- The marker says `claim_not_done` → the agent ended session 1 itself with `finish(claim="not_done")`. Session 2 still runs, so read on.
- No marker at all → the agent said done in session 1, or a cap ended the run. Check Terminal 2 at the end.

◆ INSTRUCTOR NOTE

Say: "This is the gap. Anthropic's team calls it a context reset with a structured handoff: a clean session, plus a file that carries the state across."

Step 8 Read what was written at the stop

WHERE: browser, screen 1, harness panel, "Harness decisions"

TYPE: Click "Expand" next to "Harness decisions".

YOU SHOULD SEE: the three harness decisions so far: the two notes from Step 5 and the session stop from Step 7. The session stop names `session-handoff.md` and `progress.md`.

LOOK AT: the files the stop names. Compare them with the four state files:

State file	Written in session 1 by	What it holds at the stop
`progress.md`	The agent, then the harness	Done, In progress, Next; a log line
`session-handoff.md`	The harness and the agent	Machine summary; the agent's notes
`docs/exec-plans/active/F2.md`	The agent	Steps with times; two decisions
`feature_list.json`	Nobody	Not in this run

WHAT IT MEANS: at the stop, the state layer reads the repository and writes the machine summary into `session-handoff.md`. The summary lists the last commits, the files changed, the visible test result, open TODO markers, and the migration heads. The layer also appends one line to the session log in `progress.md`, then commits both files. The commit message is `harness: session 1 handoff for F2`, where `F2` is Feature 2: Comments on issues. The harness writes these facts whatever the agent did.

The agent's notes sit under "Notes from the agent". In this run they list what it read, what is left ("exactly one migration", the three endpoints, seed comments, the board form), and the house rules to watch. `feature_list.json` belongs to the scope layer, which starts in Section 3: Scope and sandbox, so this run has none.

✓ CHECKPOINT

The session stop decision names `session-handoff.md` and `progress.md`.

IF IT LOOKS DIFFERENT

- No session stop decision → the run has no state layer, or session 1 never stopped (Step 7).

◆ INSTRUCTOR NOTE

Say: "The handoff has two owners. The harness writes the facts it can read from git, and the agent writes notes. If the agent runs out of turns before writing its notes, the facts are still there."

Step 9 See session 2 resume and read the state files first

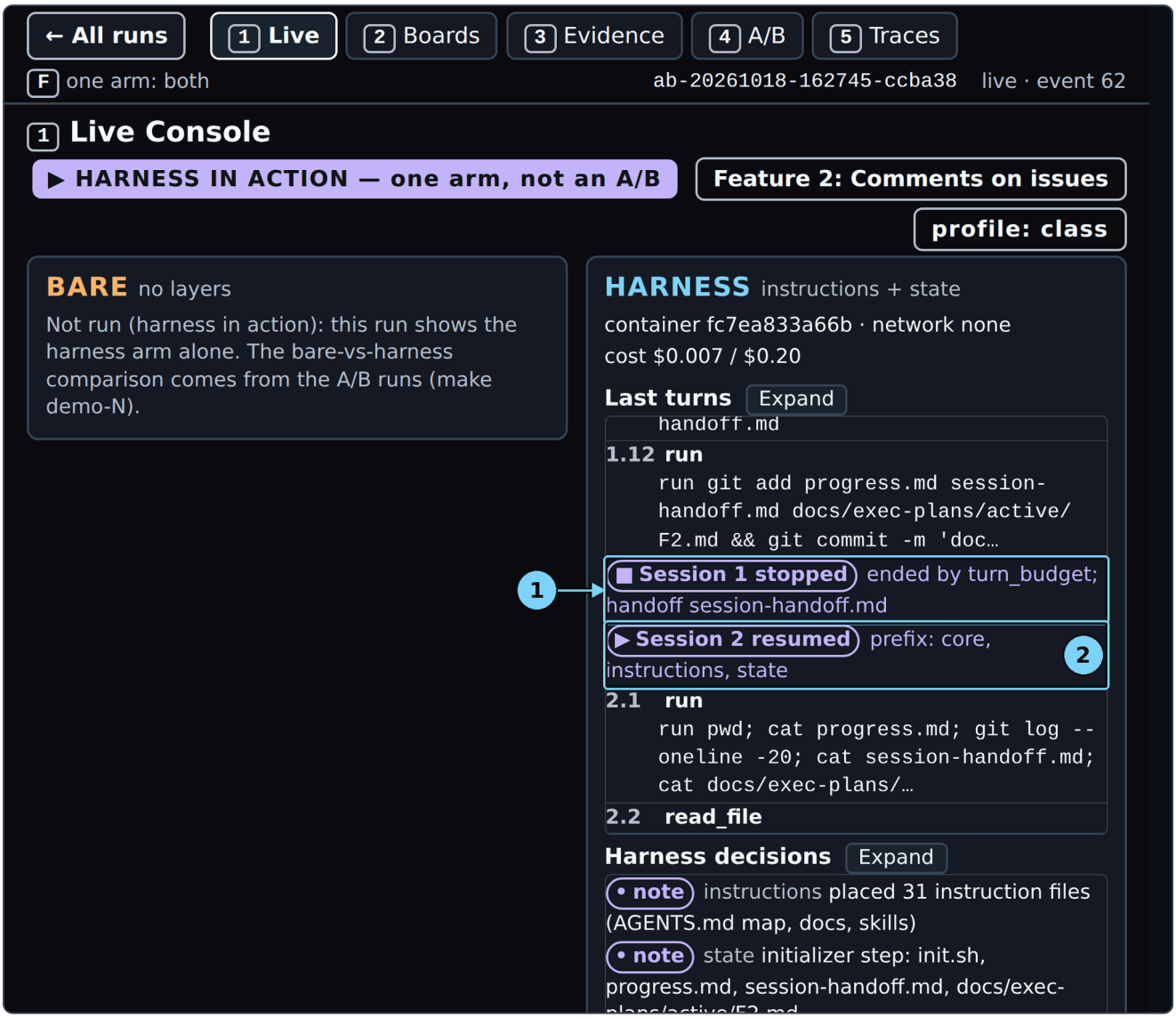
WHERE: browser, screen 1, harness panel, the turn list

TYPE: nothing. Read the rows under the stop marker.

YOU SHOULD SEE: the second marker, "Session 2 resumed · prefix: core, instructions, state", and then turn 2.1. Turn 2.1 began 1 min 12 s into the run. Its call and the start of its output:

OUTPUT

```
2.1 run pwd; cat progress.md; git log --oneline -20; cat session-handoff.md;
    cat docs/exec-plans/active/F2.md; bash init.sh
exit=0
/work
# Progress
...
## Done
- Read feature spec and handbook; initialization and existing test suite verified.
## In progress
- Implement comments API, persistence, migration, ...
```



The key moment: session 1 stops, and session 2's first call reads the state files.

1. Session 1 stopped
2. Session 2 resumed

LOOK AT: the tool target of turn 2.1. It reads `progress.md`, the git log, `session-handoff.md`, and the exec plan, then runs `init.sh`.

WHAT IT MEANS: session 2 has no memory of session 1. Its first call, before it changes any code, reads the files the last session left. It sees session 1's notes: what was done, what is in progress, what comes next. "prefix: core, instructions, state" means the harness added its own resume text after the runner's neutral resume sentence.

✓ CHECKPOINT

Turn 2.1 reads `progress.md` and `session-handoff.md`.

IF IT LOOKS DIFFERENT

- Turn 2.1 reads product files first → the agent skipped the routine. Note it; the referee still checks continuity.

◆ INSTRUCTOR NOTE
This is the moment of the section. Say: "This is a fresh session with an empty context. Its first move is to read the progress file, the git log, and the handoff. That is the Getting up to speed routine from Anthropic's harness."

Step 10 Watch session 2 build the feature

WHERE: browser, screen 1, harness panel, the turn list

TYPE: nothing. Scroll the turn list with the mouse wheel.

YOU SHOULD SEE:

- Turns 2.2 to 2.17 read the spec, the house rules, the issue code, the existing migration, and the add-endpoint skill.
- Turn 2.18 patches `app/db/models.py` and adds `migrations/versions/0002_comments.py`.
- Turns 2.19 to 2.29 add the comments repository, service, and API, and register the router in `app/main.py`. They also patch `app/seed.py`, the board route, and the issue page.
- Turns 2.30 to 2.37 run the migration and the tests, write `tests/visible/test_comments.py`, and fix it.
- Turn 2.38 updates the three state files.

A red tool name marks a call that failed. Turns 2.21 and 2.24 are patches that did not apply, and the agent retried both.

LOOK AT: turn 2.18, the one migration, and the last turn, 2.38.

WHAT IT MEANS: session 2 writes the product code that session 1 only planned. Watch for one migration, not two: two migration heads is the failure this section is about. Session 2 used all 38 turns and never called `finish`, so the run records "agent said: no claim". Its last turns are the wrap-up again.

✓ CHECKPOINT
Session 2 ends at turn 2.38 with a session stop, "session 2 handoff written to session-handoff.md".

IF IT LOOKS DIFFERENT

- The run ends `ABORTED: timeout` → the agent time limit ended the arm. The referee still judges it (Step 21).
- Session 2 ends early with `finish done` → normal too. The agent said done, and the test phase follows.

◆ INSTRUCTOR NOTE
Say: "Session 2 did not start over. It read the plan and built on it, with one migration."

Step 11 See the test phase

WHERE: browser, screen 1, harness panel, the turn list

TYPE: nothing.

YOU SHOULD SEE: after session 2's stop, the rows 3.1 to 3.12. This is the test phase. Turn 3.1 runs the same Getting up to speed routine. Turn 3.10 writes `tests/visible/test_comments_rules.py`, and the agent's own runs of that test file fail in turns 3.11 and 3.12. The run then ends, and the panel header shows "run completed · 62 turns".

LOOK AT: the header line "run completed · 62 turns".

WHAT IT MEANS: after the build sessions, a separate test phase writes more tests in a new session, and it reads the same state files first. Here its new test fails on the agent's code. The referee runs it five times and counts it as failing every time, not as flaky.

✓ CHECKPOINT
The header says "run completed".

IF IT LOOKS DIFFERENT
- "run aborted" → a time or cost cap ended the arm. The verdicts still come.

◆ INSTRUCTOR NOTE
Skip this step in class if time is short; it is the same routine a third time.

Step 12 Read the primary verdict

WHERE: browser, screen 1, harness panel, under "Referee"

TYPE: nothing. The pill appears a few seconds after the run ends.

YOU SHOULD SEE: "Agent said: – no claim", then the pill "✓ primary verdict ready: PASS", then "continuity failure no". "secondary checks running" shows until the full verdict arrives.

LOOK AT: the pill, and the time line under it, "launch to primary verdict".

WHAT IT MEANS: the primary verdict checks continuity. It fails in three cases:

- two migration heads, or a failed fresh upgrade (House rule 7: Migrations);
- a table created by two migrations;
- session 2 removing 30% or more of the lines session 1 added.

PASS means none of the three happened. It does not mean the feature works; the hidden tests answer that. This verdict came 5:18 after launch. The console's own clock starts a few seconds after launch, so it reads a little lower (05:11 here).

✓ CHECKPOINT

The pill reads "primary verdict ready: PASS" or "primary verdict ready: FAIL".

IF IT LOOKS DIFFERENT

- "secondary checks running" stays for about two minutes → that is normal. The other checks run after the primary one (Step 13).

◆ INSTRUCTOR NOTE

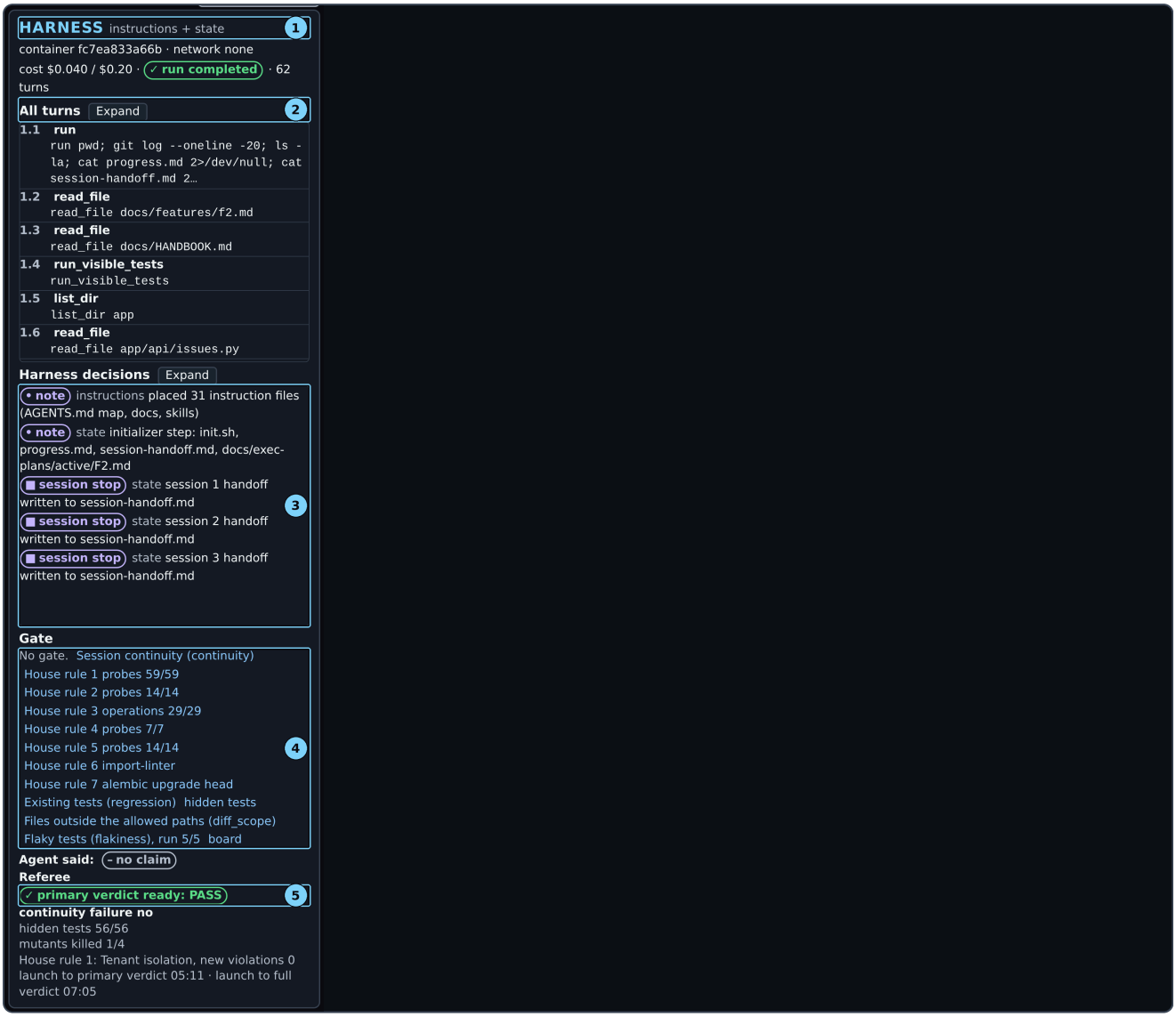
On PASS say: "Session 2 picked up the work. No continuity failure." On FAIL say: "The handoff did not carry the work. Screen 3 shows how." Then add: "Session 1 wrote no product code in this run, so there was little to redo."

Step 13 Read the full verdict

WHERE: browser, screen 1, harness panel, under "Referee"

TYPE: nothing. Wait about two more minutes.

YOU SHOULD SEE: under the pill, "hidden tests 56/56", "mutants killed 1/4", "House rule 1: Tenant isolation, new violations 0", and "launch to primary verdict 05:11 · launch to full verdict 07:05".



The harness panel at the end: three session stops under Harness decisions, then the verdict.

1. Harness arm line: its layers
2. Turns list
3. Harness decisions
4. Gate list
5. Referee pill (harness)

LOOK AT: "hidden tests 56/56", "mutants killed 1/4", and the "launch to full verdict" time. The full verdict came 7:11 after launch; the console's clock, which starts a few seconds later, shows 07:05.

WHAT IT MEANS: the full verdict adds every other check. Hidden tests are tests the agent never saw. A mutant is a copy of the code with one small bug planted on purpose, and good tests catch (kill) it. Here every hidden test passed, and the agent's tests caught 1 of the 4 mutants.

✓ CHECKPOINT

The referee lists hidden tests, mutants killed, and the launch-to-full-verdict time.

IF IT LOOKS DIFFERENT

- Hidden tests 3 of 56 → the arm built nothing. Those 3 tests already pass on the starting code, so the continuity check cannot fail.

◆ INSTRUCTOR NOTE

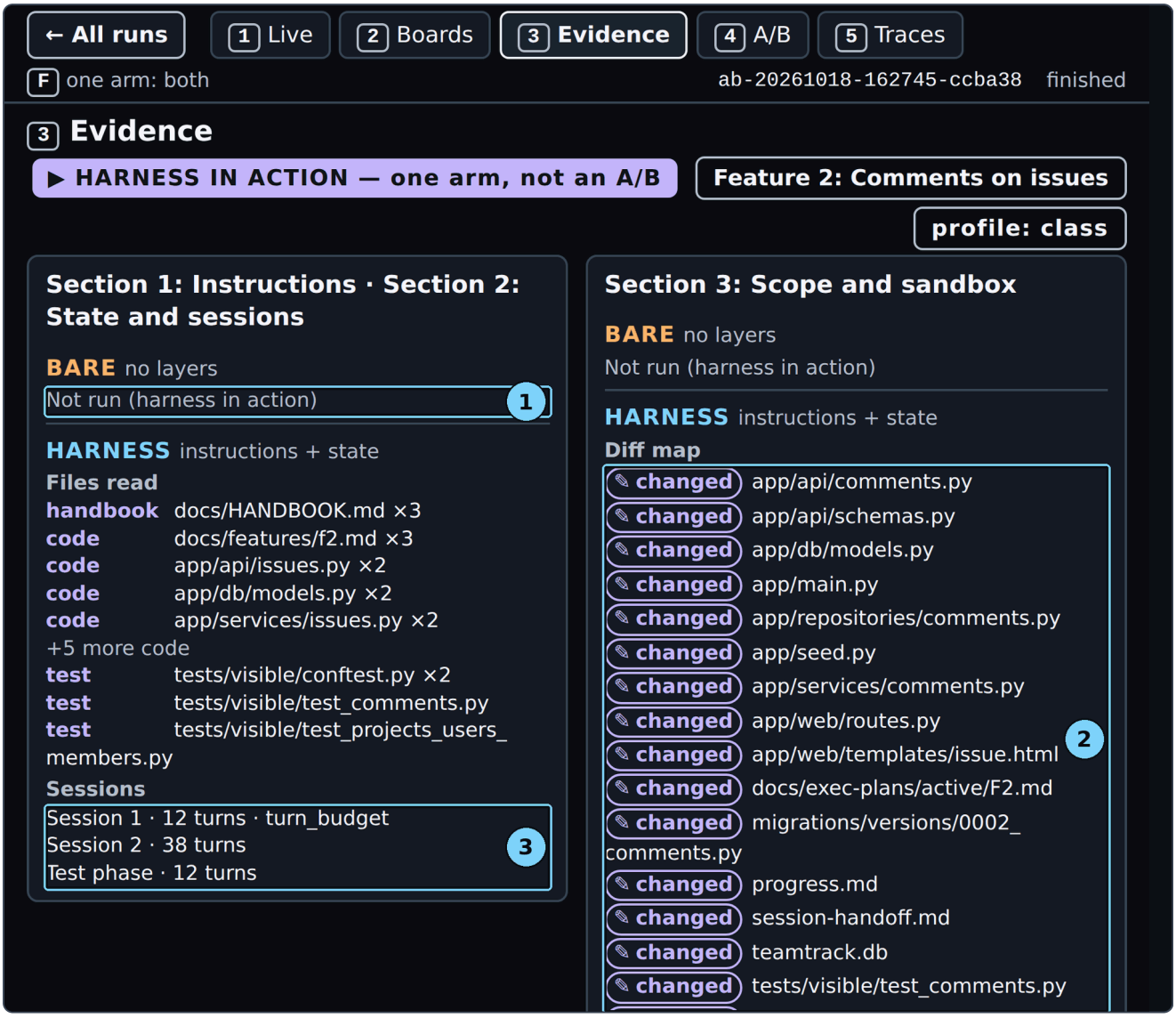
Say: "Two numbers answer two questions. Continuity asks whether session 2 broke session 1's work. The hidden tests ask whether the feature works."

Step 14 Open screen 3 and read the sessions

WHERE: browser, screen 3 (Evidence), left panel

TYPE: Press 3.

YOU SHOULD SEE: under "Sessions", the rows "Session 1 · 12 turns · turn_budget", "Session 2 · 38 turns", and "Test phase · 12 turns".



Screen 3: two build sessions and the test phase, and the files the agent changed.

1. Bare arm: Not run (harness in action)
2. Diff map (harness)
3. Sessions, including the test phase

LOOK AT: each session row, and whether any row has a "↺ redone" mark under it.

WHAT IT MEANS: "turn_budget" is the stop from Step 7. The console prints the end reason only on the first row. "Test phase" is the session after the build. A "↺ redone" mark names a file that a later session changed again. The state files never count as redone work, because they are meant to change every session; the referee leaves them out too. Those files are `progress.md`, `session-handoff.md`, the exec plan, and `feature_list.json`. In this run no row has a redone mark.

✓ CHECKPOINT

Three rows: Session 1, Session 2, and Test phase. None has a "↺ redone" mark.

IF IT LOOKS DIFFERENT

- Only one row → session 1 never stopped (Step 7).
- A row reads "Session 3" instead of "Test phase" → the console is older than this guide. Update the repository and restart `make console`.

◆ INSTRUCTOR NOTE

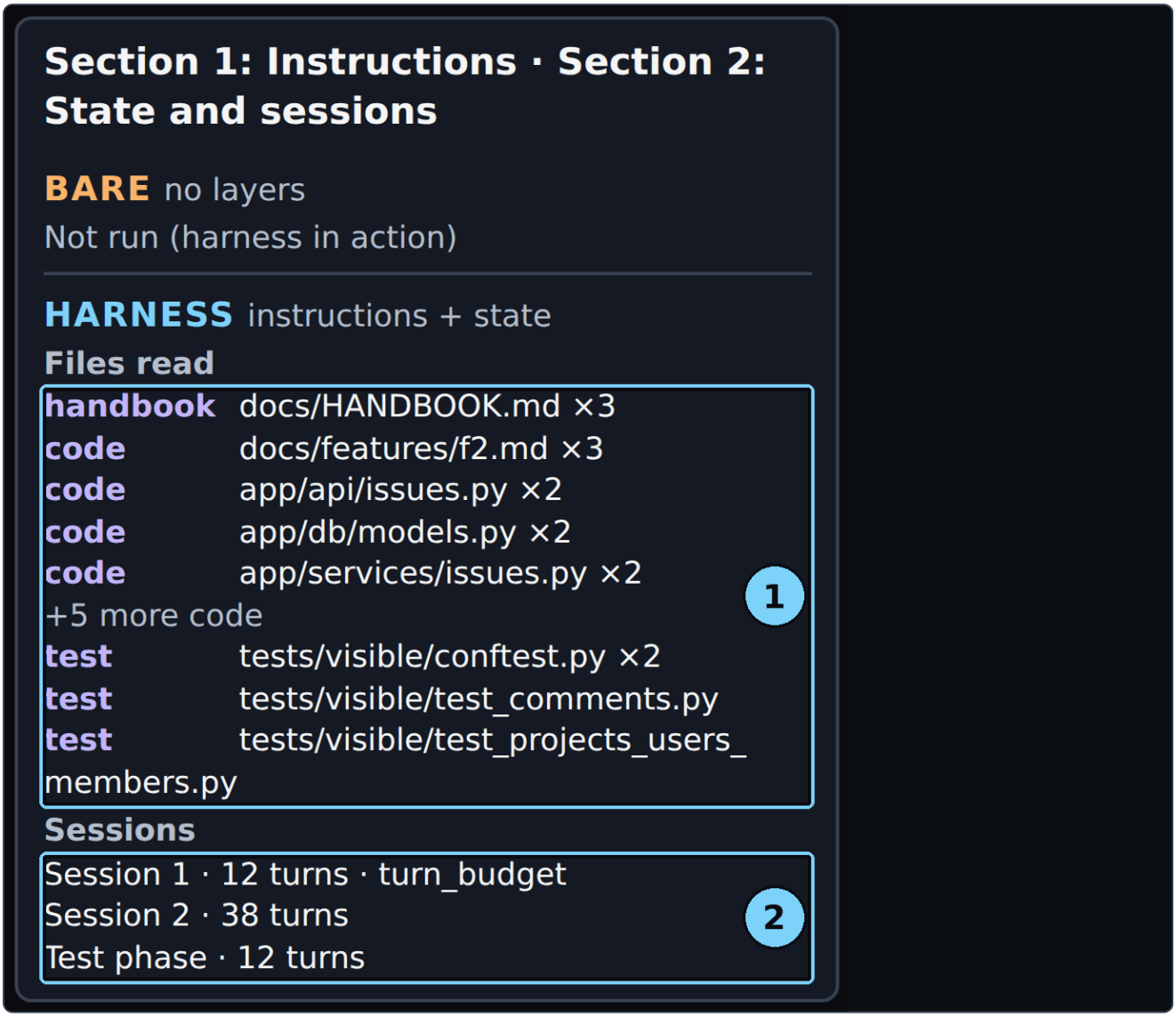
Say: "Twelve turns, then a fresh session with 38. No product file was changed by two sessions."

Step 15 Read the files the agent opened

WHERE: browser, screen 3, left panel, "Files read"

TYPE: nothing. The list is above "Sessions".

YOU SHOULD SEE: the files the agent opened with its file tools, grouped by kind. In this run: "handbook docs/HANDBOOK.md ×3", "code docs/features/f2.md ×3", `app/api/issues.py` ×2, `app/db/models.py` ×2, `app/services/issues.py` ×2, "+5 more code", and three test files.



Files read and Sessions for the harness arm.

1. Files read
2. Sessions

LOOK AT: "×3" after `docs/HANDBOOK.md` and `docs/features/f2.md`.

WHAT IT MEANS: "×3" means each of the three sessions opened the house rules and the spec. The state files are not in this list. The agent read them with `cat` inside a `run` call (turns 1.1, 2.1, and 3.1), and this list shows only files opened with the file tools. Screen 1 is where you see the state files being read.

✓ CHECKPOINT

`docs/HANDBOOK.md` and `docs/features/f2.md` each show "×3".

IF IT LOOKS DIFFERENT

- You expected `progress.md` here → see "What it means" above. Show turn 2.1 on screen 1 instead.

◆ INSTRUCTOR NOTE

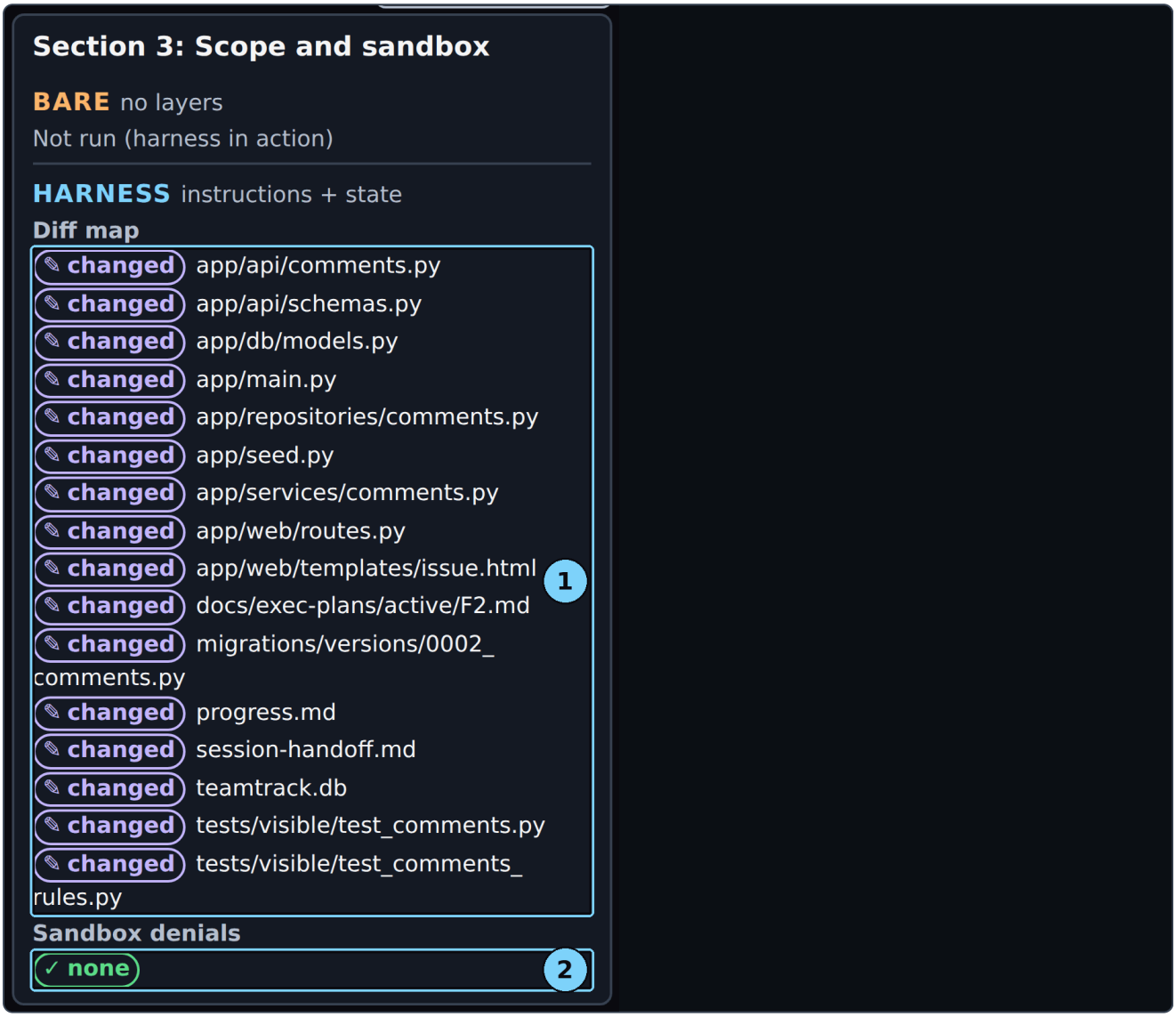
Say: "Every session read the rules and the spec again. That costs turns. The state files tell it where to look; they do not replace the reading."

Step 16 Read the diff map

WHERE: browser, screen 3, middle panel, "Diff map"

TYPE: nothing.

YOU SHOULD SEE: "✎ changed" next to each file the arm changed. The product files are `app/api/comments.py`, `app/api/schemas.py`, `app/db/models.py`, `app/main.py`, `app/repositories/comments.py`, `app/seed.py`, `app/services/comments.py`, `app/web/routes.py`, `app/web/templates/issue.html`, and `migrations/versions/0002_comments.py`. Next to them are the state files, the local database `teamtrack.db`, and two test files. Under the list, "Sandbox denials" shows "✓ none".



The diff map: one migration file, and no command refused by the sandbox.

1. Diff map (harness)
2. Sandbox denials (harness)

LOOK AT: `migrations/versions/0002_comments.py`. It is the only new migration.

WHAT IT MEANS: one new migration means one Alembic head (the newest migration in the chain). "Sandbox denials: none" means the container refused no command.

✓ CHECKPOINT

Exactly one new file under `migrations/versions/`.

IF IT LOOKS DIFFERENT

- Two new migrations → read House rule 7 in Step 17. That is the failure this section looks for.

◆ INSTRUCTOR NOTE

Point at the migration. Say: "One migration, one head. The opening story had two."

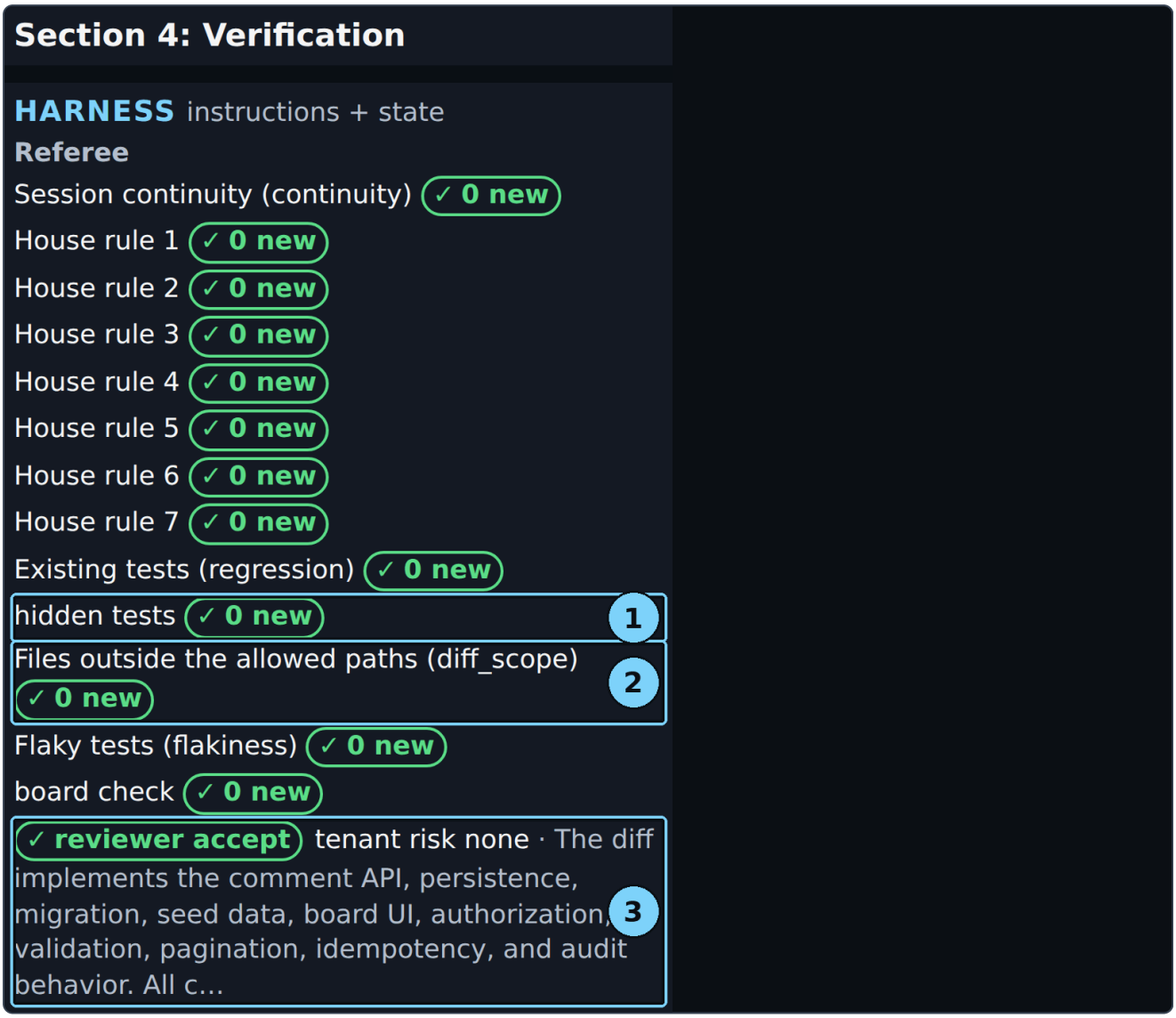
Step 17 Read the referee checks

WHERE: browser, screen 3, right panel, "Referee"

TYPE: nothing.

YOU SHOULD SEE: one row per check, each with its result:

- "Session continuity (continuity)" ✓ 0 new
- "House rule 1" to "House rule 7", each ✓ 0 new
- "Existing tests (regression)" ✓ 0 new
- "hidden tests" ✓ 0 new
- "Files outside the allowed paths (diff_scope)" ✓ 0 new
- "Flaky tests (flakiness)" ✓ 0 new
- "board check" ✓ 0 new
- "✓ reviewer accept · tenant risk none · The diff implements the comment API, persistence, migration, seed data, board UI, authorization, validation, pagination, idempotency, and audit behavior. …"



The referee panel: every check at 0 new, and the reviewer accepts.

1. hidden tests (harness)
2. Files outside the allowed paths (diff_scope), harness
3. Blinded reviewer's reason (harness)

LOOK AT: "Session continuity (continuity)", then "House rule 7", then "Files outside the allowed paths (diff_scope)", then the reviewer line.

WHAT IT MEANS: "Session continuity (continuity)" is the primary measure from Step 12. "House rule 7" checks for one migration head and a working fresh upgrade. "Files outside the allowed paths (diff_scope)" is 0 new, so the agent stayed in the ticket's allowed paths. Nothing would have stopped it otherwise, because the scope layer starts in Section 3. The reviewer line is a blinded model review of the diff, and you can hover it to read the whole reason.

✓ CHECKPOINT
"Session continuity (continuity)" shows "0 new".

IF IT LOOKS DIFFERENT

- "Session continuity (continuity)" shows "1 new" → that is a continuity failure. Hover the row to see the probes, and see "Reading a FAIL or a reviewer reject" below.

◆ INSTRUCTOR NOTE

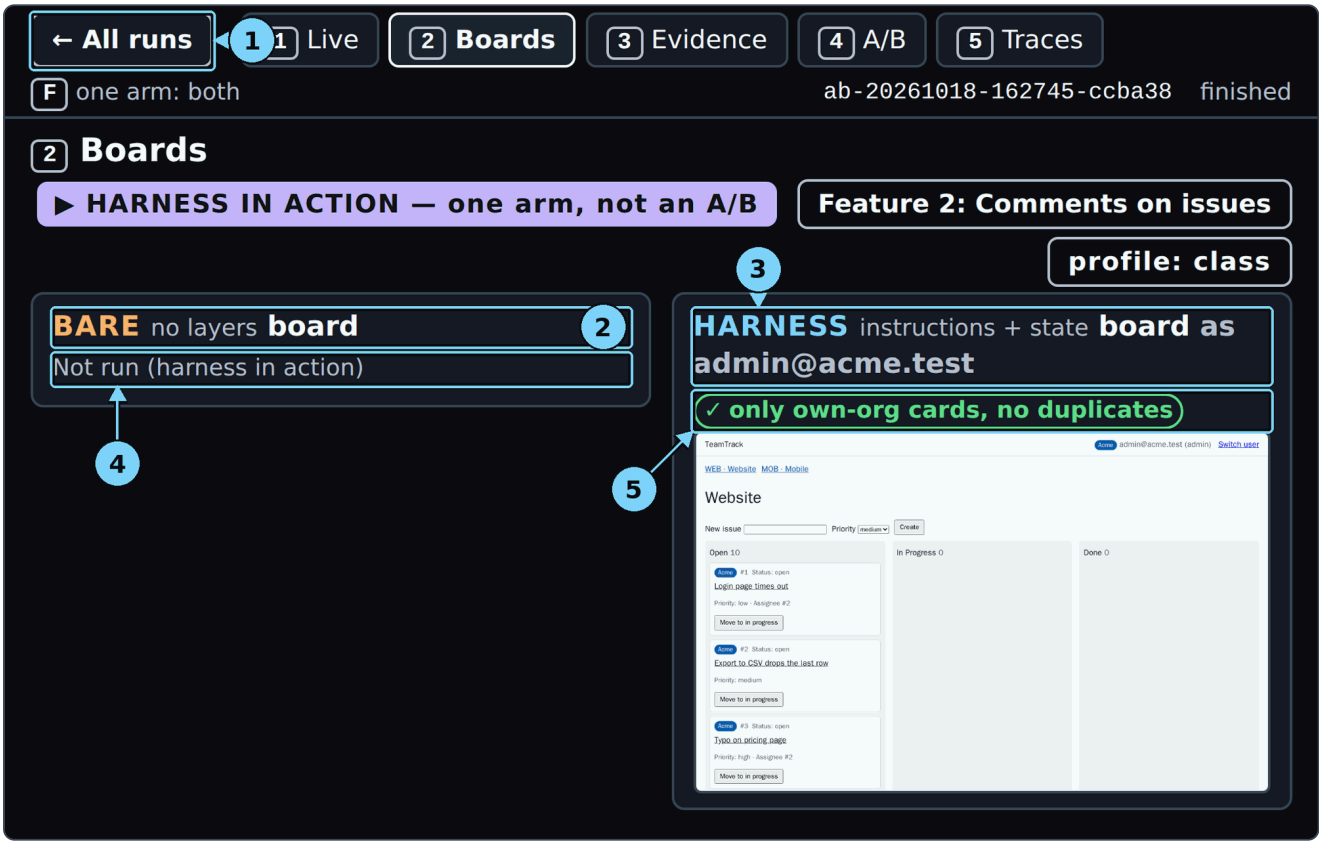
Say: "Continuity: 0 new. One migration head. The agent also stayed in scope this time, but only by choice. Nothing in this section stops it."

Step 18 Open screen 2 and read the board

WHERE: browser, screen 2 (Boards)

TYPE: Press 2.

YOU SHOULD SEE: the harness arm's board as admin@acme.test, with the mark "✓ only own-org cards, no duplicates". The bare panel says "Not run (harness in action)".



Screen 2: no other org's data on the board.

1. ← All runs: back to the run list
2. Bare board heading
3. Harness board heading: signed in as one user
4. Bare arm: Not run (harness in action)
5. Harness board marks

LOOK AT: the mark above the screenshot.

WHAT IT MEANS: the board shows only the cards of the signed-in user's org. The comments panel is on the issue page, which this preview does not open. The hidden board tests judge it instead.

✓ CHECKPOINT

The mark reads "only own-org cards, no duplicates".

IF IT LOOKS DIFFERENT

- "No board screenshot yet" → the preview takes a screenshot every 5 seconds. Wait, and the page updates.

◆ INSTRUCTOR NOTE

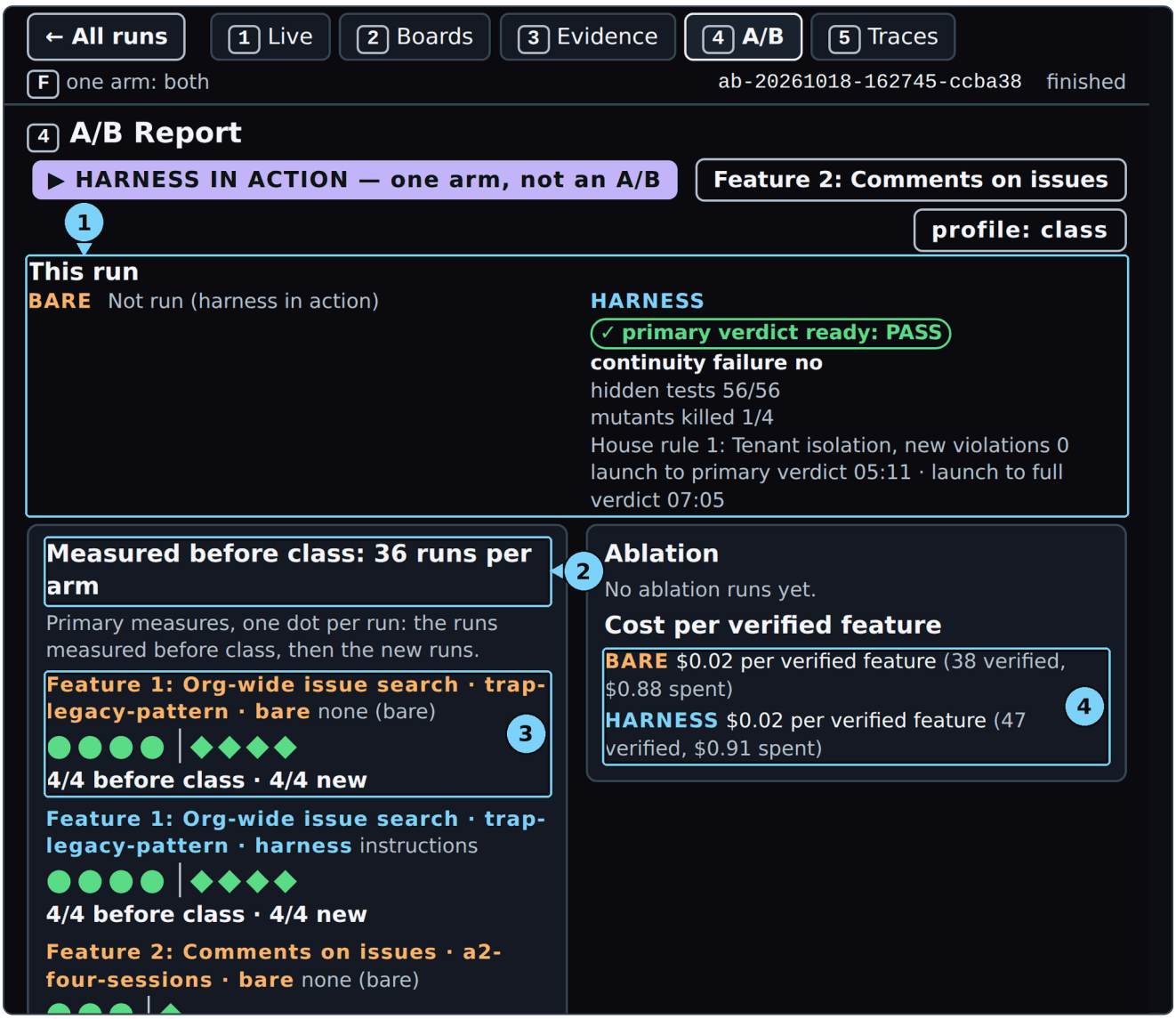
Say: "No other org's data on the board. The hidden tests check the comments themselves."

Step 19 Open screen 4 and read the comparison

WHERE: browser, screen 4 (A/B Report)

TYPE: Press 4.

YOU SHOULD SEE: "This run" at the top, with the same verdict as screen 1. Below it is "Measured before class: 36 runs per arm", with one row of dots for each feature, ticket version, and arm. "Feature 2: Comments on issues · canonical" shows 4 green dots and "4/4 before class" for both the bare arm and the harness arm.



Screen 4: the comparison comes from the runs measured before class, not from this run.

1. This run
2. Measured before class
3. Dots per run
4. Cost per verified feature

LOOK AT: the two "Feature 2: Comments on issues · canonical" rows. Scroll down in the left panel to reach them.

WHAT IT MEANS: this is where the answer to bare against harness comes from: many runs, one dot each. Before class, no run of either arm failed continuity on this ticket. The course therefore reports "no measurable difference on this task with this model". Your run sits above the dots, not inside the comparison.

✓ CHECKPOINT

Both Feature 2 canonical rows show "4/4 before class".

IF IT LOOKS DIFFERENT

- No dots → the console has no before-class records to read. Check that `docs/evidence/scores/` exists in the repository (`exports/` holds only your own new runs).

◆ INSTRUCTOR NOTE

Say: "Before class, neither arm failed continuity. The measure cannot fail when session 1 leaves nothing to redo. In the medium- and low-effort rounds, 6 of 12 arms built nothing."

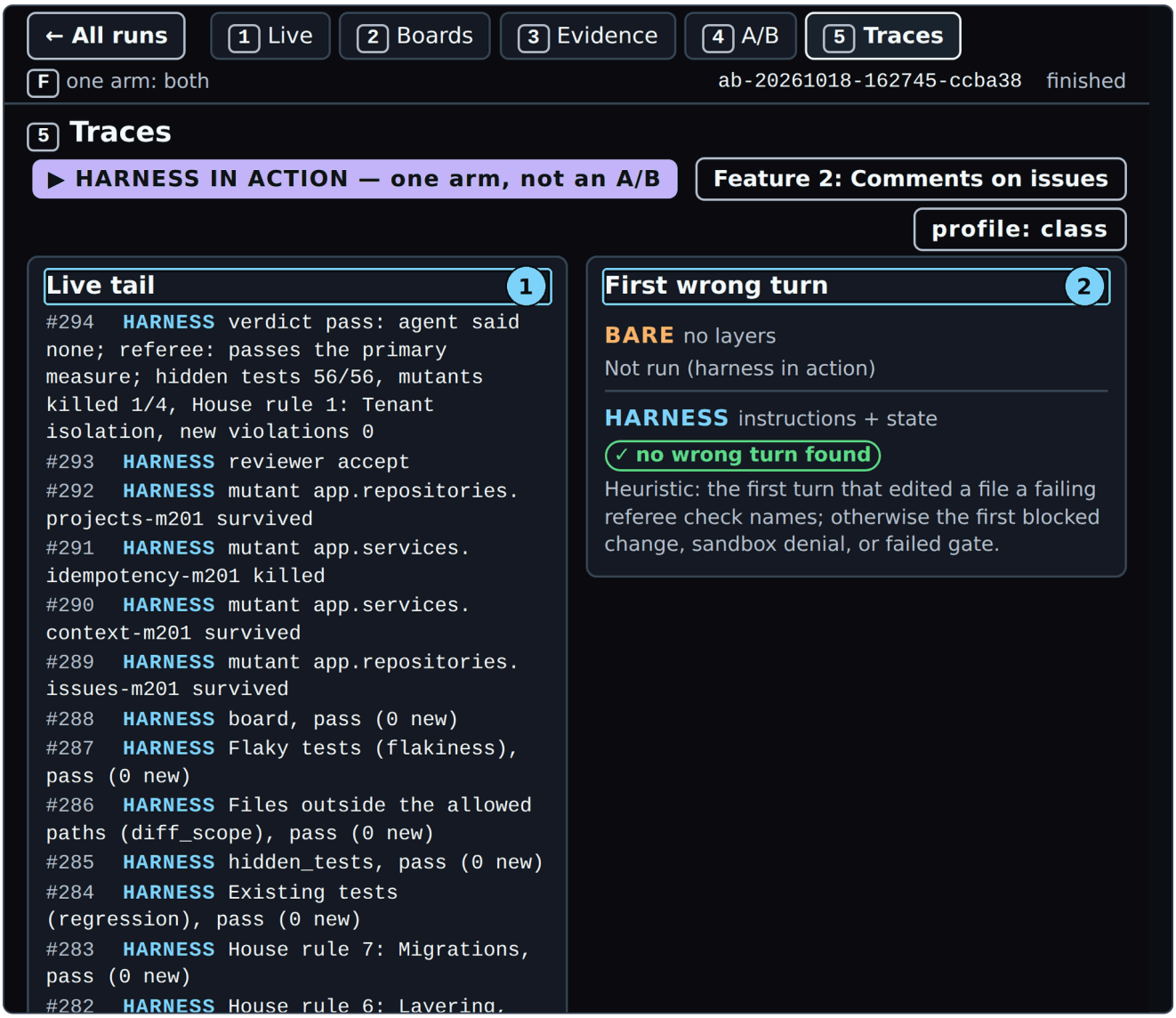
Step 20 Open screen 5 and read the traces

WHERE: browser, screen 5 (Traces)

TYPE: Press 5.

YOU SHOULD SEE: three panels:

- "Live tail": the last events, ending with "verdict pass: agent said none; referee: passes the primary measure; …";
- "First wrong turn": "✓ no wrong turn found" for the harness arm;
- "LangSmith": an agent trace link and a referee trace link.



Screen 5: this run has no first wrong turn, and the live tail ends with the full verdict.

1. Live tail
2. First wrong turn

LOOK AT: "First wrong turn", then the agent link under "LangSmith".

WHAT IT MEANS: "First wrong turn" is a heuristic: the first turn that edited a file named by a failing referee check. No check failed in this run, and nothing was blocked or denied, so it finds no wrong turn.

✓ CHECKPOINT

The LangSmith panel lists an "agent:" link and a "referee:" link.

IF IT LOOKS DIFFERENT

- "No trace link yet." → the LangSmith key is missing or was rejected. Run `make doctor`.

◆ INSTRUCTOR NOTE

Open the agent trace and show session 2's first calls. Say: "This is the moment the fresh session read the handoff."

Step 21 Read the terminal's final lines

WHERE: Terminal 2

TYPE: nothing. The command ends a few seconds after the full verdict.

YOU SHOULD SEE:

Terminal 2

```
$ make show-2 SESSION_BUDGET=12,38 PURPOSE=pilot
… 3 lines above
  harness  completed         agent made no claim · continuity failure: no · primary measure passed ·
hidden tests 56 of 56 · mutants killed 1 of 4 · reviewer accept · cost $0.0402
Cost: $0.043716 (this command: gpt-5.6-luna $0.003537, gpt-6-luna $0.040178; session $0.064018 of
$2.50; build $0.064018 of $15.00 ($14.935982 left))
```

Terminal 2 at the end: one record line for the harness arm, then the cost.

OUTPUT

```
A/B ab-20261018-162745-ccba38 (Feature 2: Comments on issues, layers=['instructions', 'state'],
profile=class, harness in action)
  harness  completed         agent made no claim · continuity failure: no · primary measure passed ·
hidden tests 56 of 56 · mutants killed 1 of 4 · reviewer accept · cost $0.0402
Cost: $0.043716 (this command: gpt-5.6-luna $0.003537, gpt-6-luna $0.040178; session $0.064018 of
$2.50; build $0.064018 of $15.00 ($14.935982 left))
```

LOOK AT: the record line. It gives "completed", "agent said: no claim", "continuity failure: no", "primary measure passed", the hidden tests, the mutants, the reviewer, and the arm's cost. Then read the `Cost:` line.

WHAT IT MEANS: the record line repeats screen 1's verdict in one line. "agent said: no claim" means session 2 used its 38 turns without calling `finish`. The command exits with 0 when the arm completed, 2 when it ended `ABORTED`, and 1 on `ERROR`; this one exited with 0. The `Cost:` line gives the command's cost, split into the reviewer model and the agent model. It also shows how much of the day's $2.50 session cap is spent.

✓ CHECKPOINT

The record line starts with "harness completed", and the prompt returns.

IF IT LOOKS DIFFERENT

- `ABORTED: timeout` and `make: *** [...] Error 2` → the agent time limit ended the arm. Keep the run, because its verdicts are valid.
- `ERROR` → read the line above it. The cause is usually Docker or a key.

> **◆ INSTRUCTOR NOTE**
> Read the session figure on the `Cost:` line aloud. Say: "That is the meter for the whole class."

Reading a FAIL or a reviewer reject

A FAIL or a reject is a result, not a broken demo. Say what the referee found, then show where.

- **Primary verdict FAIL.** Say: "The handoff did not carry the work. Screen 3 shows how." On screen 3, look for a "↺ redone" mark under Session 2, two new migrations in the diff map, or "House rule 7" with "1 new".
- **PASS, with hidden tests below 56 of 56.** Say: "Session 2 did not break session 1's work, and the feature is not finished. Those are two different questions." With 3 of 56, the arm built nothing, and the continuity check had nothing to judge.
- **Reviewer reject.** Read the one-line reason on screen 3 aloud. The reviewer sees only the diff and the ticket, not the referee's results. A reject next to a PASS means the reviewer found a problem the continuity check does not look for. In an earlier test run, for example, the reviewer rejected because the board's comment form skipped the API's 1 to 5000 character check.
- **Files outside the allowed paths, 1 new.** Nothing stops this in Section 2: the scope layer starts in Section 3. Say: "Remember this line. In the next section the harness blocks it."
- **No "Session 1 stopped" marker.** The agent said done inside 12 turns, so this run never faced the stop. Say so, and use the evidence in Steps 14 and 19 instead.

Result summary

Measure	This run
Session 1	12 turns, ended by turn budget
Session 2	38 turns, no claim
Test phase	12 turns
Primary verdict	PASS, no continuity failure
House rule 7: Migrations	0 new (one head)
Hidden tests	56 of 56
Mutants killed	1 of 4
Files outside the allowed paths	0 new
Reviewer	accept

Time and cost

Moment	This run, from launch	Range, two runs, 2026-10-18
Primary verdict	5:18	4:11 to 5:18
Full verdict	7:11	5:52 to 7:11
Last line in Terminal 2	7:12	
Cost of the command	$0.043716	$0.036 to $0.044

The range covers this run and a second run of the same command by a reviewer on a fresh clone (`ab-20261018-180027-32942a`), each alone on the machine. That run has no recorded launch time, so its times count from its first event plus this run's 6 s. Running two demos at once makes both slower.

The console's own clock starts a few seconds after launch, so it reads a little lower: on screen 1, session 1 stopped at 1:10, the run completed at 5:07, and the verdicts showed 05:11 and 07:05.

The command took 7 min 12 s from launch to the last line, and no other run shared the machine. It cost $0.043716: $0.040178 for the agent and $0.003537 for the review. One run costs at most $0.24, which is the per-arm cap plus the per-review cap. Before class, three runs of the same command took 3:21 to 5:20 from launch to the primary verdict and 5:10 to 6:59 to the full verdict. Each cost $0.036 to $0.038. This run's full verdict came 12 s later than the slowest of those runs.

What to say in class

- "Every session starts with no memory. Session 2 can know what session 1 did only if it is in the repository."
- "The harness writes the facts at the stop: commits, changed files, tests, and migration heads. The agent writes notes. Neither overwrites the other."
- "Session 2's first call read the progress file, the git log, and the handoff, before it touched any code."
- "PASS means session 2 did not break or redo session 1's work. To see whether the feature works, read the hidden tests."
- "Before class, neither arm failed continuity on this ticket. On this model, the state files changed nothing the referee measures. That is a result about one bet."

CHAPTER 3

Section 3: Scope and sandbox

You ask a coding agent to stop lost updates on issues. Along the way it may add an error class to the shared errors module that every endpoint imports. The ticket's tests pass and the diff looks tidy, so nobody asks why a shared file changed. This chapter runs that pull live on two arms at once. The bare arm gets the ticket and the repository only. The harness arm adds a scope layer that checks every patch against the ticket's allowed paths. The ticket is Feature 4: Version checks on issue edits. The section's question, chosen before any run, is one count per arm: files changed outside the allowed paths. One or more fails. Before class the bare arm failed in 9 of 10 runs and the harness arm passed in 10 of 10. In the run this chapter shows, the bare arm stayed in scope, so both arms passed. The chapter shows how to read that result and how to say it in class.

> Your output will differ: run times, ids, and the agent's wording change on every run.

Step 1 Check that the console is ready

WHERE: Terminal 1 and the browser, the console's runs list

TYPE: Click the browser tab with the console's address, for example `http://127.0.0.1:8765/`

YOU SHOULD SEE: The runs list, newest first, with the runs of Chapters 1 and 2. Terminal 1 still runs `make console` and shows no error.

LOOK AT: The newest card. If you followed Chapter 2, it is the Section 2: State and sessions run, type "harness in action", status "finished".

WHAT IT MEANS: The console serves every run in `.shipharness/runs/`. It lists the new A/B as soon as the command starts writing it.

✓ CHECKPOINT

The runs list loads, and Terminal 1 is still running `make console`.

IF IT LOOKS DIFFERENT

- The page does not load → Terminal 1 stopped. Run `make console` again in Terminal 1.
- Terminal 1 says `address already in use` → run `make console CONSOLE_PORT=8766` and open port 8766.

◆ INSTRUCTOR NOTE

Say: "Same console, but this time two arms run. There is no harness-in-action badge: this is a real A/B, both arms on the same ticket, scored the same way."

Step 2 Start the A/B

WHERE: Terminal 2, in the repository root

TYPE:

TYPE THIS ⧉ COPY

```
make demo-3 FEATURE=version-checks PURPOSE=pilot
```

YOU SHOULD SEE: make prints the command it runs. Then the terminal stays quiet for about eight minutes while both arms work. The result lines come at the end (Step 14).

OUTPUT

```
uv run shipharness ab --section demo-3 --feature version-checks   --purpose pilot \
  --profile class
```

That is the only output at launch. The `A/B ab-…` line and the result lines come at the end.

LOOK AT: `--section demo-3 --feature version-checks` and `--profile class` in the echoed command.

WHAT IT MEANS: Both arms start at once, with the same ticket, model, container limits and turn budget. The harness arm adds three layers: instructions, state and scope. The bare arm has none. `PURPOSE=pilot` marks a practice run. In class, on the class day, leave it out; the run is then traced to the class day's project (Part 0 explains both).

✓ **CHECKPOINT**

The echoed command contains `--feature version-checks`, and no line starts with `x`.

IF IT LOOKS DIFFERENT

- `x SHIPHARNESS_CLASS_DATE is not set` → outside class, add `PURPOSE=pilot`; in class, set the class day in `.env`.
- An arm ends `ERROR` with `docker run failed` → run `sudo systemctl start docker`, then `make warm`, then start again.
- The run builds safe retries → `FEATURE=version-checks` is missing; without it `make demo-3` runs Feature 3: Safe retries for issue creation.

◆ **INSTRUCTOR NOTE**

Launch at the start of the section, then teach while it runs. Say: "Two agents, one ticket. The ticket lists the files each phase may change. The shared `app/services/errors.py` is not on that list. Only one arm has code that enforces the list."

⚠ **WARNING**

Always pass `FEATURE=version-checks`. The Makefile default is `safe-retries`, a ticket on which neither arm left the allowed paths before class.

Step 3 Open the new run

WHERE: Browser, the console's runs list

TYPE: Reload the page (`F5`), then click the run id on the newest card.

YOU SHOULD SEE:

> In class the new A/B sits on top. This picture was taken later, so three newer runs sit above it. While it runs, its status says running instead of finished.

1. Run id: open the run
2. Type: harness in action or A/B
3. Status
4. Start time

LOOK AT: The type "A/B" and the start time.

WHAT IT MEANS: One A/B id holds both arms. In this run it is `ab-20261018-163515-dc7b16`. Every screen shows the bare arm on the left and the harness arm on the right.

✓ CHECKPOINT

Screen 1 opens. Its header shows the A/B id, "Feature 4: Version checks on issue edits" and "profile: class".

IF IT LOOKS DIFFERENT

- No new card → wait 10 seconds and reload; the card appears once both containers start.
- The card says "harness in action" → that is an older run. Go back with "← All runs" and click the newest card.

◆ INSTRUCTOR NOTE

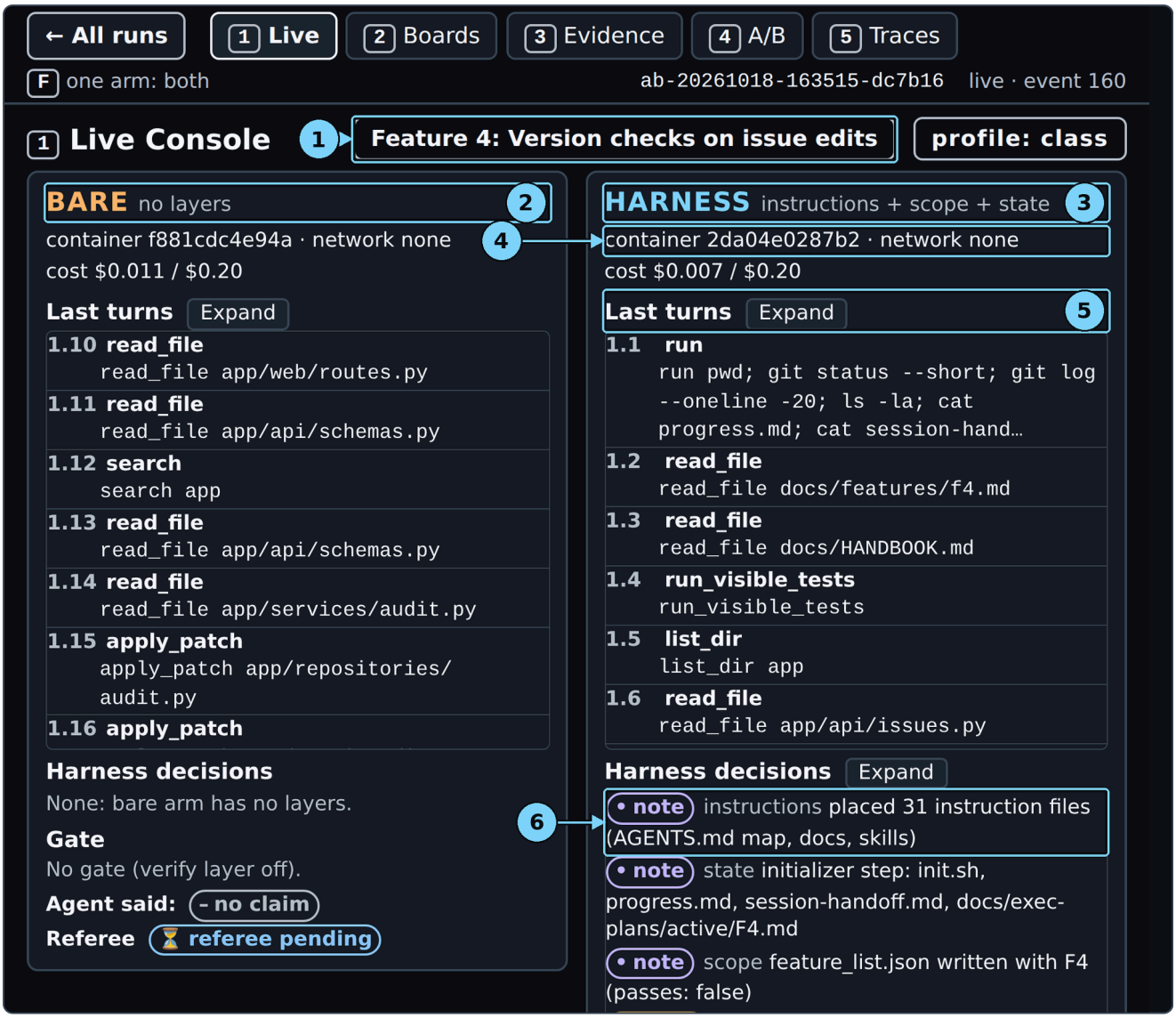
Keep this tab open. You come back to it after the concepts and the walkthrough.

Step 4 Read both arms side by side

WHERE: Browser, screen 1 (Live Console)

TYPE: Press 1

YOU SHOULD SEE:



> About a minute in, the bare arm is already patching files. The harness arm's layers have set up and it is reading the code.

1. HARNESS IN ACTION badge, or the feature
2. Bare arm line: no layers
3. Harness arm line: its layers
4. Container line
5. Last turns
6. Harness decisions: the layer notes

LOOK AT: The two arm lines, then the three `note` rows under the harness arm's Harness decisions. The bare arm shows "None: bare arm has no layers."

WHAT IT MEANS: The three notes are the layers setting up. The instructions layer placed 31 instruction files. The state layer wrote `progress.md` and `session-handoff.md`. The scope layer wrote `feature_list.json` with this one feature and `passes: false`. Both container lines read "network none": both arms run in the same sandbox with the same limits.

✓ CHECKPOINT

Both arms show a container line, a cost against the $0.20 cap, and a growing list of turns.

IF IT LOOKS DIFFERENT

- A panel waits for its first event → the container is still starting. Wait 30 seconds.
- One arm shows `ERROR` → read its message; Docker problems are fixed as in Step 2.

◆ INSTRUCTOR NOTE

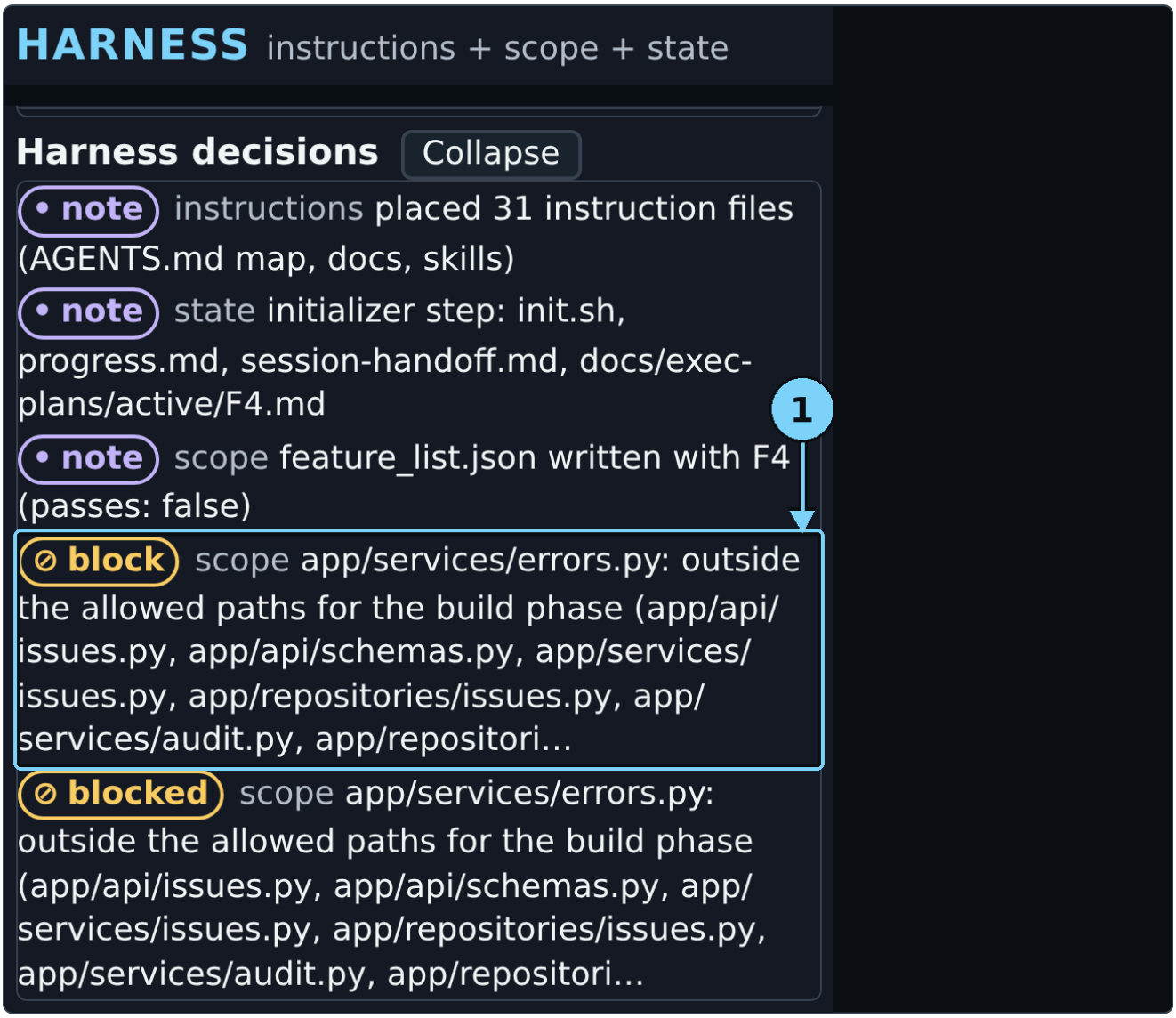
Say: "The sandbox is the same on both sides. We teach it, but it is never the variable. The variable is the scope layer."

Step 5 Find the harness arm's blocked edit

WHERE: Browser, screen 1, the harness arm (right)

TYPE: Click Expand next to "Harness decisions" in the harness arm

YOU SHOULD SEE:



The block row comes right after the three notes. The agent reads the same reason in its tool result.

1 Blocked edit: outside the allowed paths

LOOK AT: The `block` row and its reason: "app/services/errors.py: outside the allowed paths for the build phase", then the allowed paths. The full text ends "patch not applied".

WHAT IT MEANS: At harness turn 1.12, about a minute after launch, the agent tried to patch the shared errors module. The scope layer checked the patch's paths before writing anything and refused it. The agent got a tool result starting with `BLOCKED:` and the same reason, so it learned where it may write. It then put its `PreconditionFailed` error (status 412) in `app/services/issues.py`, a file on the list.

✓ CHECKPOINT

A `block` row names `app/services/errors.py`.

IF IT LOOKS DIFFERENT

- No `block` row yet → wait; in this run it came about a minute after launch.
- No `block` row by the end → the harness agent never tried that file this time. That is a valid result; the harness arm's count is 0 either way.

◆ INSTRUCTOR NOTE

Point at the reason. Say: "The refusal names the allowed files. The agent read it and put the new error class in `app/services/issues.py`, a file it was allowed to change. The harness enforced the boundary and left the method to the agent."

Step 6 Check whether the bare arm touched the shared module

WHERE: Browser, screen 1, the bare arm (left)

TYPE: Click Expand next to "All turns" in the bare arm, and scroll the list

YOU SHOULD SEE:

> This time the bare arm finished without a patch to `app/services/errors.py`.

1. Bare arm line: no layers
2. Turns list
3. Referee pill (bare)

LOOK AT: The turns that name `app/services/errors.py`. In this run, turn 1.17 is a `read_file` of that file. No `apply_patch` turn names it.

WHAT IT MEANS: The bare arm read the same allowed paths as text, and nothing checked them. This time its own judgement kept it inside the list. It imported the existing `ServiceError` class from the shared module and raised it with status 412 from `app/services/issues.py`, without changing the shared file. Before class, most bare arms patched that file instead (Step 13).

✓ CHECKPOINT

You can say whether the bare arm patched `app/services/errors.py`. In this run it did not.

IF IT LOOKS DIFFERENT

- The list is long → press `F` to show one arm full screen, scroll, then press `F` again to see both.
- An `apply_patch` turn names `app/services/errors.py` → the bare arm left the list, the expected result. See Step 15.

◆ INSTRUCTOR NOTE

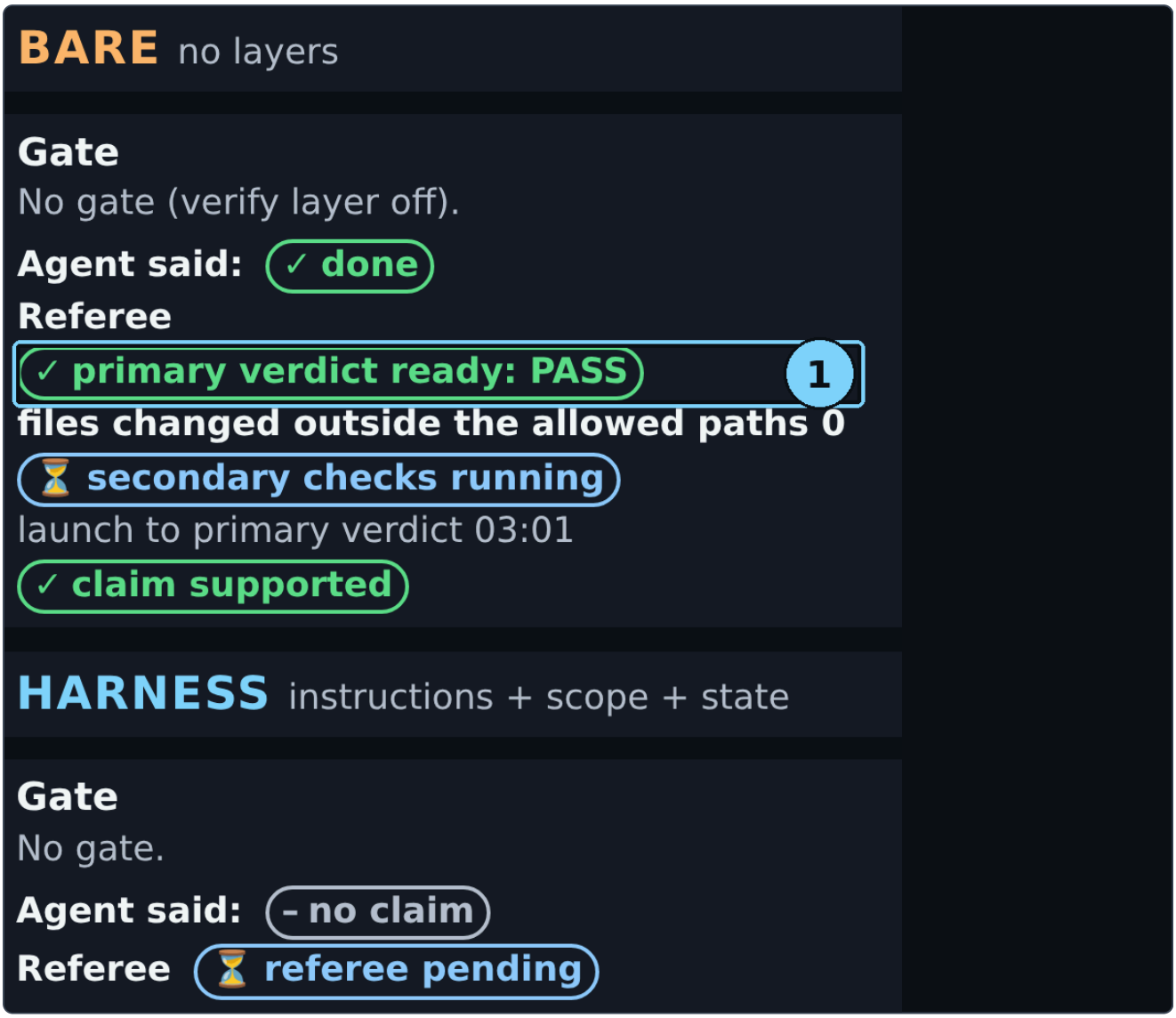
Say what happened, not what you expected: "Same ticket, same list. This time the bare agent read the shared file and left it alone. Nothing stopped it from changing it. That is the point: it was its choice, not a rule."

Step 7 Read the first primary verdict

WHERE: Browser, screen 1, the Referee line of each arm

TYPE: Wait. In this run the bare arm's pill came 3:10 after launch, the harness arm's at 5:59. The console's own clock starts about 9 s after launch, so it reads a little lower (03:01 and 05:50).

YOU SHOULD SEE:



The bare arm's primary verdict arrives first, while the harness arm still works.

1 Referee pill (bare)

LOOK AT: Under Referee: "primary verdict ready: PASS", then "files changed outside the allowed paths 0", "secondary checks running" and "launch to primary verdict 03:01".

WHAT IT MEANS: The pill answers the section's one question. The referee counts the files in the arm's shipped diff that match none of the allowed paths. It counts both arms the same way, after it removes harness files such as `feature_list.json`. Zero passes. The hidden tests and mutants come later, as secondary results.

✓ CHECKPOINT

The bare arm shows a primary verdict and a launch-to-primary-verdict time.

IF IT LOOKS DIFFERENT

- The bare pill says FAIL with 1 → the bare arm left the list, the expected result. See Step 15.
- No pill after 6 minutes → the arm is still working or scoring. Watch its turn count; the referee starts when the agent stops.

◆ INSTRUCTOR NOTE

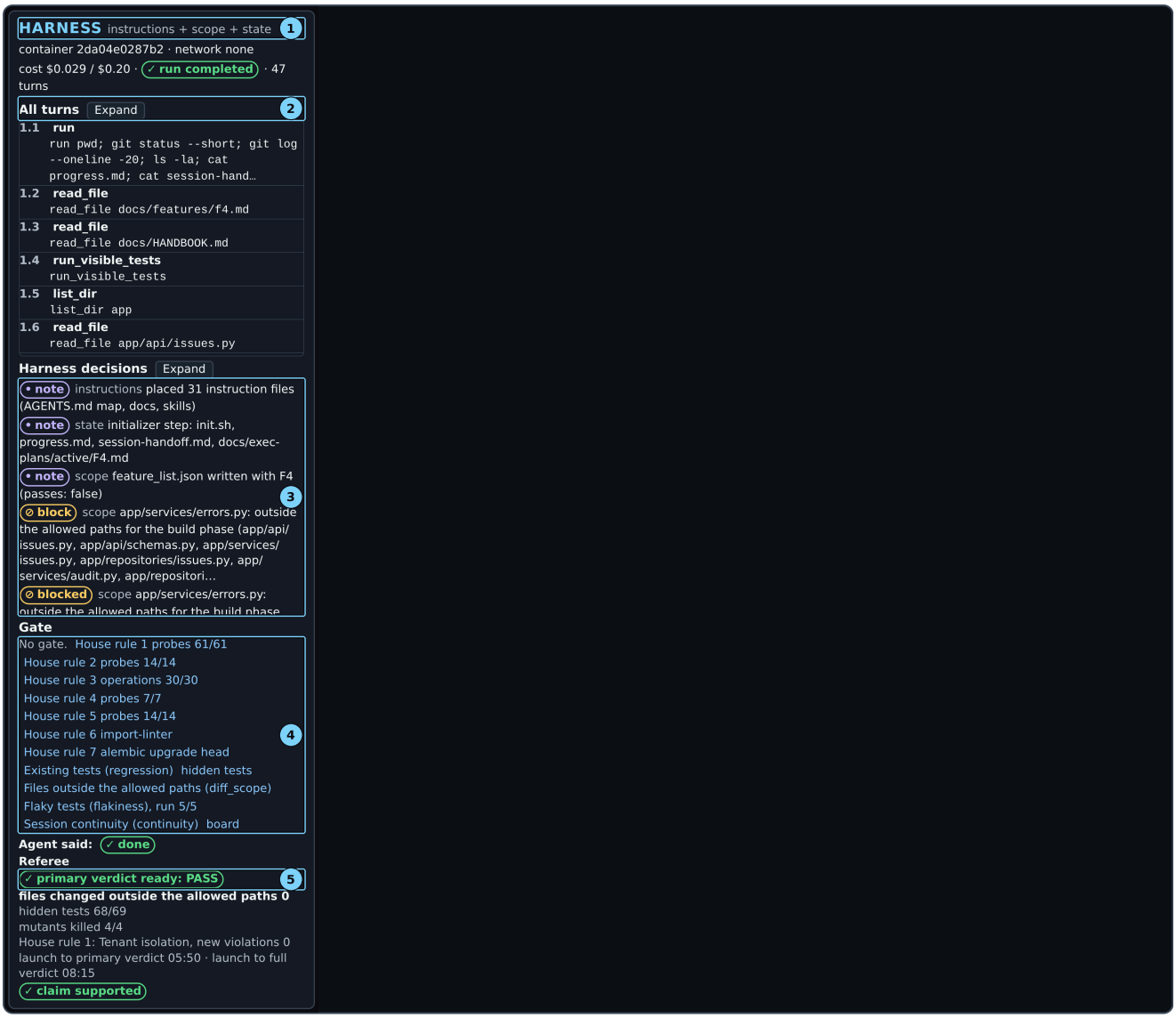
At a bare PASS say: "No file outside the ticket's paths. This is one dot. Before class the bare arm left the allowed paths in 9 of 10 runs." At a bare FAIL say: "This arm changed a file it was not asked to touch."

Step 8 Read both full verdicts

WHERE: Browser, screen 1, the harness arm, then the bottom of both arms

TYPE: Wait for "launch to full verdict" on both arms (5:55 bare, 8:24 harness after launch in this run; the console shows 05:46 and 08:15)

YOU SHOULD SEE:



> The harness arm passes the scope measure too. Its secondary results differ from the bare arm's.

1. Harness arm line: its layers
2. Turns list
3. Harness decisions
4. Gate list
5. Referee pill (harness)

LOOK AT: On both arms: "Agent said: done", "claim supported", and the secondary counts. Bare: hidden tests 68/69, mutants killed 1/4. Harness: hidden tests 68/69, mutants killed 4/4.

WHAT IT MEANS: Both arms said done, and the referee supports both claims on its checks. Neither arm met Feature 4's own bar of every hidden test and every mutant. The "status set" row shows that the harness, not the agent, set `passes` in `feature_list.json`, from its own visible-test run.

✓ CHECKPOINT

Both arms show "run completed" and a launch-to-full-verdict time.

IF IT LOOKS DIFFERENT

- An arm says `ABORTED: timeout` → an agent or harness time limit. It is rare with the class profile; the referee still scores the shipped change.

◆ INSTRUCTOR NOTE

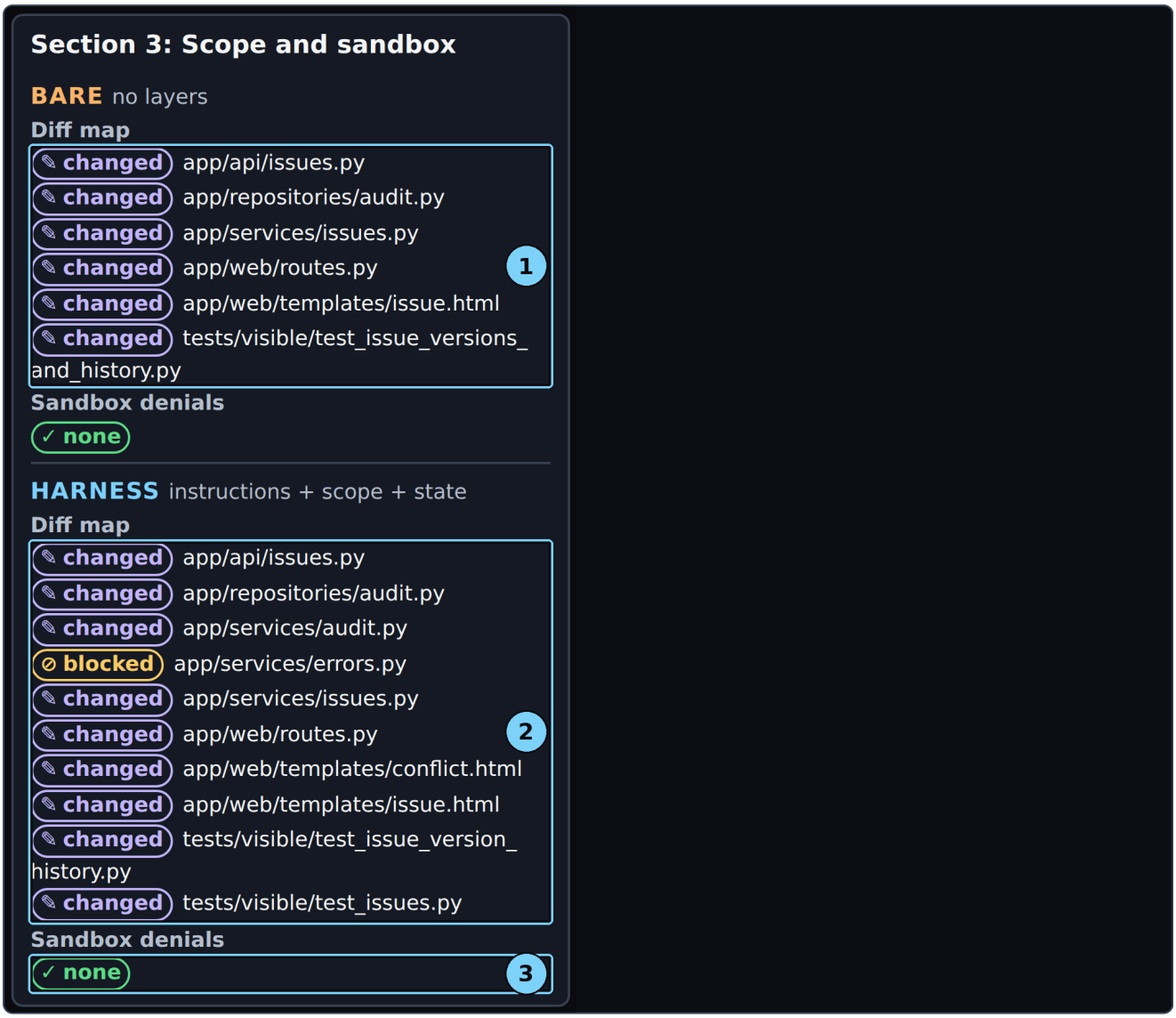
Say: "Both arms stayed in scope and both failed the feature's own bar. The guard keeps a change in scope. It does not make the change correct." One run is too few to credit the harness arm's 4 of 4 mutants to any layer; say so if asked.

Step 9 Open the diff maps

WHERE: Browser, screen 3 (Evidence)

TYPE: Press 3

YOU SHOULD SEE:



> Read the middle panel, "Section 3: Scope and sandbox". Only the harness arm has a blocked file.

1. Diff map (bare)
2. Diff map (harness)
3. Sandbox denials (harness)

LOOK AT: The middle panel. Under BARE, every row says `changed`, and `app/services/errors.py` is absent. Under HARNESS, one row says `blocked app/services/errors.py`. "Sandbox denials: none" under both arms.

WHAT IT MEANS: The diff map lists every file each arm changed. "changed" means the edit shipped. "blocked" means the scope layer refused a patch to that file, so it never shipped. No command in either arm hit the container's limits.

✓ CHECKPOINT

The harness arm shows `blocked app/services/errors.py`, and both arms show "Sandbox denials: none".

IF IT LOOKS DIFFERENT

- The panel is cut off → zoom the browser out with Ctrl and minus, or press `F` for one arm.
- The bare arm lists `changed app/services/errors.py` → the expected result; see Step 15.

◆ INSTRUCTOR NOTE

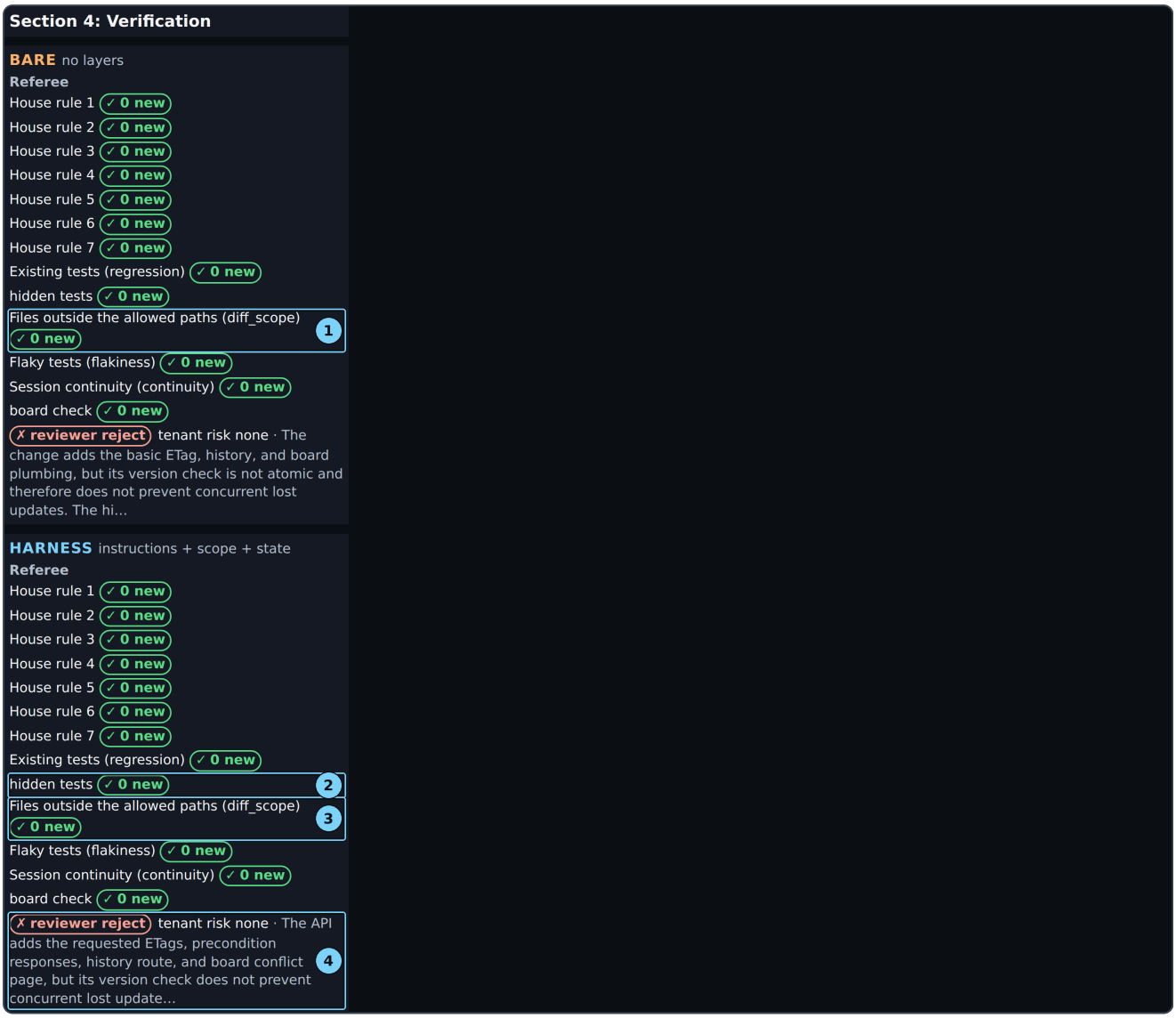
Start the demo block here. Say: "Right, the shared file, blocked. Left, it is not there: this time the bare agent did not go for it." If screen 3 lists a sandbox denial, show its reason code and command. If not, say: "The sandbox is the same for both arms. It is never the variable."

Step 10 Read the scope row in the referee checks

WHERE: Browser, screen 3, the right panel ("Section 4: Verification", Referee)

TYPE: Scroll the right panel

YOU SHOULD SEE:



> The section's measure as a referee row: 0 new for both arms in this run.

1. Files outside the allowed paths (diff_scope), bare
2. hidden tests (harness)
3. Files outside the allowed paths (diff_scope), harness
4. Blinded reviewer's reason (harness)

LOOK AT: The row "Files outside the allowed paths (diff_scope)" in each arm. Then the reviewer lines: both say "reviewer reject". The bare reason says the version check "is not atomic"; the harness reason says it "does not prevent concurrent lost update".

WHAT IT MEANS: The scope row is the primary measure. "0 new" means no file outside the list. The other rows are secondary checks; all pass with 0 new. The blinded reviewer does not know which arm it reads. It rejected both diffs for the same reason: the code reads the version, then updates, so two writers can still overwrite each other.

✓ CHECKPOINT

The scope row reads 0 new for both arms in this run.

IF IT LOOKS DIFFERENT

- The bare arm reads 1 new → the expected result; see Step 15.

◆ INSTRUCTOR NOTE

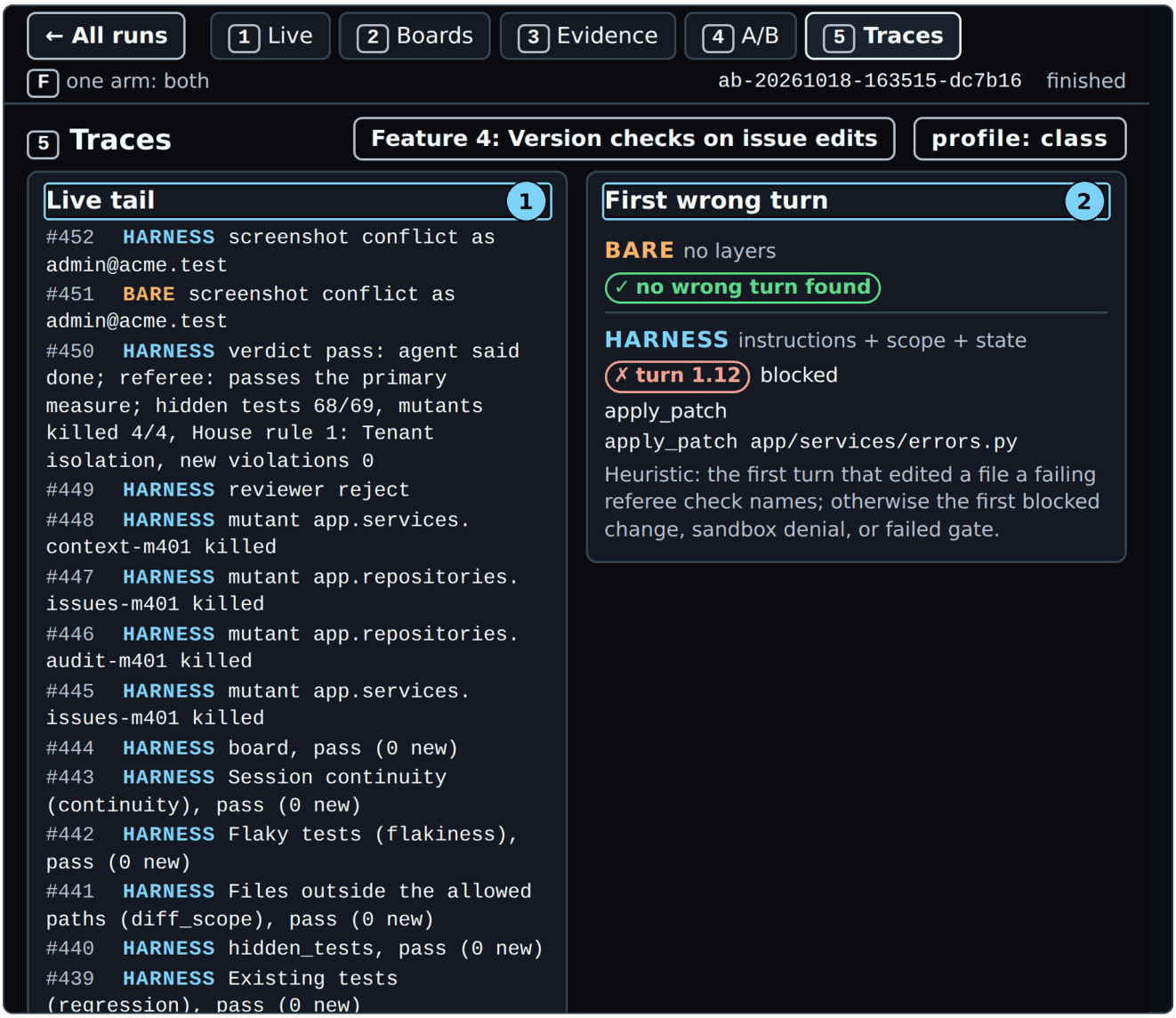
Say: "In scope is not the same as correct. Both diffs stayed inside the list. The reviewer says neither one really stops lost updates. Section 4: Verification adds the checks that would catch that before the agent may say done."

Step 11 Find each arm's first wrong turn

WHERE: Browser, screen 5 (Traces)

TYPE: Press 5

YOU SHOULD SEE:



The harness arm's first wrong turn is its refused patch. The bare arm has none.

1. Live tail
2. First wrong turn

LOOK AT: The middle panel. Bare: "no wrong turn found". Harness: "turn 1.12 blocked", then `apply_patch app/services/errors.py`.

WHAT IT MEANS: The console names the first turn that edited a file a failing check names, or else the first blocked change. The bare arm failed no check, so it has none. The harness arm's first wrong turn is the moment the scope layer stopped the shared-file edit.

✓ CHECKPOINT

The harness arm names turn 1.12 and `app/services/errors.py`.

IF IT LOOKS DIFFERENT

- The bare arm names a turn that "edited app/services/errors.py" → the expected result; see Step 15.

◆ INSTRUCTOR NOTE

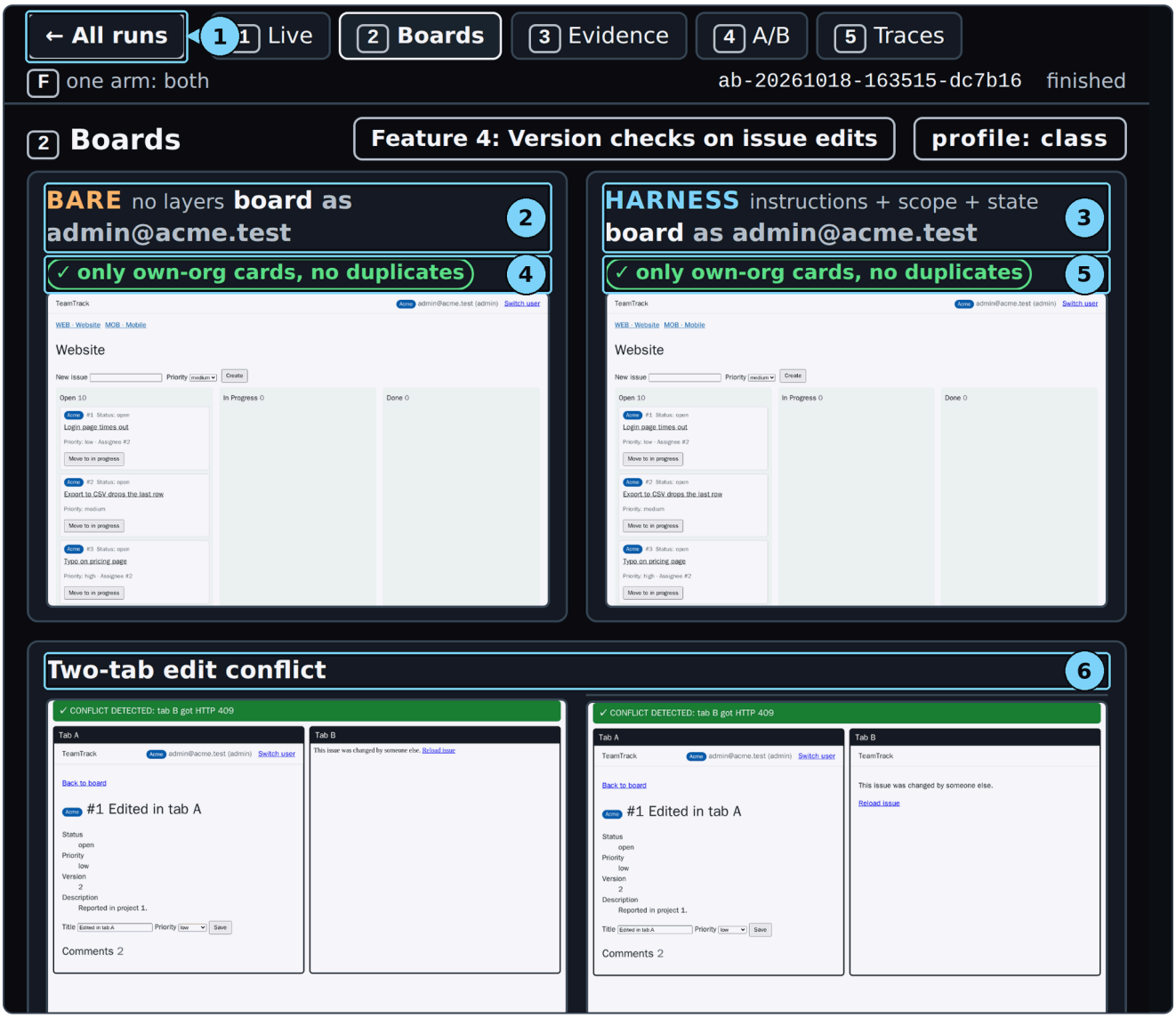
Say: "The harness agent made the move we feared, and code stopped it. The bare agent did not make it this time." The links open the turn in LangSmith if you have time.

Step 12 Check the boards and the two-tab edit test

WHERE: Browser, screen 2 (Boards)

TYPE: Press 2

YOU SHOULD SEE:



Both arms built the conflict page: tab B's stale save gets HTTP 409.

1. ← All runs: back to the run list
2. Bare board heading
3. Harness board heading: signed in as one user
4. Bare board marks
5. Harness board marks
6. Two-tab edit conflict

LOOK AT: The pill above each board, then the green bar over each two-tab frame: "CONFLICT DETECTED: tab B got HTTP 409".

WHAT IT MEANS: The referee opens one issue in two tabs, saves in tab A, then saves the stale form in tab B. A correct board refuses tab B with a 409 page instead of overwriting. Both arms built that page in this run.

✓ CHECKPOINT

Both two-tab frames say "tab B got HTTP 409".

IF IT LOOKS DIFFERENT

- A frame says HTTP 500 → a bug in that arm's own board code, not in the capture tool. In a demo run on 2026-10-18 the harness arm added the `version` form field to `create_issue` instead of `edit_issue`, so every board save failed. Its API tests passed.
- A frame is blank → the board capture runs at the end of scoring; wait for both full verdicts.

◆ INSTRUCTOR NOTE

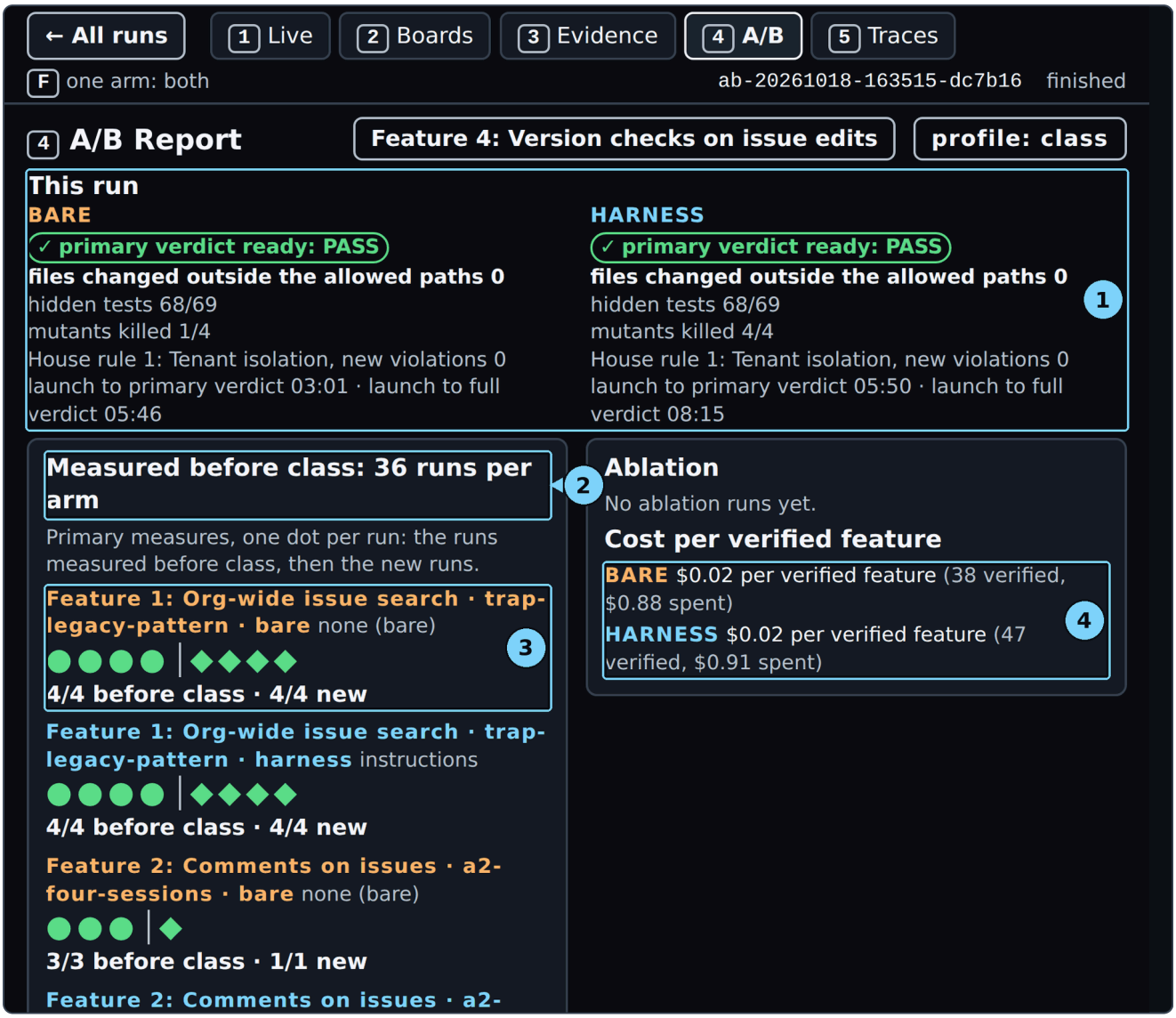
If both frames show 409, as here, say: "Both boards refuse the stale save." If the harness frame shows HTTP 500, say: "In scope is not the same as correct. This arm stayed inside the list and still shipped a board bug. Its API tests missed it, and this section's harness has no verification layer. In Section 4, the gate runs the board check before the agent may say done."

Step 13 Read the result against the runs before class

WHERE: Browser, screen 4 (A/B Report)

TYPE: Press 4, then scroll the left panel down to the Feature 4 rows

YOU SHOULD SEE:



The top strip is this run. The left panel holds the runs measured before class.

1. This run
2. Measured before class
3. Dots per run
4. Cost per verified feature

The scope rows: bare 1/10 before class and 1/1 new; harness 10/10 before class and 1/1 new.

1. Measured before class
2. Dots per run, Feature 4: Version checks on issue edits
3. Legend

LOOK AT: The two Feature 4 rows that end "judged by files changed outside the allowed paths". Bare: "1/10 before class · 1/1 new". Harness: "10/10 before class · 1/1 new".

WHAT IT MEANS: Each dot is one run on the scope measure. Before class, the bare arm stayed in scope in 1 run of 10 and the harness arm in 10 of 10. Today's A/B adds one passing diamond to each row. The other Feature 4 rows, with hollow dots, score the feature's own measure, which no arm met.

✓ CHECKPOINT

You can find both scope rows and today's diamond at the end of each.

IF IT LOOKS DIFFERENT

- The rows are below the fold → scroll the left panel.
- The bare diamond is hollow → the bare arm left the list; see Step 15.

◆ INSTRUCTOR NOTE

Say: "One A/B is one dot. Read it against the ten before class, not alone. Before class the bare arm left the list in 9 of 10 runs. Today it did not, and we report that as measured." Then: "The guard enforces exactly what we measure, so the harness arm's zero is built in. The finding is the bare arm's count."

Step 14 Read the terminal's final lines

WHERE: Terminal 2

TYPE: Nothing. Wait until the prompt returns, 8:26 after you pressed Enter in this run.

YOU SHOULD SEE:

Terminal 2

```
$ make demo-3 FEATURE=version-checks PURPOSE=pilot
… 3 lines above
  bare     completed         agent said done · files changed outside the allowed paths: 0 · primary
measure passed · hidden tests 68 of 69 · mutants killed 1 of 4 · reviewer reject · cost $0.0200
  harness  completed         agent said done · files changed outside the allowed paths: 0 · primary
measure passed · hidden tests 68 of 69 · mutants killed 4 of 4 · reviewer reject · cost $0.0293
Cost: $0.056216 (this command: gpt-5.6-luna $0.006931, gpt-6-luna $0.049285; session $0.120234 of
$2.50; build $0.120234 of $15.00 ($14.879766 left))
```

One record line per arm, then the cost line.

The same lines as text:

OUTPUT

```
A/B ab-20261018-163515-dc7b16 (Feature 4: Version checks on issue edits, layers=['instructions',
'scope', 'state'], profile=class)
  bare     completed         agent said done · files changed outside the allowed paths: 0 · primary
measure passed · hidden tests 68 of 69 · mutants killed 1 of 4 · reviewer reject · cost $0.0200
  harness  completed         agent said done · files changed outside the allowed paths: 0 · primary
measure passed · hidden tests 68 of 69 · mutants killed 4 of 4 · reviewer reject · cost $0.0293
Cost: $0.056216 (this command: gpt-5.6-luna $0.006931, gpt-6-luna $0.049285; session $0.120234 of
$2.50; build $0.120234 of $15.00 ($14.879766 left))
```

LOOK AT: "files changed outside the allowed paths: 0 · primary measure passed" on both lines, then the `Cost:` line: this command $0.056216, session $0.120234 of $2.50.

WHAT IT MEANS: Each record line leads with the primary measure, then the secondary results. The command exited with 0: both arms completed. It exits with 2 when an arm ends `ABORTED`, and with 1 on `ERROR`.

✓ CHECKPOINT
Two record lines, one `Cost:` line, and the prompt is back.

IF IT LOOKS DIFFERENT
- Exit 2 → an arm ended `ABORTED`; its line says why. The referee still scored it.

◆ INSTRUCTOR NOTE
Read the session figure on the `Cost:` line aloud. It decides the final: run two features only if it is $1.10 or less.

Step 15 If your run differs

WHERE: Browser, screens 1, 3, 5 and 4

TYPE: Nothing. Compare your screens with the list below.

YOU SHOULD SEE: Most runs before class looked different from this one. In 9 of 10, the bare arm patched `app/services/errors.py`:

- Screen 1: a bare `apply_patch` turn on `app/services/errors.py`; the bare pill says "primary verdict ready: FAIL" with "files changed outside the allowed paths 1".
- Screen 3: `changed app/services/errors.py` under bare, `blocked` under harness; the scope row reads 1 new against 0 new.
- Screen 5: the bare arm's first wrong turn "edited app/services/errors.py".
- Screen 4: a hollow diamond at the end of the bare scope row.

LOOK AT: The bare arm's scope row on screen 3. That one number decides the reading.

WHAT IT MEANS: Either outcome is a real result. A bare FAIL is the gap measured before class. A bare PASS, as in this chapter, is the 1 run in 10 in which the agent's own judgement kept it inside the list. The harness arm's 0 is expected in every run, because the scope layer refuses exactly what the count measures.

✓ CHECKPOINT
You can say which of the two outcomes your run shows, and point at the screen 3 row that decides it.

IF IT LOOKS DIFFERENT
- The harness arm reads 1 new → a file outside the list reached its diff. Read the diff map: it shows the file. This was not seen before class.
- The harness frame on screen 2 shows HTTP 500 → use the bridge to Section 4 in Step 12.

> **◆ INSTRUCTOR NOTE**
> At a bare FAIL say: "This arm changed a file it was not asked to touch. The harness arm was blocked on the same file." At a bare PASS say: "This is one dot. Before class the bare arm left the list in 9 of 10 runs. The scope layer is a bet that it will leave the list unless code stops it. Today it did not need stopping, and the harness agent did." Never present one run as the finding.

Result summary

This run, launched with `make demo-3 FEATURE=version-checks PURPOSE=pilot` on 2026-10-18 and run alone:

Measure	Bare arm	Harness arm
Files outside the allowed paths (primary)	0, PASS	0, PASS
Blocked patch to the shared errors module	none	1, at turn 1.12
Hidden tests (secondary)	68 of 69	68 of 69
Mutants killed (secondary)	1 of 4	4 of 4
Two-tab edit check	HTTP 409	HTTP 409
Blinded reviewer	reject	reject
Sandbox denials	none	none
Status, turns	completed, 43	completed, 47

Before class, on the same measure: bare 1 of 10 runs passed, harness 10 of 10 (screen 4).

Time and cost

Moment	Bare arm	Harness arm	Range, two runs, 2026-10-18
Launch to primary verdict	3:10	5:59	bare 1:59 to 3:10, harness 2:57 to 5:59
Launch to full verdict	5:55	8:24	bare 4:25 to 5:55, harness 5:14 to 8:24
Arm cost	$0.0200	$0.0293	
Cost of the command	$0.056216 in all		$0.050 to $0.056

The times count from launch, when you press Enter. The console's own clock starts about 9 s after launch, so it reads a little lower (03:01, 05:50, 05:46, 08:15). The whole command took 8:26 from pressing Enter to the prompt, and cost $0.056216 in all, both arms and both reviews. The cap is $0.48 per A/B: two arms at $0.20 each and two reviews at $0.04 each. This run ran alone on the machine.

The ranges cover this run and a second run of the same command by a reviewer on a fresh clone (`ab-20261018-180654-f1ccbe`), each alone on the machine. That run has no recorded launch time, so its times count from its first event plus this run's 7 s. Running two demos at once makes both slower.

What to say in class

- "Scope in a ticket is a request. Only the harness arm had code that checked it, and today that code refused a patch to the shared errors module."
- "The guard enforces exactly what we measure, so the harness arm's zero is built in. The finding is the bare arm's count."
- "Today the bare arm stayed in scope. That is one dot. Before class it left the list in 9 of 10 runs."
- "In scope is not the same as correct. Both arms failed Feature 4's own bar, and the reviewer rejected both. Section 4: Verification adds the checks that would catch that."

CHAPTER 4

Section 4: Verification

A coding agent adds version checks to issue edits: an `ETag` on read and an `If-Match` check on write. The header is optional, so every existing test passes whether the check works or not. The agent sees a green suite and says "done". In this chapter you run `make show-4`: the harness arm alone, with all four layers, on Feature 4: Version checks on issue edits. You watch the verify gate (the checks that run when the agent says done), the test phase, and the test filters. Then the referee scores the run with 69 hidden tests and 4 planted faults the agent never sees. The lesson fits on one screen: the gate can say yes while the referee says no.

> Your output will differ: run times, ids, and the agent's wording change on every run.

The numbers, quotes and screenshots come from a live run of `make show-4` on 2026-10-18 that ran alone on the machine. Two things did not happen in that run: the gate never refused a "done", and the agent wrote no tests in the test phase, so the filters had nothing to check. Steps 8 and 18 to 20 show what you see when those happen, taken from two recorded runs of the same command and marked as such.

> **Terms in this chapter**
>
> - **Verify gate**: The checks that run in the agent's container when it claims done; a failure sends the claim back.
> - **Test phase**: A short second phase in which the agent writes tests for the feature it built.
> - **Test filters**: Five checks a test from the test phase must pass to be kept.
> - **Hidden tests**: Held-out tests: the agent never sees them; the referee runs them after the agent stops.
> - **Mutant**: A copy of the code with one small planted fault; a test that fails on it catches it.
> - **Referee mutants**: Planted faults the agent never sees; the referee counts how many the shipped tests catch.
> - **Blinded reviewer**: A second model that reads the diff without knowing which arm wrote it, and says accept or reject.

Step 1 Check that the console is running

WHERE: Terminal 1 and the browser

TYPE: Nothing new. The console from Part 0 is still running in Terminal 1. If it is not, start it:

TYPE THIS ⧉ COPY

```
make console
```

YOU SHOULD SEE: Terminal 1 shows `Console: http://127.0.0.1:8765/`. The browser tab shows the runs list.

LOOK AT: The last line in Terminal 1. It has no error, and the browser page loads.

WHAT IT MEANS: The console reads the run's event log as it grows. Start it before you launch, so you can open the run within seconds.

✓ CHECKPOINT
The runs list loads in the browser. Terminal 2 is free, in the repository root.

IF IT LOOKS DIFFERENT

- `address already in use` → another program holds port 8765: run `make console CONSOLE_PORT=8766` and open that port.
- The page does not load → Terminal 1 was closed: start `make console` again.

◆ INSTRUCTOR NOTE
In class this section opens with the one poll of the day (2:55 to 2:57). It picks the final feature. Close the poll before you launch.

Step 2 Start the run

WHERE: Terminal 2

TYPE:

TYPE THIS ⧉ COPY

```
make show-4 PURPOSE=pilot
```

YOU SHOULD SEE: make prints the command it runs, then nothing more until the end:

OUTPUT

```
uv run shipharness ab --section demo-4 --feature version-checks --arms harness   --purpose pilot \
  --profile class --max-tests=3
```

The `A/B ab-…` line and the result line come only at the end (Step 14).

LOOK AT: `--arms harness` (one arm only) and `--max-tests=3`.

WHAT IT MEANS: `make show-4` runs the harness arm alone, with the class profile. `--max-tests=3` is a demo setting: the filters check only the first three tests the test phase proposes. Any further test is removed with the reason "not filtered: this demo filters only the first 3 proposed tests" and counts as discarded. `make show-4 MAX_TESTS=` filters every test.

✓ CHECKPOINT
Terminal 2 shows the command and no `x` error line. In the live run the command took 5 min 33 s; a second run that also ran alone took 8 min 3 s.

IF IT LOOKS DIFFERENT

- `x SHIPHARNESS_CLASS_DATE is not set` → a class run needs the class day: set it in `.env`, or keep `PURPOSE=pilot`.
- `x OpenAI refused the key` → the key is missing or rejected: fix `OPENAI_API_KEY` in `.env`, then run `make doctor`.
- The arm ends `ERROR` with `docker run failed` → Docker is not running: `sudo systemctl start docker`, then `make warm`.

◆ INSTRUCTOR NOTE

Say what `MAX_TESTS=3` is before anyone asks: a demo setting that saves minutes, disclosed, not a comparison. In class, the plan in `docs/instructor-guide.md` launches the final A/B right after this command, in a third Ubuntu window opened like Terminal 2 (Part 0, Step 2). Every time in this guide was measured with each demo running alone; two runs at once make both slower.

Step 3 Open the new run

WHERE: Browser, runs list

TYPE: Reload the page (Ctrl+R). Click the newest card.

YOU SHOULD SEE:

> In class the new run is the top card. This picture was taken later, so two newer runs sit above it.

1. Run id: open the run
2. Type: harness in action or A/B
3. Status
4. Start time

LOOK AT: The type on the new run's card. It says "harness in action", not "A/B".

WHAT IT MEANS: One arm runs. This run shows the mechanism; it measures nothing.

✓ CHECKPOINT

The run opens. The run id at the top right matches the `ab-...` id that Terminal 2 prints in its `A/B` line.

IF IT LOOKS DIFFERENT

- No new card → the list does not refresh by itself: reload again after 10 to 20 seconds.
- Two new cards → the final A/B started too: pick the card with "harness in action".

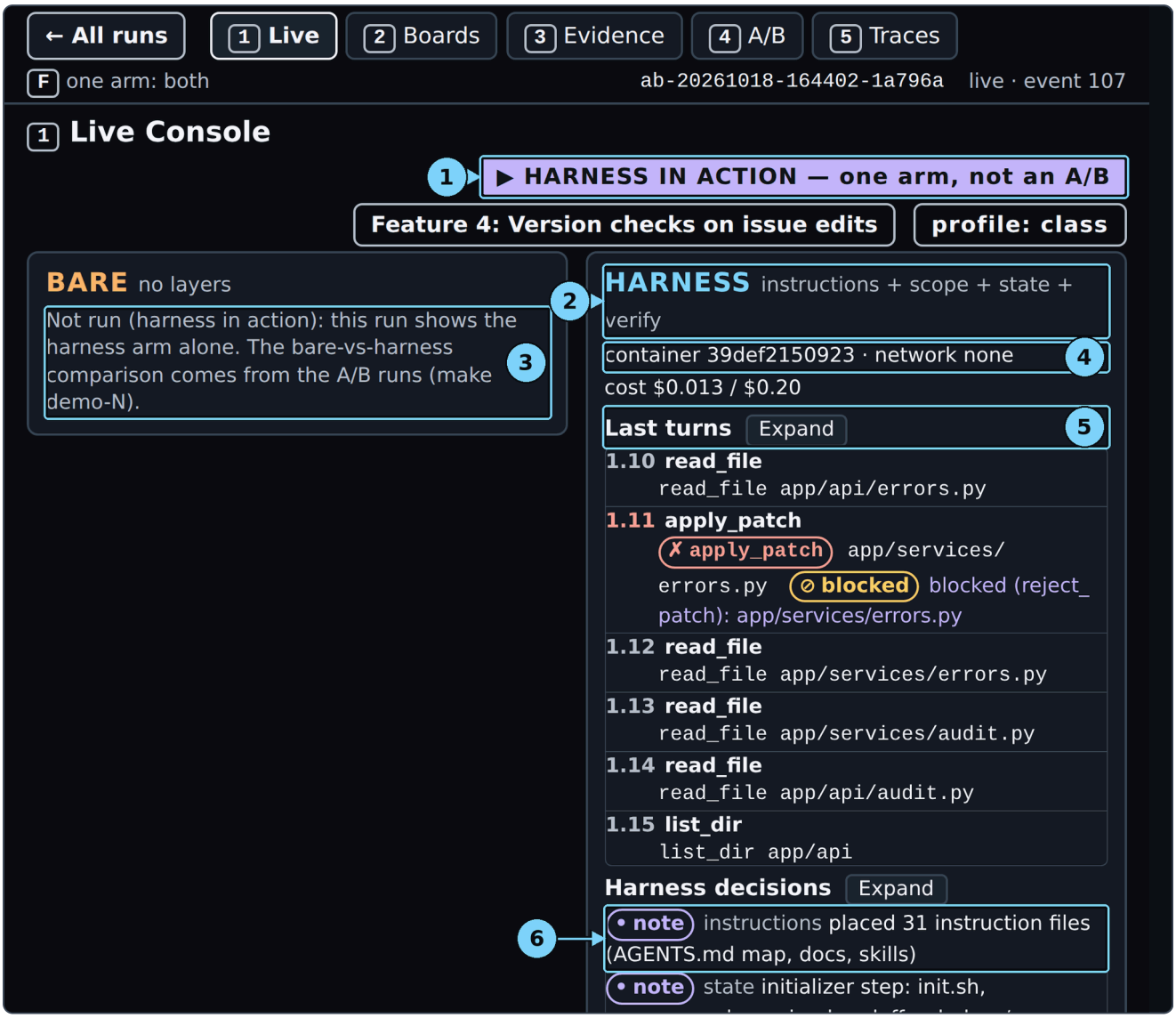
> **◆ INSTRUCTOR NOTE**
> Open this run before you launch anything else. The list puts the newest card first, and a second launch pushes this one down.

Step 4 Read the header of screen 1

WHERE: Browser, screen 1 Live

TYPE: Press 1

YOU SHOULD SEE:



Screen 1, about one minute in.

1. HARNESS IN ACTION badge, or the feature
2. Harness arm line: its layers
3. Bare arm: Not run (harness in action)
4. Container line
5. Last turns
6. Harness decisions: the layer notes

LOOK AT: The four layer names next to HARNESS (callout 2).

WHAT IT MEANS: Every layer from the course is on. The verify layer is the new one in this section.

✓ CHECKPOINT

The HARNESS column shows a container id, "network none" and a cost counter out of $0.20.

IF IT LOOKS DIFFERENT

- "no container yet" for more than a minute → the image is missing: stop the command (Ctrl+C), run `make warm`, start again.

◆ INSTRUCTOR NOTE

Point at the empty BARE column: "The comparison was made before class. Today we watch the harness work."

Step 5 Read the first harness decisions

WHERE: Browser, screen 1, "Harness decisions"

TYPE: Nothing. The notes appear within seconds of the container start.

YOU SHOULD SEE:

OUTPUT

```
note instructions placed 31 instruction files (AGENTS.md map, docs, skills)
note state initializer step: init.sh, progress.md, session-handoff.md, docs/exec-plans/active/F4.md
note scope feature_list.json written with F4 (passes: false)
note verify gate baseline reused (same checkpoint commit)
note instructions agent read ARCHITECTURE.md
```

LOOK AT: The note from the verify layer.

WHAT IT MEANS: The gate counts only new problems. It compares against the starting commit, so a problem that was already there does not block the agent.

✓ CHECKPOINT

The notes are there, and the turn list fills with turns `1.1`, `1.2`, and so on.

IF IT LOOKS DIFFERENT

- Fewer notes → scroll the decisions list, or click Expand next to "Harness decisions".

◆ INSTRUCTOR NOTE

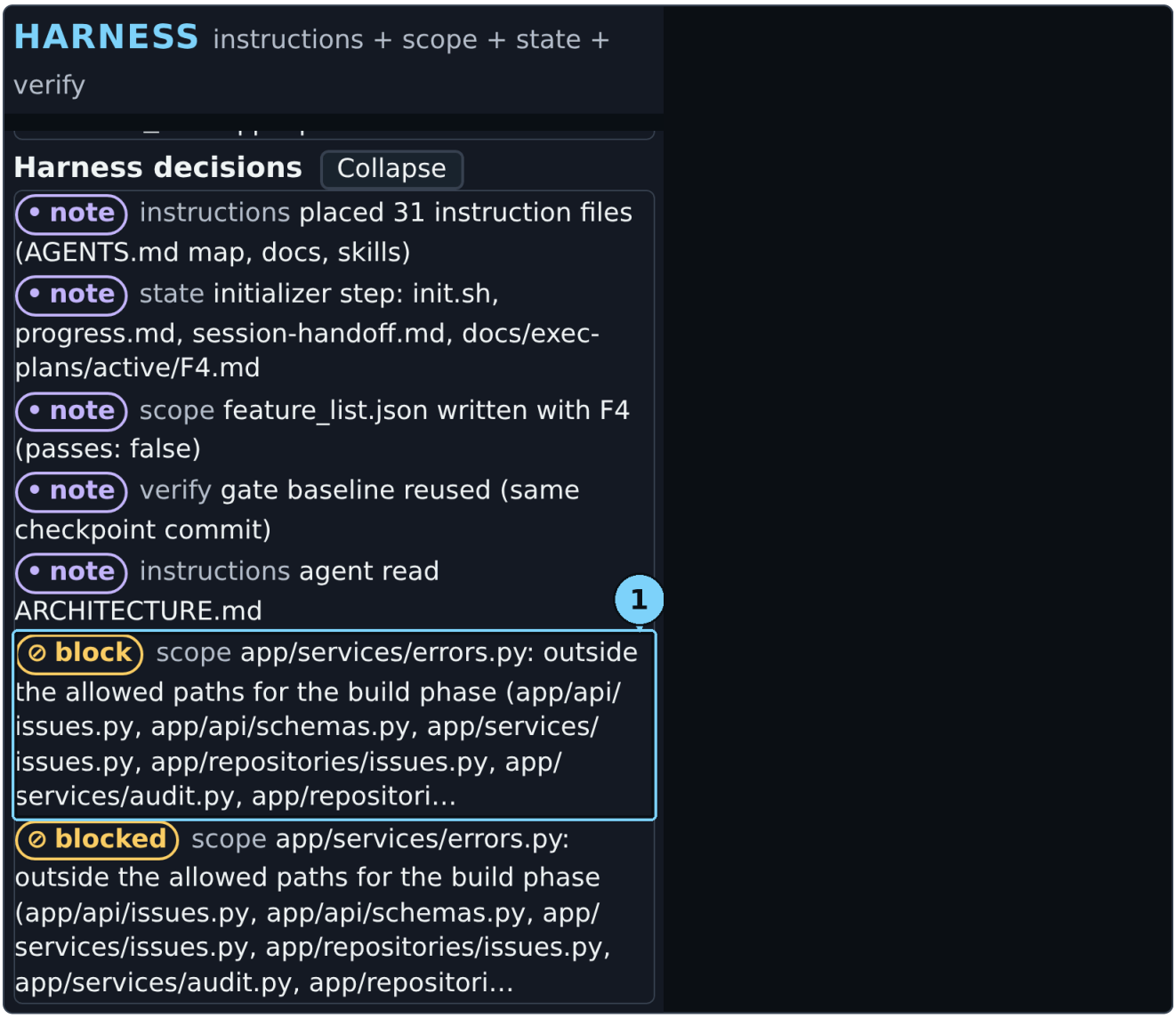
One sentence per layer, then move on. The audience has seen the first three layers in the earlier sections.

Step 6 Watch the scope layer block a write

WHERE: Browser, screen 1, the turn list and "Harness decisions"

TYPE: Nothing. In the live run this came 36 seconds after launch, at turn `1.11`.

YOU SHOULD SEE:



Screen 1: the scope layer refuses a patch outside the allowed files.

1 Turns list

LOOK AT: The block decision: `app/services/errors.py: outside the allowed paths for the build phase (...); patch not applied`.

WHAT IT MEANS: The agent tried to change a shared error module that the ticket does not need. The scope layer from Section 3: Scope and sandbox kept the change out. The agent goes on inside the allowed files.

✓ CHECKPOINT

One block decision appears. The run keeps going.

IF IT LOOKS DIFFERENT

- No block → the agent stayed in scope this time. Nothing is wrong; say so and go on.
- A different file is blocked → the agent chose another file. Read the file name aloud.

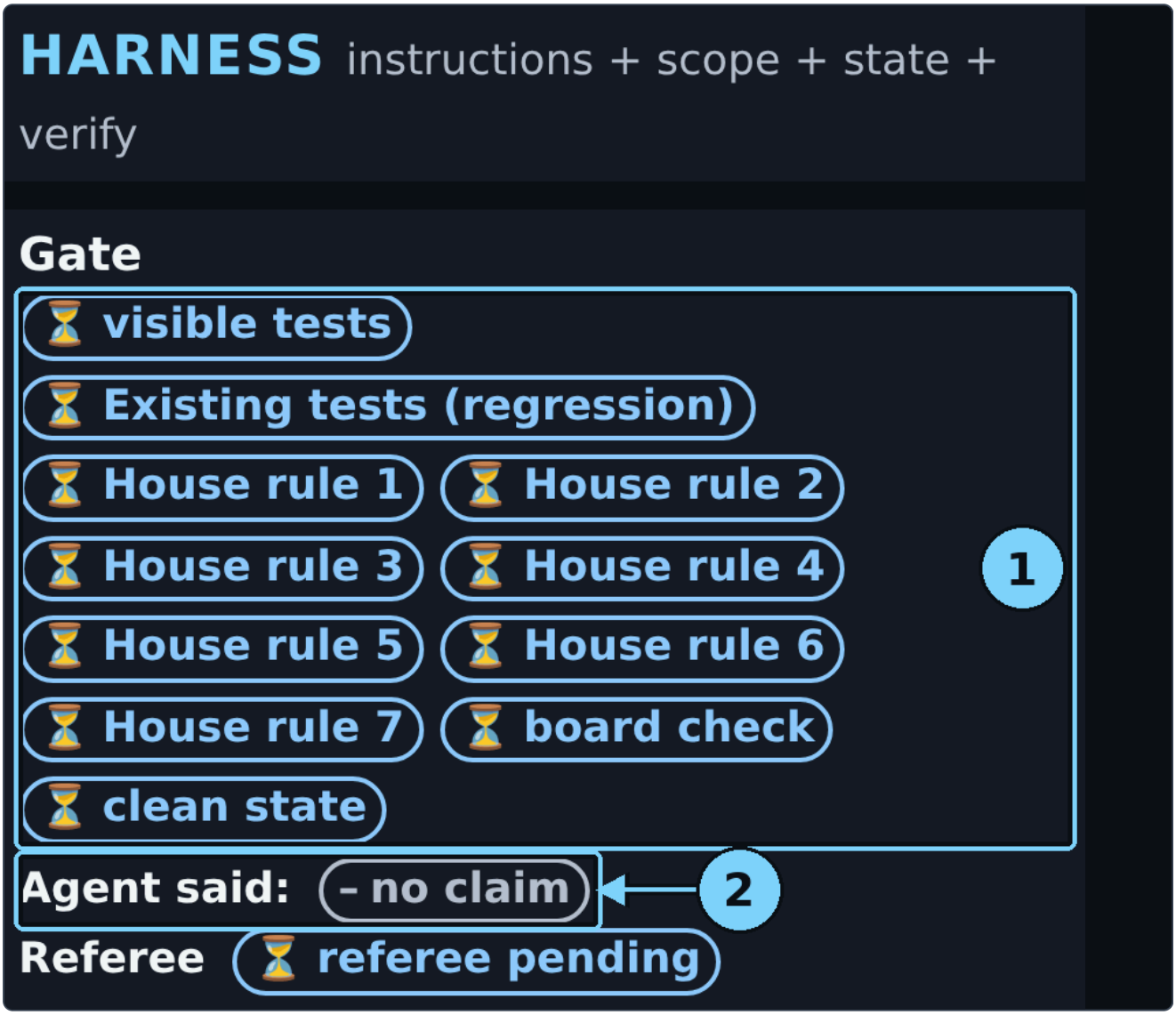
> **◆ INSTRUCTOR NOTE**
> Keep this short: "Section 3's layer, still at work." The new part comes next.

Step 7 Watch the agent claim done

WHERE: Browser, screen 1, "Gate"

TYPE: Nothing. In the live run the agent called `finish done` at turn `1.30`, 2 min 5 s after launch.

YOU SHOULD SEE:



Right after the done claim: every gate item is pending. "Agent said" shows the claim once the run ends.

1. Gate list: checks pending
2. Agent said

LOOK AT: The row of pending pills: visible tests, Existing tests (regression), House rule 1: Tenant isolation to House rule 7: Migrations (one pill each), board check, clean state.

WHAT IT MEANS: The agent said done. The verify layer does not take its word. It copies the work and runs eleven checks in the agent's container. The hidden tests and the referee mutants are not among them.

✓ CHECKPOINT

The pills turn from pending to ✓ or ✗ within about a minute.

IF IT LOOKS DIFFERENT

- No pills by the end → the agent never claimed done: go to Step 12 and read the referee counts.
- Every pill fails with `check runner failed` → the harness time limit ran out during the gate: keep the run and say what happened.

◆ INSTRUCTOR NOTE

Say: "The harness runs checks the agent does not control. A failure goes back to the agent with the first three reasons."

Step 8 If the gate refuses "done"

WHERE: Browser, screen 1, below the turn list

TYPE: Nothing.

YOU SHOULD SEE: In the live run the first claim passed, so there is no refusal line. When the gate refuses, a red line appears here. This one is from a recorded run of the same command:

OUTPUT

```
refused Gate refused 'done' (build phase, attempt 1): clean state: ruff: app/web/routes.py:193:8:
F821 Undefined name `version` → passed on attempt 2
```

LOOK AT: The failed item ("clean state"), its reason, and what came next ("passed on attempt 2").

WHAT IT MEANS: One failed item fails the gate. The agent gets "Not accepted." plus a short summary and the first three failures. It fixes them, commits, and calls `finish` again. In that recorded run the second claim passed every item.

✓ CHECKPOINT

Either no refusal line (the gate passed at once, as in the live run), or a refusal line that ends with "passed on attempt 2".

IF IT LOOKS DIFFERENT

- The line ends "still refused on attempt 2" → the agent did not fix it in time: keep going, the referee still judges the run.
- The line ends "no further attempt" → the run ended before a second claim: same as above.

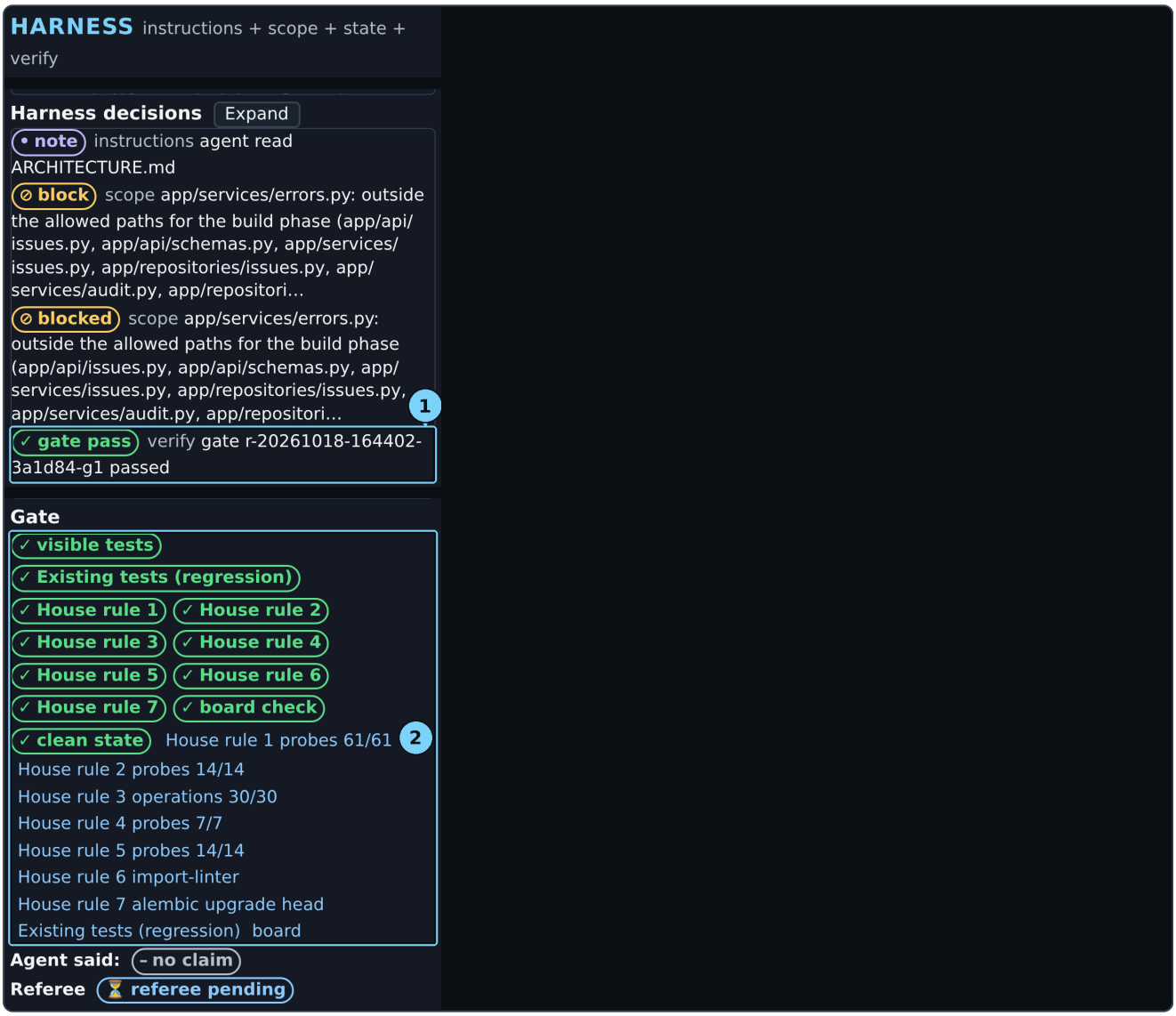
> **◆ INSTRUCTOR NOTE**
> If you see a refusal, stop here for ten seconds: "done" came back with a reason, and the agent fixed it. If you see none, say: "This time the first claim passed. When it does not, a red line appears right here."

Step 9 Read the passed gate

WHERE: Browser, screen 1, "Gate" and "Harness decisions"

TYPE: Nothing. In the live run every item passed 3 min 7 s after launch.

YOU SHOULD SEE:



> The gate passed every item at the first claim. The referee has not started yet.

1. Gate passed
2. Gate list: every check passed

LOOK AT: Eleven ✓ pills, and three decisions right after them:

OUTPUT

```
gate pass  verify gate r-20261018-164402-3a1d84-g1 passed
status_set scope F4 passes: r-20261018-164402-3a1d84-g1 at 8c243a5
note       state exec plan moved to docs/exec-plans/completed/F4.md (F4 passes)
```

WHAT IT MEANS: Hover over "visible tests": `passed=190 failed=0 errors=0`. Only a passed gate sets `passes` to true in `feature_list.json`, with the gate id and the commit as evidence. The agent cannot set it itself. The test phase starts next.

✓ CHECKPOINT

Eleven ✓ pills. "Agent said:" still shows "– no claim" (the claims appear when the run ends), and the Referee line says "referee pending".

IF IT LOOKS DIFFERENT

- A pill shows ✗ → open its tooltip for the reason; the agent gets it too and tries again (Step 8).

◆ INSTRUCTOR NOTE

Say: "Every check the agent could see is green. Remember this screen; the referee has not spoken yet."

Step 10 Watch the test phase

WHERE: Browser, screen 1, the turn list and "Harness decisions"

TYPE: Nothing. In the live run the test phase ran from 3 min 7 s to 4 min 21 s after launch.

YOU SHOULD SEE: "Session 1 stopped", then turns `2.1` to `2.12`. In the live run the agent read the spec and the existing tests, ran the visible tests, updated `progress.md`, and called `finish not_done` on its last turn. Then the verify note:

OUTPUT

```
note verify test filters: kept 0, discarded 0; no new tests to filter
```

LOOK AT: "no new tests to filter".

WHAT IT MEANS: The test phase has 12 turns. In it the agent should write tests for its own feature, and five filters decide which ones ship. This time the agent wrote none. Its own summary says so: "no new regression tests have been added". So the filters had nothing to check, and no new test ships with the feature.

✓ CHECKPOINT

The note "test filters: kept N, discarded M" appears, then "run completed".

IF IT LOOKS DIFFERENT

- "kept 1, discarded 4" or similar → the agent wrote tests. Screen 3 shows each one (Steps 18 to 20).
- "kept 0, discarded 5" → every filtered test failed a filter. A recorded run did exactly that; screen 3 shows why.

◆ INSTRUCTOR NOTE

Say: "The agent used its 12 test-phase turns without writing a test, and it said so: not done. Watch what that costs at the referee."

Step 11 Read the end of the agent's run

WHERE: Browser, screen 1, top of the HARNESS column

TYPE: Nothing.

YOU SHOULD SEE: `cost $0.023 / $0.20 · ✓ run completed · 42 turns`.

LOOK AT: "run completed" and the turn count.

WHAT IT MEANS: The agent stopped normally: 30 turns of build, then 12 turns of test phase. The referee starts now, in a fresh container.

✓ CHECKPOINT

The Referee line shows "referee pending", then the primary verdict pill within about 20 seconds.

IF IT LOOKS DIFFERENT

- `run ABORTED: timeout` → the agent hit its 420-second limit. The run is still judged; read it the same way.

◆ INSTRUCTOR NOTE

While the referee runs, ask the room: "The gate passed everything. Will the referee pass it?"

Step 12 Read the primary verdict

WHERE: Browser, screen 1, "Referee"

TYPE: Nothing. In the live run this came 4 min 37 s after launch (the console's clock starts about 8 s after launch, so it shows 04:29).

YOU SHOULD SEE:

The gate passed, and the referee's primary verdict is FAIL. The secondary checks still run.

1. Referee: secondary checks running
2. Referee pill (harness): primary verdict ready: FAIL

LOOK AT: The pill "✗ primary verdict ready: FAIL" and the values after it: "hidden tests passed 68 · mutants killed 0".

WHAT IT MEANS: The run passes only if all 69 hidden tests pass and the shipped tests catch all 4 referee mutants. One hidden test failed, and no shipped test caught any of the 4 mutants. So the answer is FAIL.

✓ CHECKPOINT

The pill is there and shows PASS or FAIL, followed by "secondary checks running".

IF IT LOOKS DIFFERENT

- The pill says PASS → both counts were full this time. Read it the same way; one run is one run.

◆ INSTRUCTOR NOTE

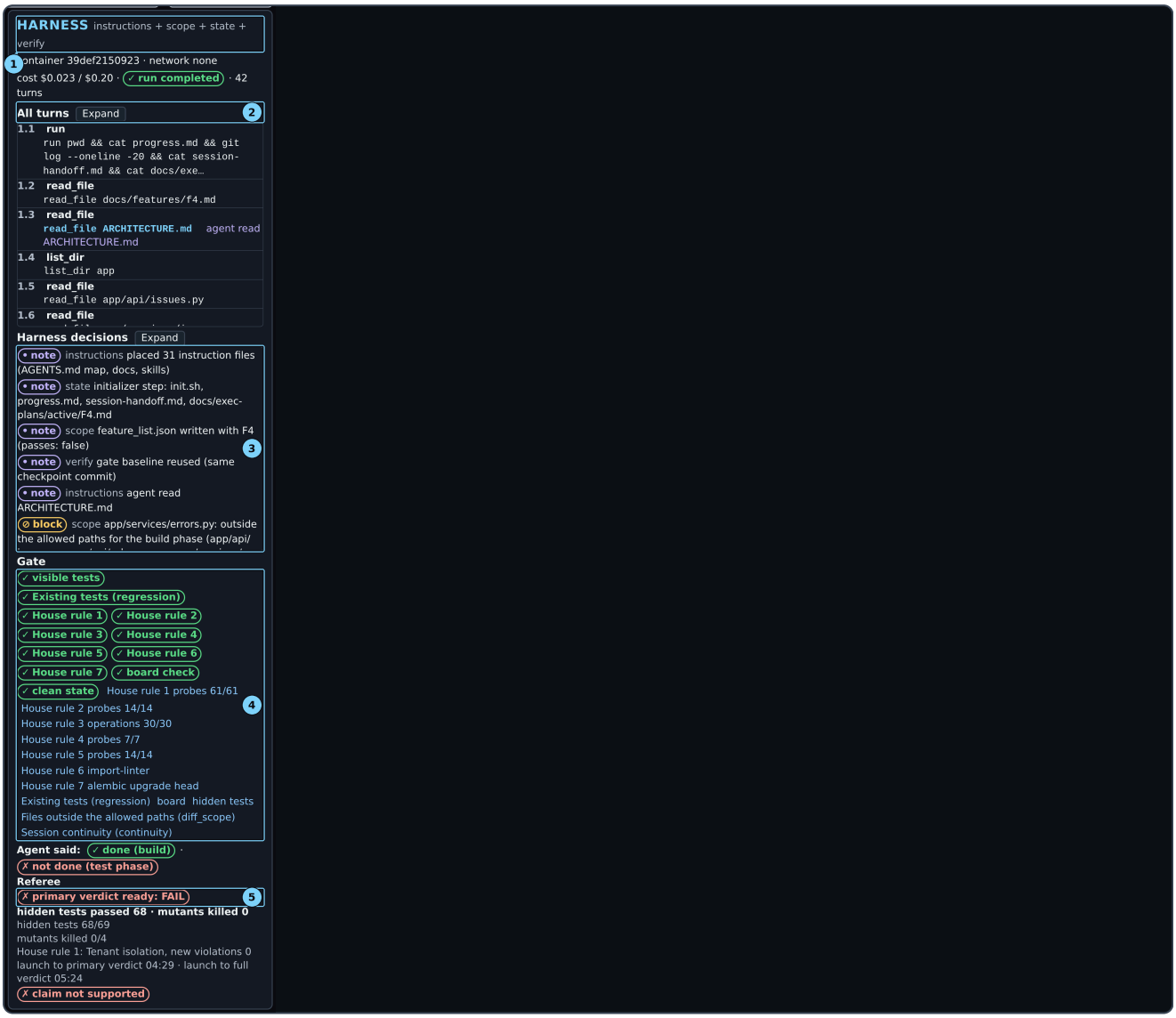
Say: "The gate said yes. The referee said no." Then explain why both can be true: the gate runs only what the agent can see. The hidden tests and the referee's mutants stay out of its reach, so they can still catch what the agent missed. This is not a broken harness. It is the harness showing where its checks stop.

Step 13 Read the full verdict and "Agent said"

WHERE: Browser, screen 1, "Agent said" and "Referee"

TYPE: Nothing. In the live run the full verdict came 5 min 32 s after launch (05:24 on the console's clock).

YOU SHOULD SEE:



Screen 1, the end of the run: the gate passed, and the referee failed the run.

1. Harness arm line: its layers
2. Turns list
3. Harness decisions
4. Gate list
5. Referee pill (harness)

LOOK AT: "Agent said: ✓ done (build) · ✗ not done (test phase)" and, under the referee, "✗ claim not supported".

WHAT IT MEANS: "Agent said:" shows the claim of each phase. The build said done, and the gate accepted it; the test phase said not done, because it wrote no tests. The referee disagrees with the build claim, and "claim not supported" puts the two side by side. The other checks (house rules, existing tests, scope) passed.

✓ CHECKPOINT

Both timings are shown, and "secondary checks running" is gone.

IF IT LOOKS DIFFERENT

- Only the primary timing → the full verdict is still running: wait up to a minute.

◆ INSTRUCTOR NOTE

How to talk about this FAIL: "The gate said yes, the referee said no. The gate checks what the agent can see. The referee checks what it cannot." Then: "Before class the bare arm failed this ticket too, in every run. So this FAIL is about the task and this model, not a verdict on the harness."

Step 14 Read the terminal's final lines

WHERE: Terminal 2

TYPE: Nothing. Wait for the prompt to come back.

YOU SHOULD SEE:

Terminal 2

```
$ make show-4 PURPOSE=pilot
… 3 lines above
  harness  completed         agent said not done · hidden tests passed: 68; mutants killed: 0 ·
primary measure failed · hidden tests 68 of 69 · mutants killed 0 of 4 · reviewer reject · cost
$0.0225
Cost: $0.025731 (this command: gpt-5.6-luna $0.003204, gpt-6-luna $0.022527; session $0.145965 of
$2.50; build $0.145965 of $15.00 ($14.854035 left))
```

The last lines in Terminal 2, captured on 2026-10-18. The arm line now reads "agent said: done (build) · not done (test phase)".

OUTPUT

```
A/B ab-20261018-164402-1a796a (Feature 4: Version checks on issue edits, layers=['instructions',
'scope', 'state', 'verify'], profile=class, harness in action)
  harness  completed         agent said not done · hidden tests passed: 68; mutants killed: 0 ·
primary measure failed · hidden tests 68 of 69 · mutants killed 0 of 4 · reviewer reject · cost
$0.0225
Cost: $0.025731 (this command: gpt-5.6-luna $0.003204, gpt-6-luna $0.022527; session $0.145965 of
$2.50; build $0.145965 of $15.00 ($14.854035 left))
```

LOOK AT: The arm line: status, claim, the two primary values, "primary measure failed", "reviewer reject", and the cost.

WHAT IT MEANS: The terminal says what screen 1 says, with the claim of each phase: "agent said: done (build) · not done (test phase)". The `Cost:` line splits the command's cost between the agent model and the reviewer model.

✓ CHECKPOINT

The prompt is back. The command exits with 0 when the arm completed (as here), 2 when it ended `ABORTED`, 1 on `ERROR`.

IF IT LOOKS DIFFERENT

- `make: *** [...] Error 2` → the arm ended `ABORTED: timeout`. It is still judged; read the line as usual.

◆ INSTRUCTOR NOTE

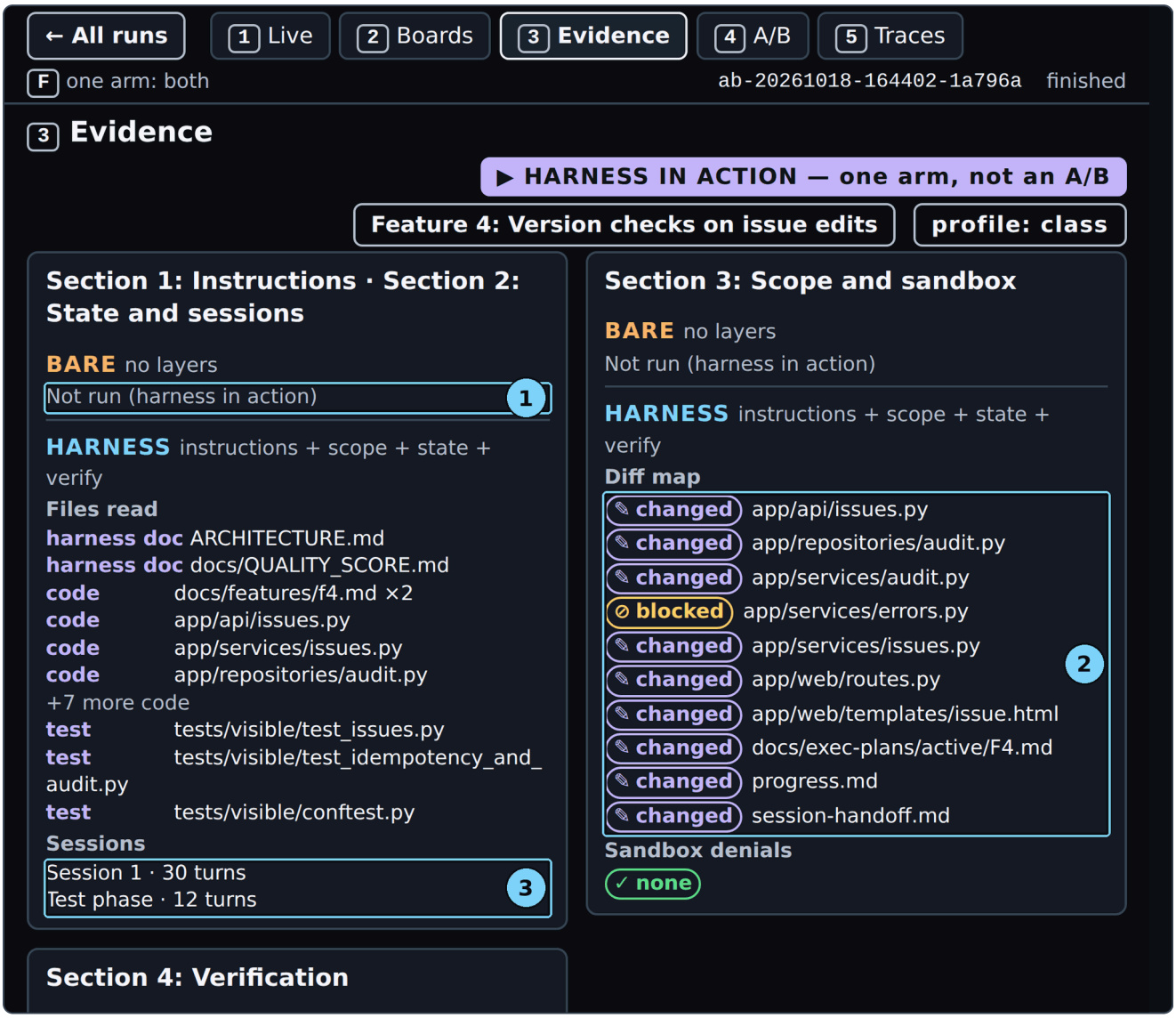
Read the two claims aloud: "The build said done and passed the gate. The test phase said not done, because it wrote no tests."

Step 15 Open screen 3 and read the sessions and the diff map

WHERE: Browser, screen 3 Evidence, the first two panels

TYPE: Press 3

YOU SHOULD SEE:



Screen 3: what the agent read and changed. The referee's checks are in the panel below.

1. Bare arm: Not run (harness in action)
2. Diff map (harness)
3. Sessions, including the test phase

LOOK AT: The two session rows and the diff map.

WHAT IT MEANS: The build took 30 turns and ended with the done claim the gate accepted. The test phase ended on its 12th turn, with the claim "not done". The diff map has no test file: the test phase only changed `progress.md`, `session-handoff.md` and the exec plan.

✓ CHECKPOINT

Two session rows and one blocked file in the diff map.

IF IT LOOKS DIFFERENT

- The test phase row is missing → the run ended before the test phase.

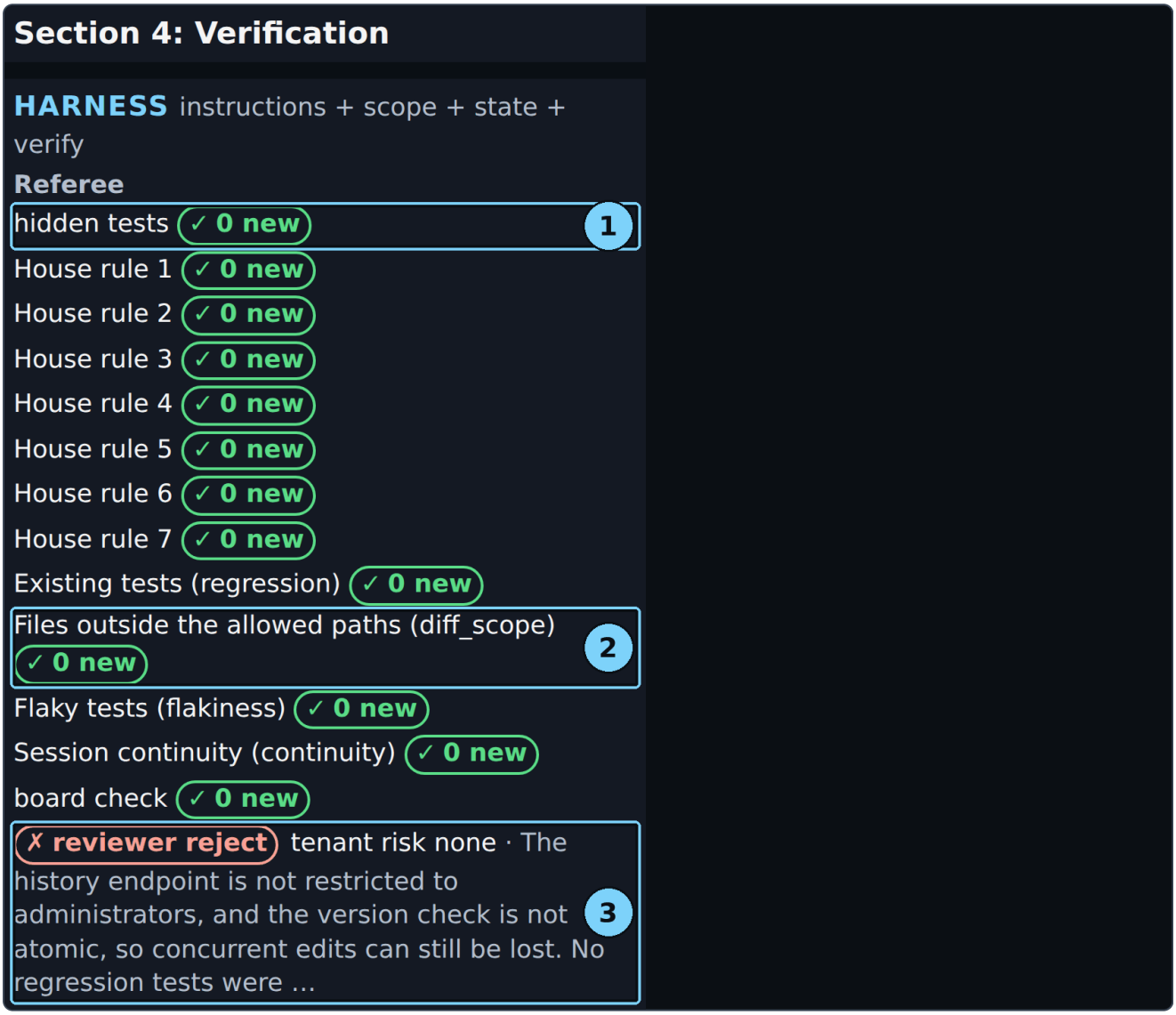
> **◆ INSTRUCTOR NOTE**
> Spend five seconds here. The verification panel on the right is the point of this screen.

Step 16 Read the referee checks

WHERE: Browser, screen 3, panel "Section 4: Verification", "Referee"

TYPE: Nothing.

YOU SHOULD SEE:



Screen 3: the referee's checks, then the blinded reviewer.

1. hidden tests (harness)
2. Files outside the allowed paths (diff_scope), harness
3. Blinded reviewer's reason (harness)

LOOK AT: Each row's pill: "hidden tests", "House rule 1" to "House rule 7", "Existing tests (regression)", "Files outside the allowed paths (diff_scope)", "Flaky tests (flakiness)", "Session continuity (continuity)", "board check", each "✓ 0 new".

Hover over a row for the full name and the number of probes.

WHAT IT MEANS: Each row counts new failures against the starting code, before the agent's change. That is why "hidden tests" shows "✓ 0 new" while one hidden test fails. At the starting commit the feature does not exist, and 66 of the 69 hidden tests fail there. The one that still fails, `test_invalid_body_with_a_stale_version_is_412`, is one of those 66. It failed before the agent started and it fails now, so it is not new. The verdict counts passes, not new failures: 68 of 69, on screen 1. "Flaky tests (flakiness)" checked 0 tests, because no new test shipped.

✓ CHECKPOINT

Thirteen check rows, and a reviewer line under them.

IF IT LOOKS DIFFERENT

- "running" pills → the referee has not finished: wait for the full verdict on screen 1.

◆ INSTRUCTOR NOTE

If someone asks why "hidden tests" shows ✓ at 68 of 69: "This row asks: did the agent break a hidden test that passed before? No. The test that fails never passed; the agent did not fix it. The verdict counts that."

Step 17 Read the reviewer's reject

WHERE: Browser, screen 3, the reviewer line under the referee checks

TYPE: Nothing. Hover over the line for the whole text.

YOU SHOULD SEE:

OUTPUT

```
✗ reviewer reject  tenant risk none · The history endpoint is not restricted to administrators, and
the version check is not atomic, so concurrent edits can still be lost. No regression tests were …
```

LOOK AT: The one-line reason after "·".

WHAT IT MEANS: A second model read the diff without knowing which arm wrote it. It found three problems. The version check is a read followed by a separate update, so two writers can still overwrite each other. The history endpoint lets members read the audit log, and the house rules say only admins may. And no test shipped. The reviewer does not decide the verdict. It is one more view next to the referee.

✓ CHECKPOINT

The line shows "reviewer accept" or "reviewer reject", with a reason.

IF IT LOOKS DIFFERENT

- No reviewer line → the review is still running, or it hit its cost cap: wait for the full verdict.

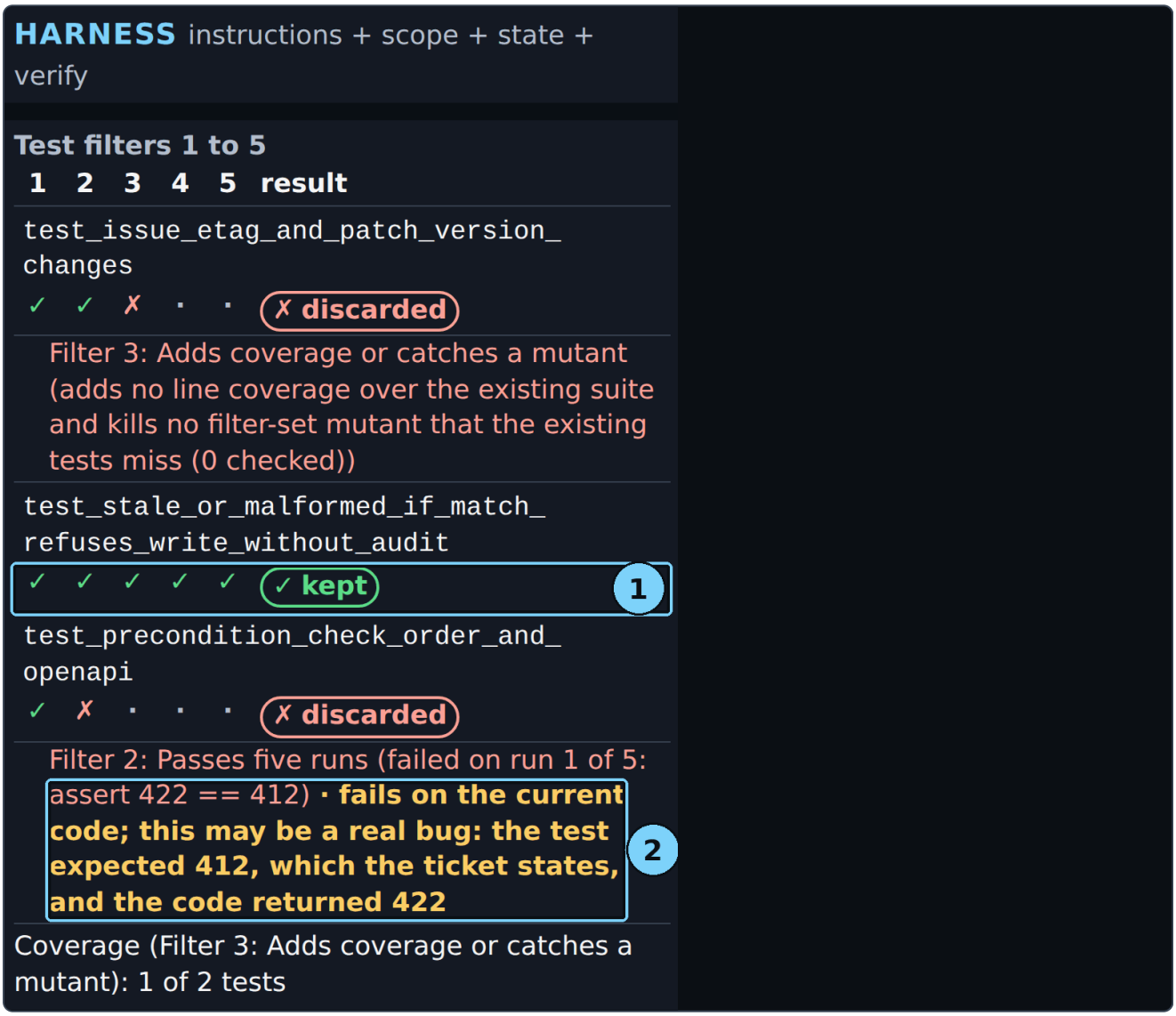
> **◆ INSTRUCTOR NOTE**
> Read the reason aloud. Then: "The reviewer found a real gap that 190 visible tests and every gate item missed."

Step 18 If you see a filter table: the kept test

WHERE: Browser, screen 3, "Test filters 1 to 5"

TYPE: Nothing. Scroll down in the right panel.

YOU SHOULD SEE: In the live run there is no filter table: the agent wrote no tests (Step 10). When the test phase writes tests, the table looks like this one, from a recorded run of the same command:



Recorded run, screen 3: one row per filtered test, five filter marks, and the result.

1. A test kept by the filters
2. fails on the current code; this may be a real bug

LOOK AT: Callout 1: `test_stale_or_malformed_if_match_refuses_write_without_audit`, ✓ ✓ ✓ ✓ ✓, kept.

WHAT IT MEANS: The kept test passed every filter. Filter 1: Builds and collects. Filter 2: Passes five runs. Filter 3: Adds coverage or catches a mutant (it added 10 covered lines). Filter 4: Real assertions. Filter 5: No source changes. It ships with the feature. In that run the filters checked three of the five proposed tests; the other two were "not filtered" and do not appear in the table.

✓ CHECKPOINT

Either no table and the note "no new tests to filter" (the live run), or one row per filtered test.

IF IT LOOKS DIFFERENT

- Every row says discarded → that happens: another recorded run kept 0 of 5.

◆ INSTRUCTOR NOTE

If the live run has no table, show this page and say: "When the agent writes tests, this is where they are judged." Name the filters once: "Filters 1 to 3 come from Automated Unit Test Improvement using Large Language Models at Meta. Filters 4 and 5 are the course's own."

Step 19 If you see a filter table: a Filter 3 discard

WHERE: Browser, screen 3, the filter table

TYPE: Nothing.

YOU SHOULD SEE: The first row of the recorded table, `test_issue_etag_and_patch_version_changes`, ✓ ✓ ✗, discarded, with this reason:

OUTPUT

```
Filter 3: Adds coverage or catches a mutant (adds no line coverage over the existing suite and kills
no filter-set mutant that the existing tests miss (0 checked))
```

LOOK AT: "adds no line coverage".

WHAT IT MEANS: The test passed, five runs out of five. But it covered no line that the existing tests do not already cover, and it caught no extra mutant. It adds cost and no protection, so it does not ship.

✓ CHECKPOINT

The row shows ✓ ✓ ✗ and the reason line.

IF IT LOOKS DIFFERENT

- The reason is cut off → hover over it, or press F to show the arm full screen.

◆ INSTRUCTOR NOTE
Say: "A passing test is not a useful test. This filter asks what the test adds."

Step 20 If you see a filter table: "this may be a real bug"

WHERE: Browser, screen 3, the filter table

TYPE: Nothing.

YOU SHOULD SEE: The last row of the recorded table (callout 2), `test_precondition_check_order_and_openapi`, ✓ ✗, discarded:

OUTPUT

```
Filter 2: Passes five runs (failed on run 1 of 5: assert 422 == 412) · fails on the current code;
this may be a real bug: the test expected 412, which the ticket states, and the code returned 422
```

LOOK AT: "the test expected 412, which the ticket states, and the code returned 422".

WHAT IT MEANS: The filters treat the current code as correct. A test that fails on it is thrown away, even when the test is right. Here the test checked the ticket's order of checks: a stale `If-Match` must give 412 before body validation gives 422. The code gave 422. The agent wrote the right test, and the filter discarded it. The live run fails the same case: its one failing hidden test is `test_invalid_body_with_a_stale_version_is_412`. The authors of Mutation-Guided LLM-based Test Generation at Meta name the limit: "it cannot detect existing faults residing in the code base".

✓ CHECKPOINT
If your run has such a row, it shows ✓ ✗ and the reason with the "may be a real bug" note.

IF IT LOOKS DIFFERENT

- No "may be a real bug" note → the failed assertion does not match a value the ticket states. The console then claims nothing; read the plain reason.
- Another recorded run showed `assert 200 == 412`: the test expected 412 and the code accepted the stale write.

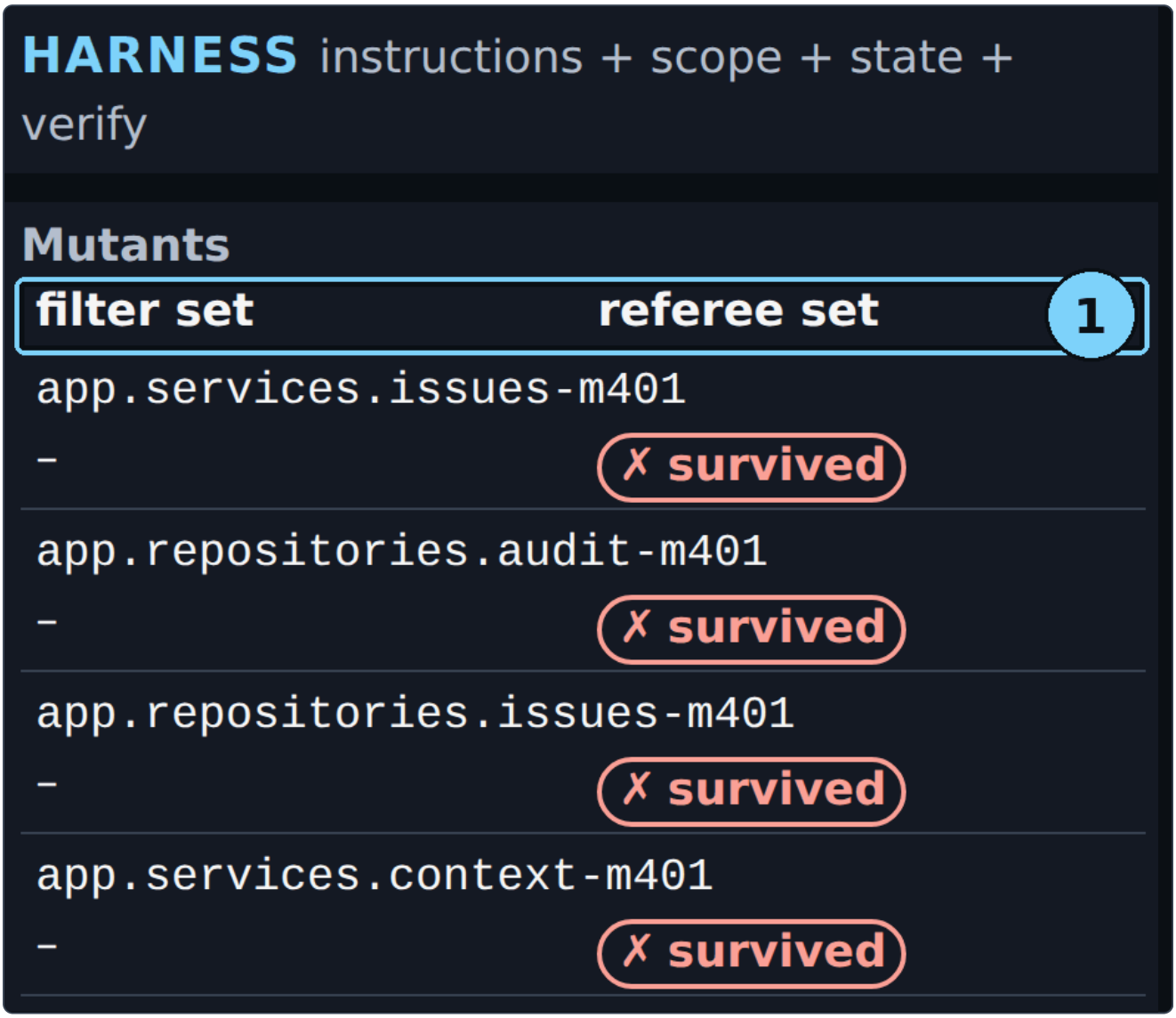
◆ INSTRUCTOR NOTE
This is the deepest point of the section. Say: "The filters keep regression tests: tests that guard what the code does now. They cannot tell a wrong test from a right test that found a bug. The console flags it; a person has to read it."

Step 21 Read the mutants table

WHERE: Browser, screen 3, "Mutants"

TYPE: Nothing. Scroll to the bottom of the right panel.

YOU SHOULD SEE:



Screen 3: the four referee mutants and what the shipped tests did to them.

1 Mutants (harness): filter set and referee set

OUTPUT

```
mutant                        filter set   referee set
app.services.issues-m401      –            ✗ survived
app.repositories.audit-m401   –            ✗ survived
app.repositories.issues-m401  –            ✗ survived
app.services.context-m401     –            ✗ survived
```

LOOK AT: The two columns, "filter set" and "referee set".

WHAT IT MEANS: There are two sets of mutants. The filter set belongs to Filter 3, inside the harness; this run used none, because no test reached the filters. The referee set is held out: the agent and the filters never see it. Each referee mutant puts one small fault in the code, and the referee runs the shipped tests on it. "survived" means no shipped test failed, so the fault went unnoticed. No new test shipped, so all 4 survived: 0 of 4 caught.

✓ CHECKPOINT

Four referee mutants, none "killed". This matches "mutants killed 0/4" on screen 1.

IF IT LOOKS DIFFERENT

- A row says "killed" → a shipped test caught that fault. Count the killed rows against the number on screen 1.
- A row says "error" → that mutant did not run cleanly and counts as not caught. A recorded run had one.
- The filter table shows a kept test, and every referee mutant still says "survived" → the kept test caught none of the held-out faults. See the instructor note.

◆ INSTRUCTOR NOTE

Say: "No test written, no fault caught. The gate cannot see this; the referee can."

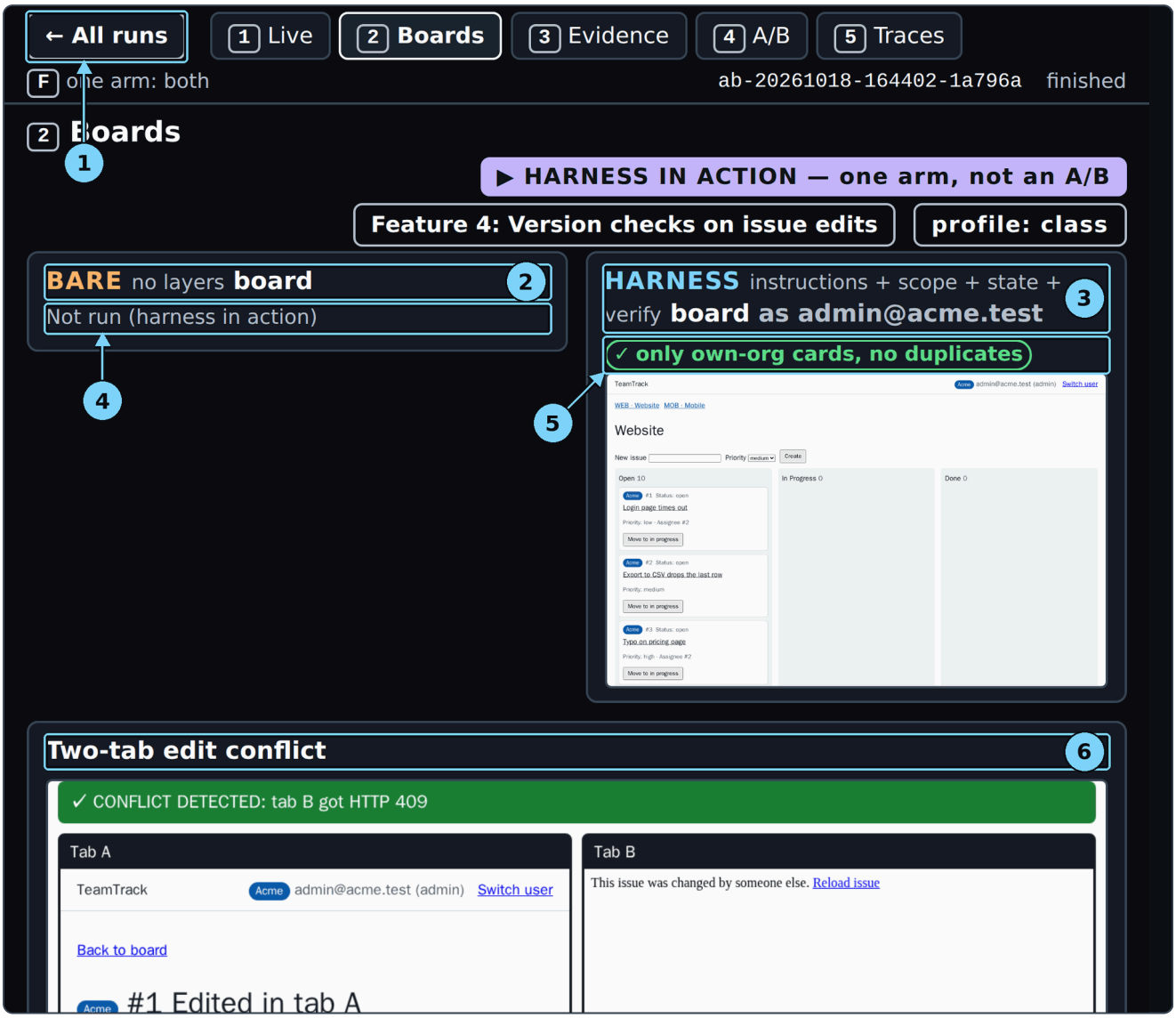
If a test was kept and the referee set still shows 0 killed: the test passed Filter 3 because it covered lines the suite did not, or caught a mutant of the filter set. Neither means it checks the behaviour the referee's planted faults break. The referee set is held out, so the filters never saw it. Say: "The filters kept this test because it reaches new code. Reaching code is not checking it: none of the referee's planted faults made it fail. A kept test is a better test, not proof the feature is guarded."

Step 22 Open screen 2 and read the two-tab conflict

WHERE: Browser, screen 2 Boards

TYPE: Press 2

YOU SHOULD SEE:



Screen 2: the board stops the lost update from a stale tab.

1. ← All runs: back to the run list
2. Bare board heading
3. Harness board heading: signed in as one user
4. Bare arm: Not run (harness in action)
5. Harness board marks
6. Two-tab edit conflict

LOOK AT: Callout 6, and tab B's page: "This issue was changed by someone else."

WHAT IT MEANS: Tab A saves, then tab B saves from its stale form. Tab B gets HTTP 409 and a "changed by someone else" page. The board part of the feature works. The referee still found one hidden test failing at the API.

✓ CHECKPOINT

The conflict panel shows "CONFLICT DETECTED".

IF IT LOOKS DIFFERENT

- No conflict panel → the conflict screenshot comes after the full verdict: wait a few seconds and press 2 again.

◆ INSTRUCTOR NOTE

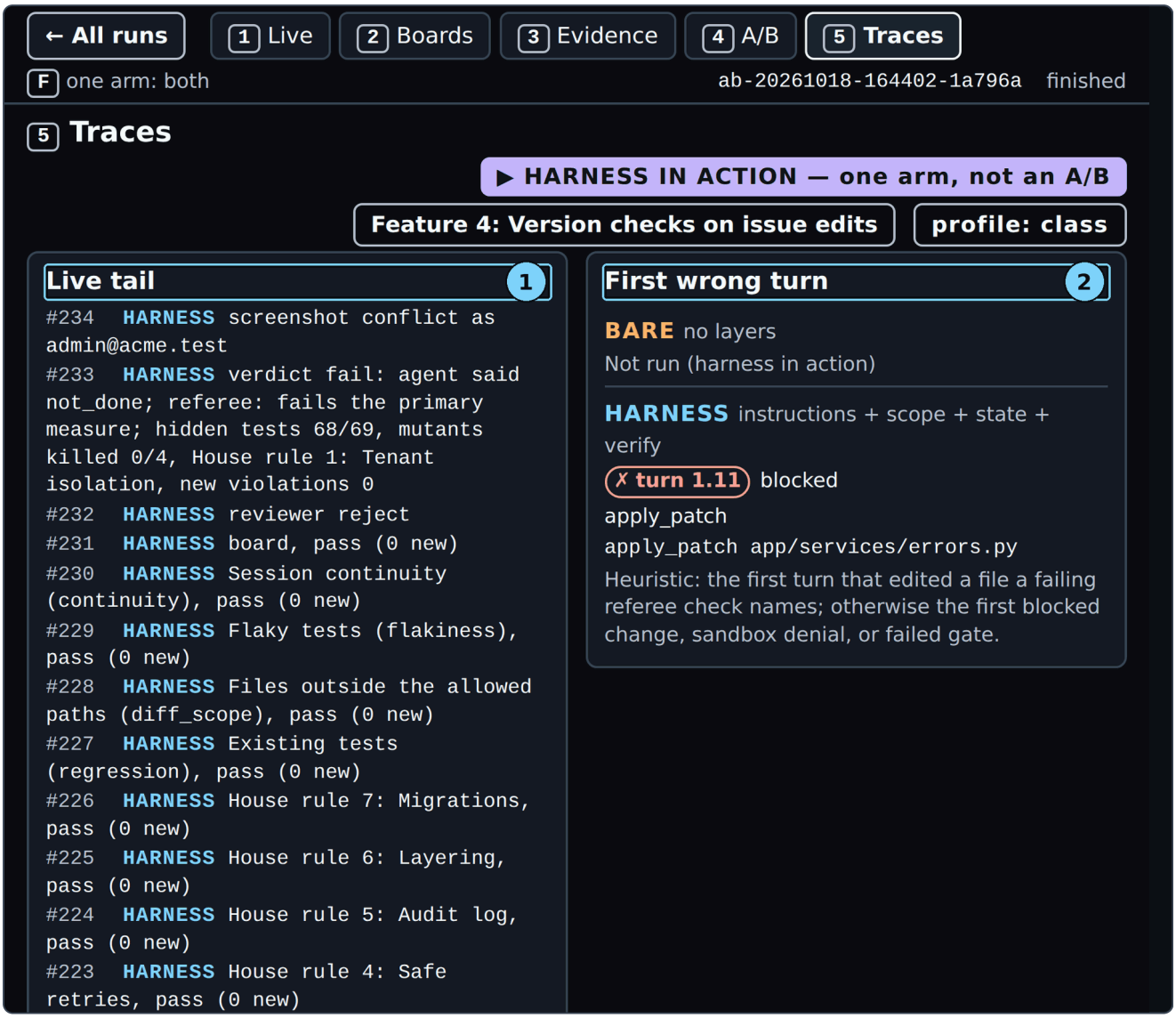
Say: "Two tabs edit the same issue; the second save is refused. The board check runs on every arm."

Step 23 Open screen 5 and read the traces

WHERE: Browser, screen 5 Traces

TYPE: Press 5

YOU SHOULD SEE:



Screen 5: where the run first went wrong. The LangSmith links are below the bottom edge of this picture.

1. Live tail
2. First wrong turn

LOOK AT: The first wrong turn.

WHAT IT MEANS: The console's rule takes the first turn that edited a file a failing referee check names; otherwise the first blocked change. No referee check row failed here, so it shows the blocked write.

✓ CHECKPOINT

Two LangSmith links are listed.

IF IT LOOKS DIFFERENT

- "No trace link yet" → tracing is off or the key is missing: the run is still valid; skip the links.

◆ INSTRUCTOR NOTE

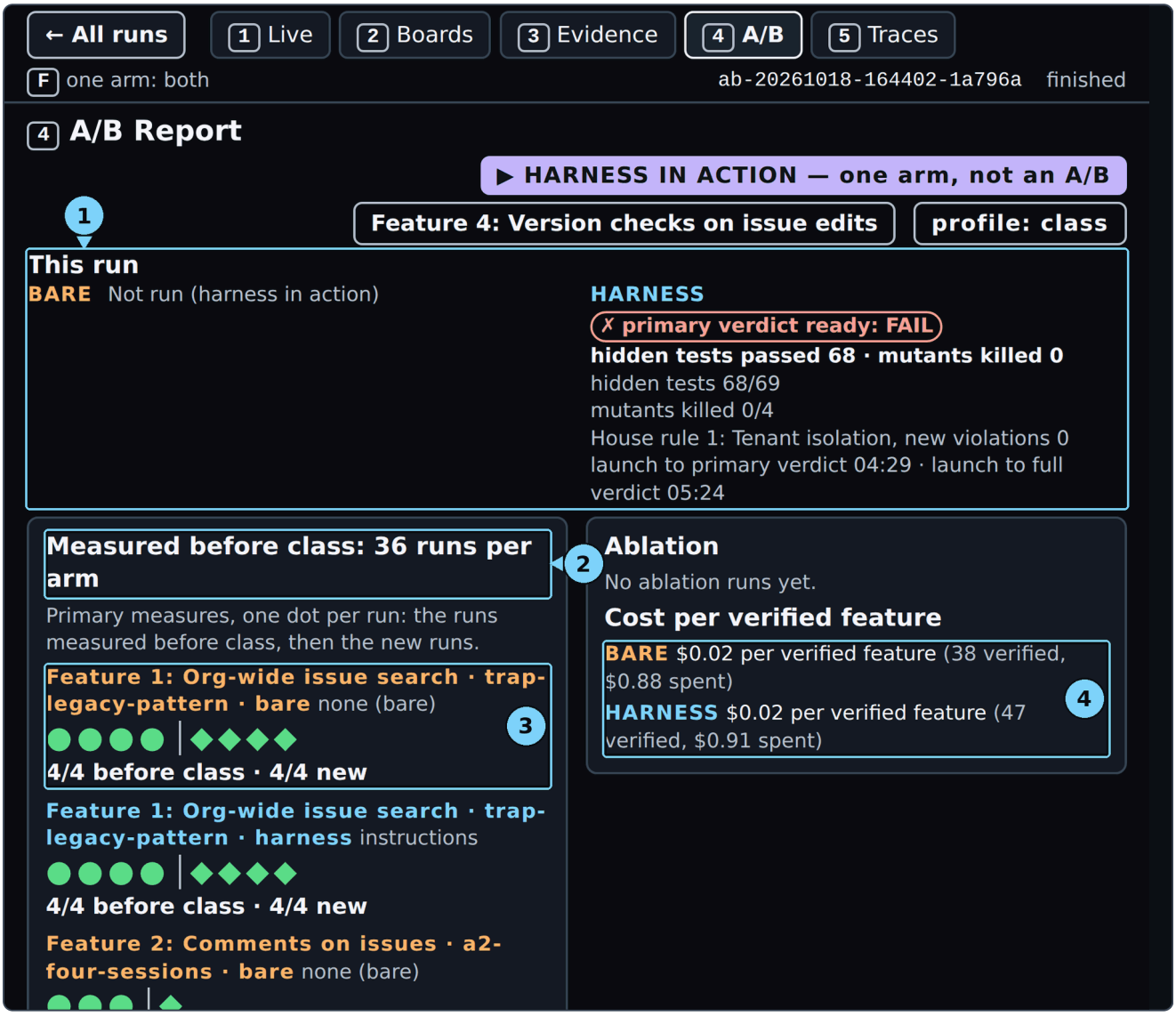
Open the agent link only if there is time. It shows every model call of the run.

Step 24 Open screen 4 and put the run in context

WHERE: Browser, screen 4 A/B Report

TYPE: Press 4. Scroll the left panel down to the Feature 4 rows.

YOU SHOULD SEE:



Screen 4: this run on top, the runs measured before class below.

1. This run
2. Measured before class
3. Dots per run
4. Cost per verified feature

LOOK AT: The two rows "Feature 4: Version checks on issue edits · canonical · bare" and "Feature 4: Version checks on issue edits · canonical · harness": two hollow dots each (○, failed) and "0/2 before class". These rows score this section's measure, all hidden tests and all referee mutants. Below them are two more Feature 4 rows that end "judged by files changed outside the allowed paths", with filled dots: bare 1/10, harness 10/10. Those are Section 3's measure, the scope count from Chapter 3, not this one.

WHAT IT MEANS: Before class, neither arm passed this section's measure on this ticket once. Today's run sits in "This run", not among the dots: it is one arm, shown to see the mechanism.

✓ CHECKPOINT

Both canonical Feature 4 rows without "judged by" read "0/2 before class", with no filled dot.

IF IT LOOKS DIFFERENT

- The page says "no runs" → the console started outside the repository: stop it and run `make console` from the repository root.
- You see filled dots on Feature 4 → you are reading a scope row ("judged by files changed outside the allowed paths"). That is Section 3's measure; read the two rows above it.

◆ INSTRUCTOR NOTE

Say: "No measurable difference on this task with this model: both arms fail. Today's run is not a dot in the comparison." If someone points at the filled Feature 4 dots: "Those rows count files outside the allowed paths, the Section 3 question. On this section's question, all hidden tests and all mutants, both arms are 0 of 2."

Step 25 Go back to the runs list

WHERE: Browser

TYPE: Click "← All runs" (or press H or Esc)

YOU SHOULD SEE: The runs list, with this run's card now "finished".

LOOK AT: The status on the card.

WHAT IT MEANS: The run is complete and saved. You can open it again at any time; the console serves recorded runs.

✓ CHECKPOINT

The card says "finished".

IF IT LOOKS DIFFERENT

- Still "running" → reload the page.

> **◆ INSTRUCTOR NOTE**
> Move on to the final A/B (Chapter 5), or to the "What we measured" slide. In class the final A/B is already running: open its card in the run list.

Result summary

What	This run
Arm	harness, all four layers, class profile
Status	completed, 42 turns (30 build, 12 test phase)
Blocked writes	1 (`app/services/errors.py` , turn 1.11)
Gate	passed on the first claim, 11 of 11 items, 190 visible tests
Agent said	done (build) · not done (test phase)
Tests proposed / filtered / kept	0 / 0 / 0 ("no new tests to filter")
Hidden tests	68 of 69
Referee mutants killed	0 of 4 (4 survived)
Primary verdict	FAIL (claim not supported)
Reviewer	reject

Time and cost

Measure	This run (ran alone)	Range, two runs, 2026-10-18
Launch to blocked write	0:36	
Launch to done claim	2:05	
Launch to gate passed	3:07	
Launch to primary verdict	4:37 (console clock 04:29)	4:37 to 6:54
Launch to full verdict	5:32 (console clock 05:24)	5:32 to 8:02
Command, launch to prompt	5 min 33 s	5:33 to 8:03
Cost of the command	$0.025731 ($0.022527 agent, $0.003204 review)	$0.026 to $0.028
Cost cap	at most $0.24 (`MAX_RUN_COST_USD` plus `MAX_REVIEW_COST_USD`)	

Times run from the make command's launch. The console's own clock starts about 8 s after launch, so it reads a little lower.

The range covers this run and a second run of the same command by a reviewer on a fresh clone (`ab-20261018-181301-c08d1b`), each alone on the machine. That run has no recorded launch time, so its times count from its first event plus this run's 7 s.

Every time in this guide was measured with each demo running alone. In class, the plan starts the final A/B next to this run, and two runs at once make both slower. No run measured by how much.

What to say in class

- "The gate checks what the agent can see. The referee checks what it cannot. In this run the gate said yes and the referee said no."
- "A refused 'done' goes back to the agent with the first three reasons. It fixes them while it still has the context."
- "In this run the agent wrote no tests, so no planted fault was caught: 0 of 4. The gate could not see that; the referee could."
- "The filters keep tests that guard the code as it is. In a recorded run they threw away the one test that found this bug, and the console flagged it."
- "Before class both arms failed this ticket in every run. One run shows the mechanism; it is not a dot in the comparison."

Sources

- Source 1: Automated Unit Test Improvement using Large Language Models at Meta, https://arxiv.org/abs/2402.09171, Alshahwan et al. (Meta), 2024-02-14
- Source 2: Mutation-Guided LLM-based Test Generation at Meta, https://arxiv.org/abs/2501.12862, Foster et al. (Meta), 2025-01-22

CHAPTER 5

Final project: the whole harness, live A/B

You have a real feature to ship with an agent, and four harness layers you could switch on. The easy path is to run the feature once with every layer on, then keep whatever looked better. One run cannot tell you that. Model output changes from run to run, and a second run might split the other way. In this chapter you run one live A/B on Feature 5: Bulk close. The bare arm gets the ticket and the repository. The harness arm also gets all four layers: instructions, state, scope and verify. One referee scores both arms, and you read the result as one dot next to the runs measured before class.

Feature 5 has no primary measure. The referee judges each arm on all its checks, so the pill on screen 1 reads "verdict ready: PASS" with "judged on all checks" under it. The comparison is in the counts: hidden tests, mutants killed, files outside the allowed paths, the gate, and the blinded reviewer. Every number, time and image below comes from one live run on 2026-10-18 (A/B `ab-20261018-164951-bd677b`), which ran alone on the class machine.

> Your output will differ: run times, ids, and the agent's wording change on every run.

Step 1 Check that the console is running

WHERE: Terminal 1 and the browser

TYPE: Click the browser tab with the console at `http://127.0.0.1:8765`. If Terminal 1 runs no console, start it there:

TYPE THIS ⧉ COPY

```
make console
```

YOU SHOULD SEE: The console's run list: "Runs, newest first", one entry per run with its section, feature, type ("harness in action" or "A/B"), status and start time.

LOOK AT: The top entry. It is the last run you started.

WHAT IT MEANS: The console reads every run from disk. It shows a live run as it happens and a finished run in full.

✓ **CHECKPOINT**
The run list loads, and Terminal 1 shows no error (the console is quiet by default).

IF IT LOOKS DIFFERENT

- The page does not load → the console is not running: `make console` in Terminal 1.
- The text is too small or too large → press Ctrl+0 to reset the browser zoom.

◆ INSTRUCTOR NOTE
Say: "This is the last run of the day. Both arms run, and the harness arm has all four layers."

Step 2 Start the final A/B

WHERE: Terminal 2

TYPE:

TYPE THIS ⧉ COPY

```
make final FEATURES=bulk-close PURPOSE=pilot
```

YOU SHOULD SEE: The command and one line, right away. No shell loop is printed.

OUTPUT

```
Final A/B: Feature 5: Bulk close
```

The `A/B ab-…` line and the arm lines come only at the end (Step 18).

LOOK AT: The line "Final A/B: Feature 5: Bulk close". The next lines come only when both arms are judged, about 9 minutes later in this run.

WHAT IT MEANS: `make final` runs one A/B per feature you name, and prints one "Final A/B: …" line per feature. `FEATURES=bulk-close` picks Feature 5. `PURPOSE=pilot` keeps a practice run off the class day's cost cap. In class, with `SHIPHARNESS_CLASS_DATE` set in `.env`, leave it out.

✓ CHECKPOINT
Terminal 2 shows "Final A/B: Feature 5: Bulk close" and waits.

IF IT LOOKS DIFFERENT

- `x SHIPHARNESS_CLASS_DATE is not set` → add `PURPOSE=pilot`, or set the class date in `.env`.
- `x Docker is not running` → `sudo systemctl start docker`, then `make warm`, then start again.
- `x feature 'version-checks': the final project runs bulk-close, watchers, or labels` → name one of those three.

◆ INSTRUCTOR NOTE
In class this starts at the Section 4: Verification launch, next to `make show-4`, with the poll's winner first: `make final FEATURES=<winner>,<runner-up>`. Each feature is its own A/B. Every time in this guide was measured with each demo running alone. Running the final A/B and `make show-4` at once makes both slower; no run measured by how much.

Step 3 Refresh the run list and open the new run

WHERE: Browser, run list

TYPE: Click "← All runs" (or press H), refresh the page, then click the top run id.

YOU SHOULD SEE:

The run list, newest first. The image was taken after the run finished.

1. Run id: open the run
2. Type: harness in action or A/B
3. Status
4. Start time

LOOK AT: The top entry: "Final project · Feature 5: Bulk close · A/B".

WHAT IT MEANS: The type "A/B" means both arms run. The console opens the run on screen 1, Live.

✓ CHECKPOINT

Screen 1 shows two panels, BARE on the left and HARNESS on the right.

IF IT LOOKS DIFFERENT

- No new entry → wait 10 to 20 seconds and refresh. The run appears at its first event.
- The entry says "harness in action" → that is a section run; go back and pick the "A/B" entry.

◆ INSTRUCTOR NOTE

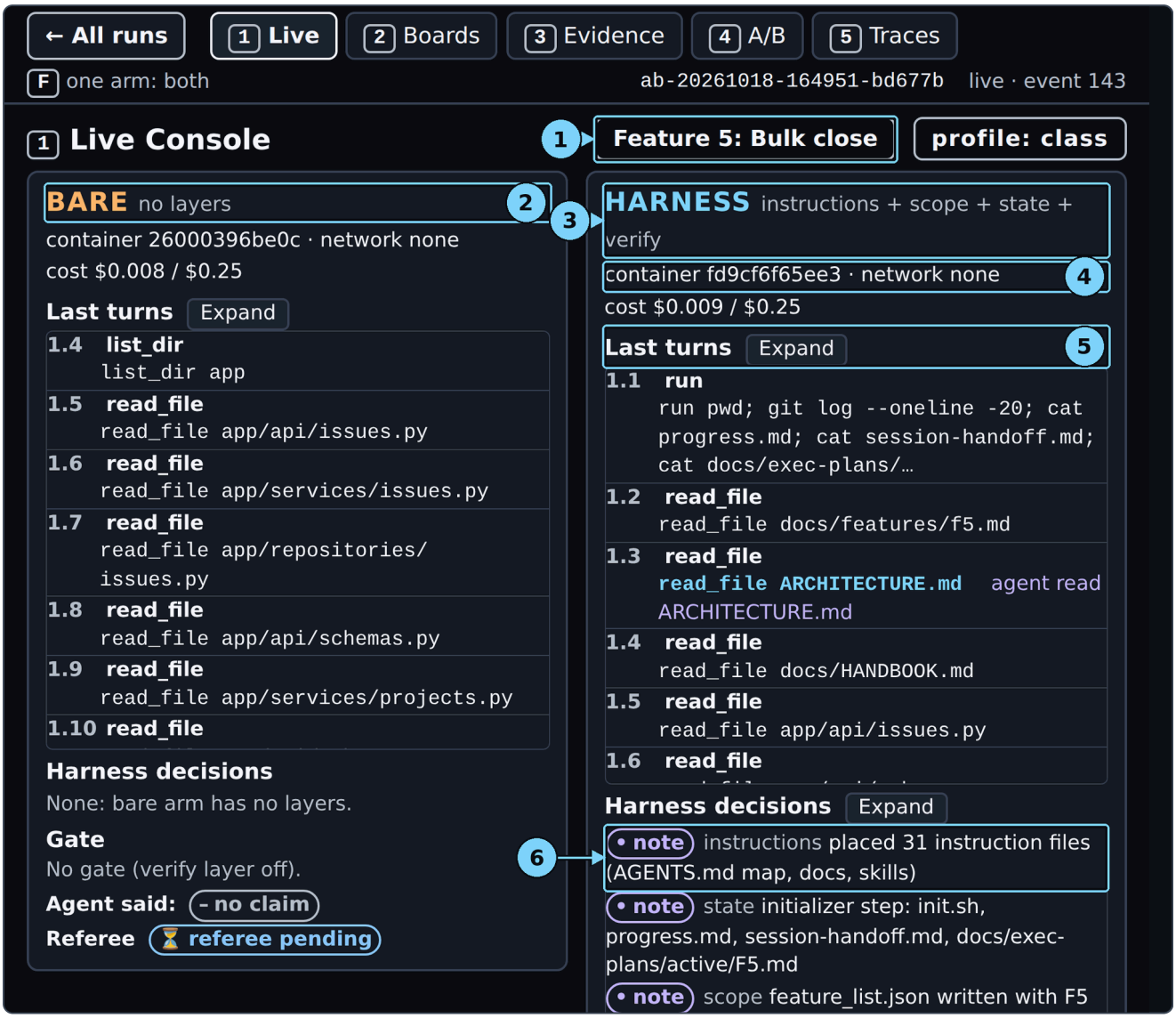
Say: "Same ticket, model, tools, sandbox, turn budget and cost cap. Only the harness differs."

Step 4 Read screen 1 at minute 1

WHERE: Browser, screen 1

TYPE: Press 1

YOU SHOULD SEE:



About one minute in. Both arms are reading code; only the harness arm has harness decisions.

1. Feature badge: Feature 5: Bulk close
2. Bare arm line: no layers
3. Harness arm line: its layers
4. Container line
5. Last turns
6. Harness decisions: the layer notes

LOOK AT: The harness arm's "Harness decisions":

- instructions: "placed 31 instruction files (AGENTS.md map, docs, skills)";
- state: `initializer step: init.sh, progress.md, session-handoff.md, docs/exec-plans/active/F5.md`;
- scope: `feature_list.json written with F5 (passes: false)`;
- verify: "gate baseline reused (same checkpoint commit)";
- instructions: "agent read ARCHITECTURE.md".

WHAT IT MEANS: Each layer reports what it placed before turn 1. The feature list starts at `passes: false`, and only a passed gate may flip it. "Gate baseline reused" means `make warm` cached the baseline, so the verify layer spends no time on it now. The bare arm shows "None: bare arm has no layers", "No gate (verify layer off)", "Agent said: no claim" and "referee pending".

✓ CHECKPOINT

Both arms list turns under "Last turns", and the harness arm shows the four layer notes.

IF IT LOOKS DIFFERENT

- The harness arm says "gate baseline unavailable" and stops with 0 turns → the baseline was not cached: Ctrl+C, `make warm`, start again.
- No turns after 2 minutes → look at Terminal 2 for an `ERROR` line (Docker or the network).

◆ INSTRUCTOR NOTE

Read the four notes aloud, one per layer. Say: "This is the whole harness from Sections 1 to 4, on one arm. The bare arm gets none of it."

Step 5 Wait, and what to do meanwhile

WHERE: Browser, screen 1; Terminal 2

TYPE: No command.

YOU SHOULD SEE: The bare arm finishes first. The harness arm takes longer, because its gate, its test phase and its test filters run before it is judged. Measured before class, its full verdict came 8:41 to 15:22 after launch; in this run, 9:10.

Console clock	Screen	What appears
0:02	1	Harness: the four layer notes
2:04	1	Harness: said done; the gate pills appear, pending
2:06	1	Bare: "Agent said: done"
3:17 to 3:20	1	Bare: run completed (38 turns), "verdict ready: PASS"
3:37	1	Harness: gate passed; session 1 handoff; test phase starts
5:54	1	Bare: full verdict
6:18 to 7:14	1, 3	Harness: 2 tests proposed and filtered
7:14 to 7:15	1	Harness: run completed (35 turns), "verdict ready: PASS"
9:03	1	Harness: full verdict

LOOK AT: The event count at the top right ("live · event N"). It rises while the run is live.

WHAT IT MEANS: The harness arm does more work, on purpose. The gate re-runs the house-rule checks in the arm's container before it accepts "done", and the test phase filters each proposed test. That is the cost of the harness, in minutes. The console clock starts at the run's first event, a few seconds after the make command.

✓ CHECKPOINT

The event count keeps rising until the header says "finished".

IF IT LOOKS DIFFERENT

- The count stops for more than 3 minutes → look at Terminal 2. An `ERROR` line means Docker or the network failed; use the replay fallback in the appendix.

◆ INSTRUCTOR NOTE

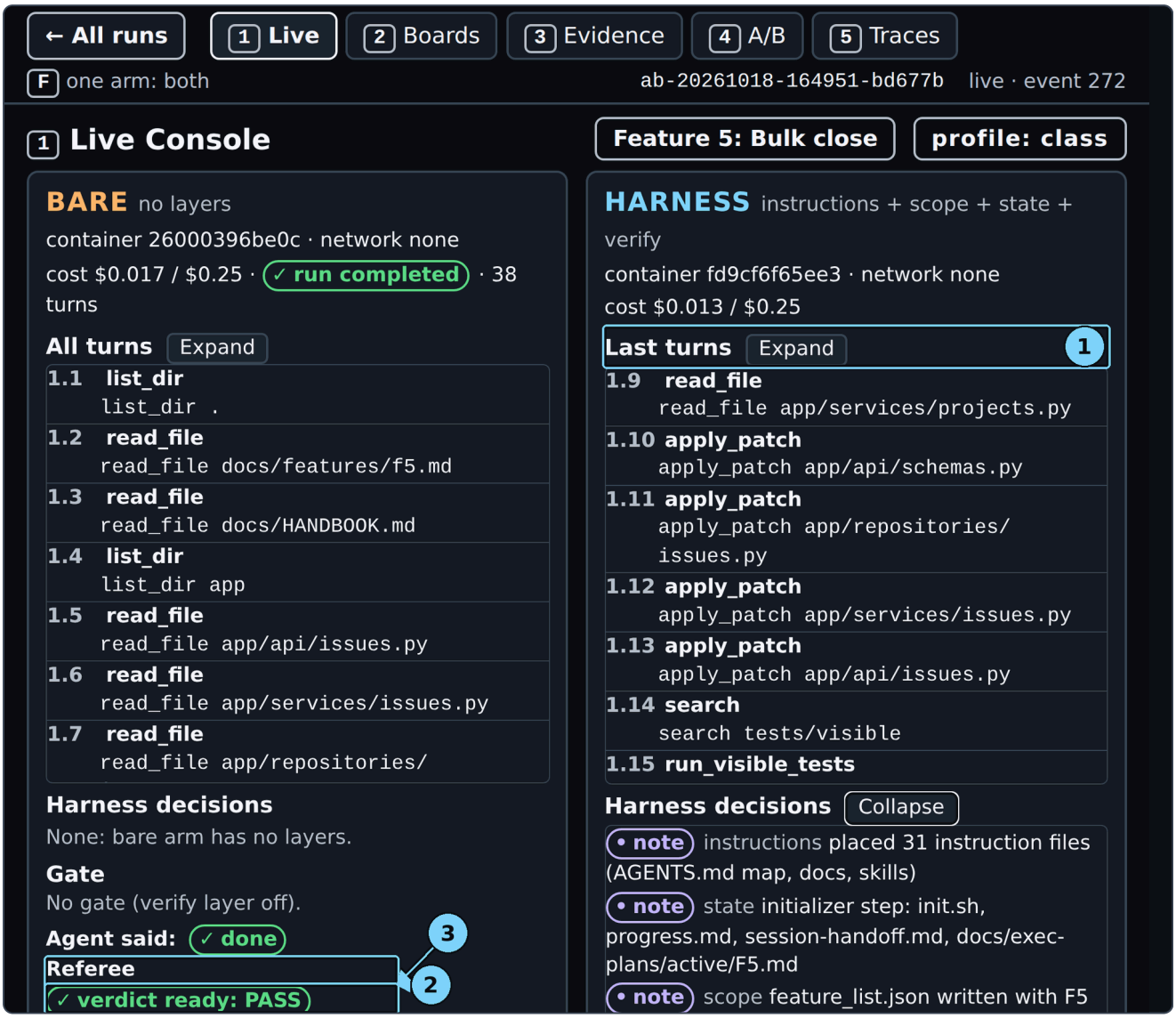
In class you teach Section 4 meanwhile and come back at the final block. Practising alone, press F to show one arm full screen (both, bare, harness), or click Expand next to "Last turns".

Step 6 Read the bare arm's verdict pill

WHERE: Browser, screen 1, BARE panel

TYPE: Press 1

YOU SHOULD SEE:



The bare arm is judged first, while the harness arm still works.

1. Harness still working
2. Bare referee pill: verdict ready: PASS
3. Referee heading (bare)

LOOK AT: The Referee block: "verdict ready: PASS", "judged on all checks", "secondary checks running", "launch to primary verdict 03:18". Next to it: "Agent said: done" and "claim supported".

WHAT IT MEANS: Feature 5 has no primary measure, so there is no single question to answer early. "verdict ready: PASS" says only that the run was judged and nothing failed so far. It says nothing yet about the feature. The counts come with the full verdict.

✓ CHECKPOINT

The bare pill reads "verdict ready: PASS" with "judged on all checks" under it.

IF IT LOOKS DIFFERENT

- The pill says "primary verdict ready" → you opened a section run; check the feature badge.
- The bare status reads "ABORTED: timeout" → the arm hit its 420 s agent time limit. It is still judged; read its counts as usual.

◆ INSTRUCTOR NOTE

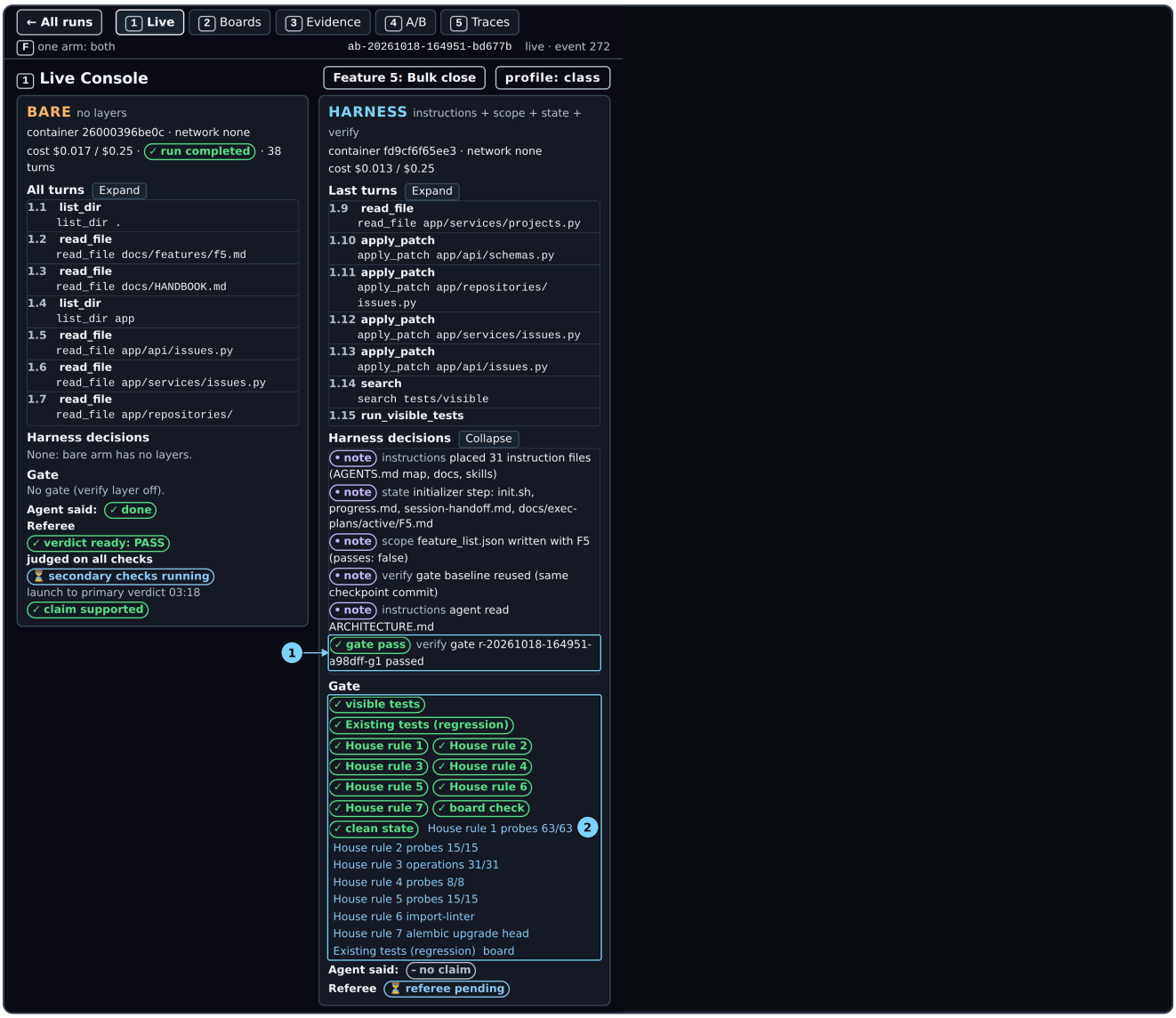
Say: "No primary measure, so this PASS is not news. Wait for hidden tests and mutants."

Step 7 Watch the harness gate and the handoff

WHERE: Browser, screen 1, HARNESS panel

TYPE: Press 1, then click Expand next to "Harness decisions".

YOU SHOULD SEE:



> The gate passed every item at the first attempt, so the agent's "done" can count. The claim line still reads "no claim" until the run ends.

1. Gate passed
2. Gate list: every check passed

LOOK AT: The decisions in order: "gate pass", "session stop: session 1 handoff written to session-handoff.md", `status set: F5 passes: … at 49704cd`, `exec plan moved to docs/exec-plans/completed/F5.md (F5 passes)`. Then the gate pills: visible tests, existing tests, one pill per house rule, board, clean state.

WHAT IT MEANS: When the agent says done, the verify layer runs the gate in the arm's own container. It checks the visible tests (217 passed), the existing tests, and every house rule: House rule 1: Tenant isolation, House rule 2: Roles, House rule 3: API conventions, House rule 4: Safe retries, House rule 5: Audit log, House rule 6: Layering, House rule 7: Migrations. Only a passed gate flips `passes` to true. Then the state layer writes the handoff, and session 2, the test phase, starts.

✓ CHECKPOINT

Every gate pill is green. The harness panel still shows "Agent said: no claim"; after the run it shows "done".

IF IT LOOKS DIFFERENT

- A line "Gate refused 'done' (attempt 1) ..." → the gate failed an item and the agent keeps working; look for "passed on attempt 2".
- A gate pill fails on a timeout → the machine is busy; the arm may end "ABORTED: timeout". It is still judged.

◆ INSTRUCTOR NOTE

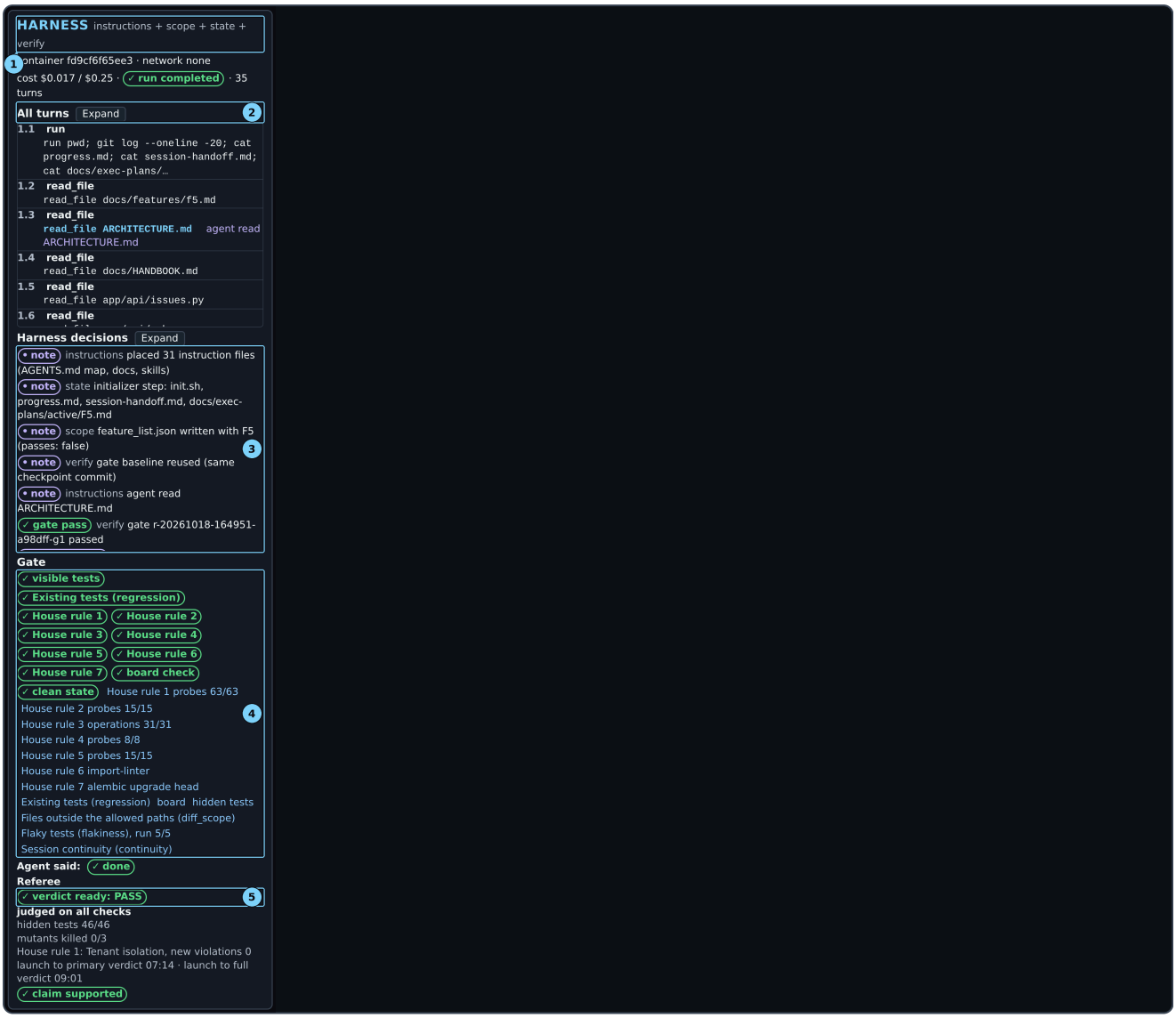
If the gate refuses, say: "The agent said done; the gate said not yet. That is the verify layer doing its job." In this run the gate passed at the first attempt.

Step 8 Watch the harness test phase and its filters

WHERE: Browser, screen 1, HARNESS panel

TYPE: Press 1. Scroll the harness decisions to the end.

YOU SHOULD SEE:



> The harness arm after the run: its turns, decisions, gate list and referee pill. Expand the decisions to see the test filter result.

1. Harness arm line: its layers
2. Turns list
3. Harness decisions
4. Gate list
5. Referee pill (harness)

LOOK AT: The last decisions: "session 2 handoff written to session-handoff.md", then "test filters: kept 1, discarded 1".

WHAT IT MEANS: In the test phase the agent proposes tests. Each must pass Filter 1: Builds and collects, Filter 2: Passes five runs, Filter 3: Adds coverage or catches a mutant, Filter 4: Real assertions, and Filter 5: No source changes. This run proposed 2 tests. One failed Filter 3: it added no coverage over the existing suite. The other, `test_bulk_close_rejects_duplicate_ids`, passed all five and was kept.

✓ CHECKPOINT

"test filters: kept N, discarded M" appears, then "run completed".

IF IT LOOKS DIFFERENT

- "kept 0" → every proposed test failed a filter. That is a result, not a failure; screen 3 gives the reason per test.

◆ INSTRUCTOR NOTE

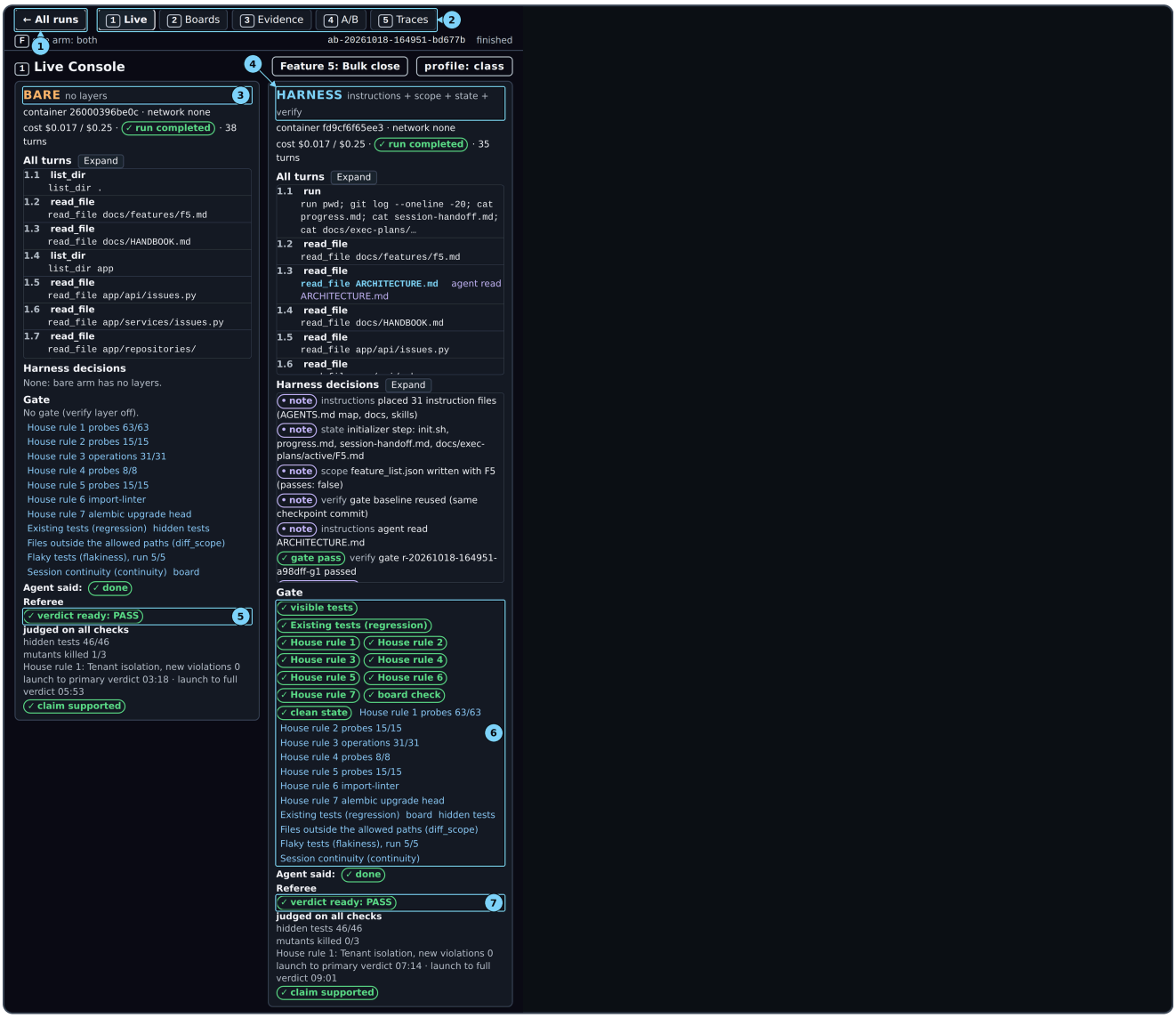
Say: "Many tests an agent writes add nothing the suite did not already check. The filters throw those away before they ship."

Step 9 Read both full verdicts side by side

WHERE: Browser, screen 1

TYPE: Press 1. Press F to cycle one arm full screen if the text is small.

YOU SHOULD SEE:



> Both arms judged. The pills are the same; the counts are where the arms differ.

1. ← All runs: back to the run list
2. Screen tabs 1 to 5
3. Bare arm line: no layers
4. Harness arm line: its layers
5. Referee pill (bare)
6. Gate list
7. Referee pill (harness)

LOOK AT: Each Referee block, then read this table aloud:

	Bare arm	Harness arm
Status	run completed, 38 turns	run completed, 35 turns
Agent said	done	done
Gate	none (verify layer off)	every item passed, attempt 1

	Bare arm	Harness arm
Verdict pill	verdict ready: PASS	verdict ready: PASS
Hidden tests	46/46	46/46
Mutants killed	1/3	0/3
House rule 1: Tenant isolation, new violations	0	0
Launch to primary verdict (console)	03:18	07:14
Launch to full verdict (console)	05:53	09:01
Cost	$0.017 of $0.25	$0.017 of $0.25

WHAT IT MEANS: Both arms built a working feature: 46 of 46 hidden tests, which the agent never saw. They differ in how. The bare arm's own tests caught one of three planted bugs; the harness arm's tests caught none. The harness arm passed a gate the bare arm never faced. "claim supported" on both arms means the agent said done and the referee agrees.

✓ CHECKPOINT

Both Referee blocks show hidden tests, mutants killed and both launch-to times.

IF IT LOOKS DIFFERENT

- "secondary checks running" stays → the reviewer or the mutants still run; wait a minute.
- "claim unsupported" → the agent said done and the referee disagrees; screen 3 shows which check failed.

◆ INSTRUCTOR NOTE

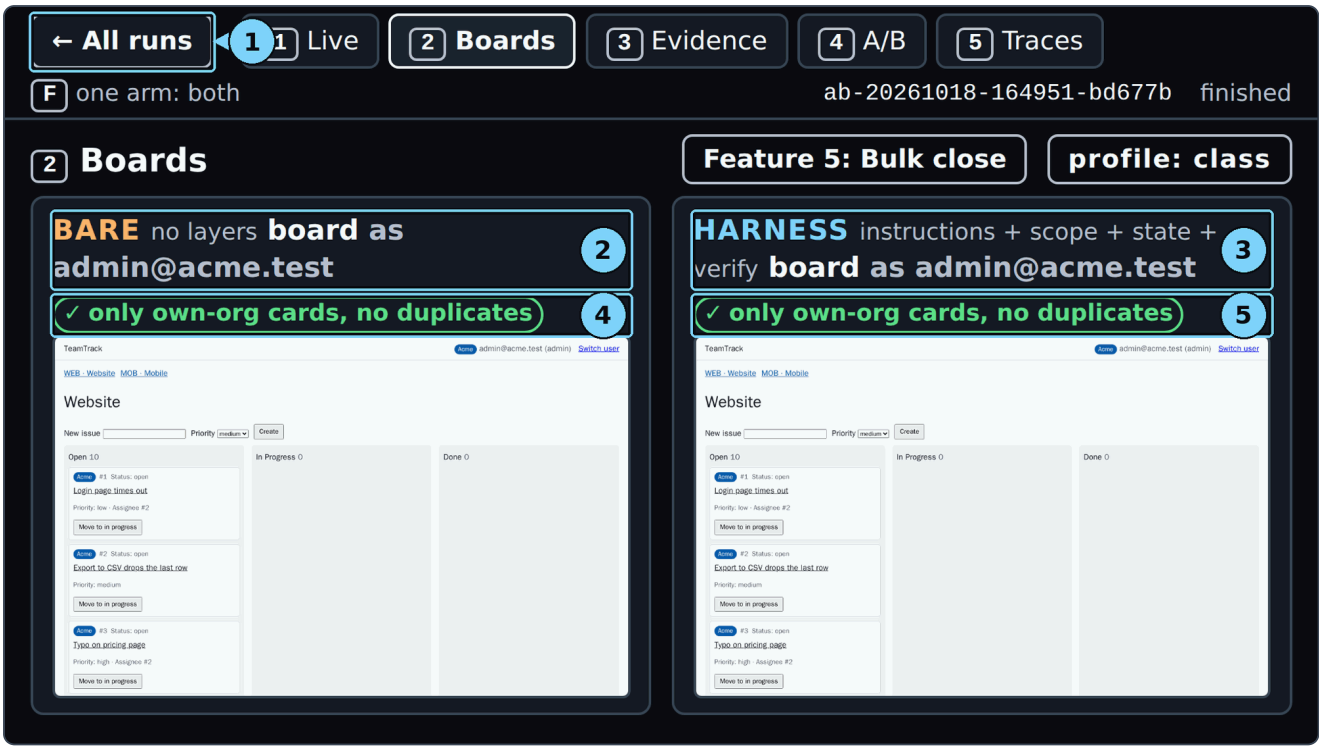
Both pass, as here: "The difference is in how, not whether: mutants killed, scope, gate." One fails: name the failed count, then ask whether the cause is the layers or the machine; a timeout is about time. "Agent said: no claim": the arm ran out of turns before it claimed done, and the referee still judges what it shipped.

Step 10 Read the boards

WHERE: Browser, screen 2

TYPE: Press 2

YOU SHOULD SEE:



Feature 5 adds an endpoint, not a board view, so both boards look the same as before.

1. ← All runs: back to the run list
2. Bare board heading
3. Harness board heading: signed in as one user
4. Bare board marks
5. Harness board marks

LOOK AT: The pill above each board and the org badge on each card.

WHAT IT MEANS: The console takes a board screenshot per arm as an Acme user. A card from another org would carry a ⚠ mark, a duplicate a ⧉ mark. Neither arm leaked.

✓ CHECKPOINT

Both boards show "only own-org cards, no duplicates".

IF IT LOOKS DIFFERENT

- A board is blank → the arm's app did not start; screen 3's "board check" row says why.

◆ INSTRUCTOR NOTE

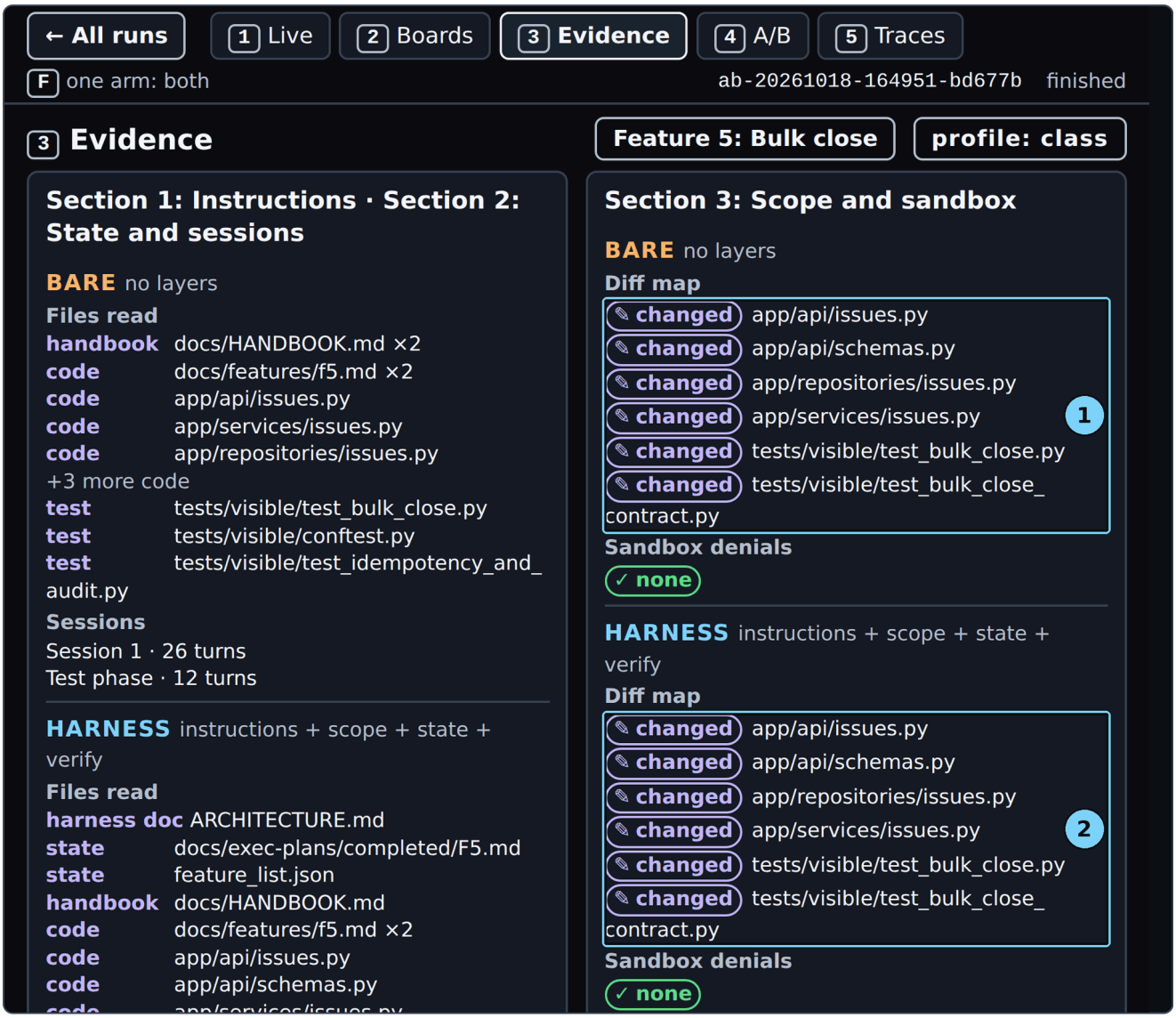
Keep it short: "No leak on either board. Bulk close changes the API, not the board."

Step 11 Screen 3, instructions and state: files read and sessions

WHERE: Browser, screen 3, left column

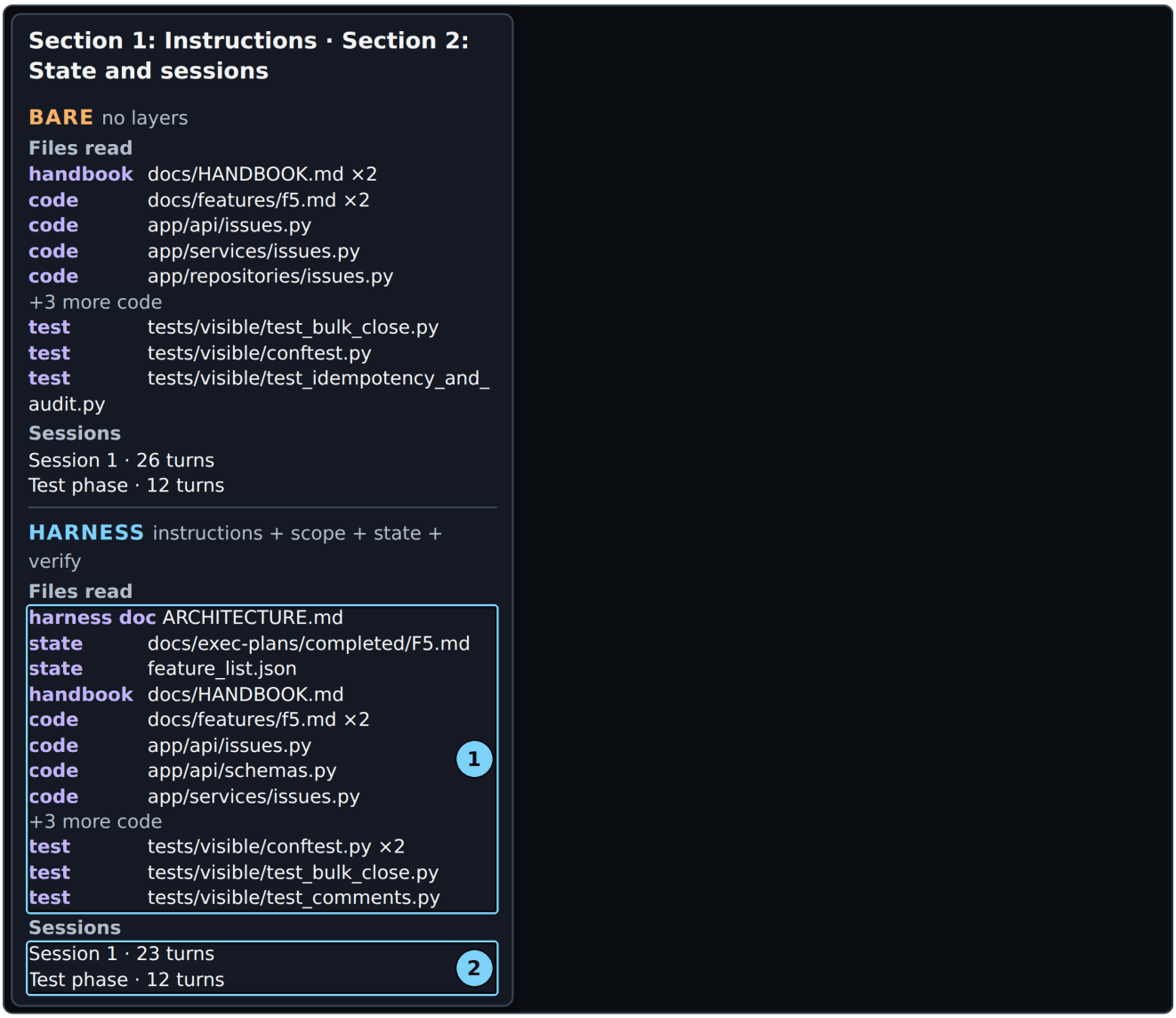
TYPE: Press 3

YOU SHOULD SEE:



Evidence for instructions, state and scope. The referee rows are not in this view; step 13 shows them.

1. Diff map (bare)
2. Diff map (harness)



Files read and sessions.

1. Files read (harness)
2. Sessions (harness)

LOOK AT: "Section 1: Instructions · Section 2: State and sessions". The harness arm read `ARCHITECTURE.md` (harness doc), `docs/exec-plans/completed/F5.md` and `feature_list.json` (state), then the handbook and the code. The bare arm read `docs/HANDBOOK.md`, the feature spec and the code. Sessions: bare "Session 1 · 26 turns", "Test phase · 12 turns"; harness "Session 1 · 23 turns", "Test phase · 12 turns".

WHAT IT MEANS: The instructions layer put a map in front of the agent, and the agent opened the architecture document it points to. The state layer's exec plan and feature list carried the work from the build session into the test phase.

✓ CHECKPOINT

The harness arm lists a "harness doc" row and "state" rows under "Files read".

IF IT LOOKS DIFFERENT

- No harness doc row → the agent opened no harness document this time. That is a result; say so.

◆ INSTRUCTOR NOTE

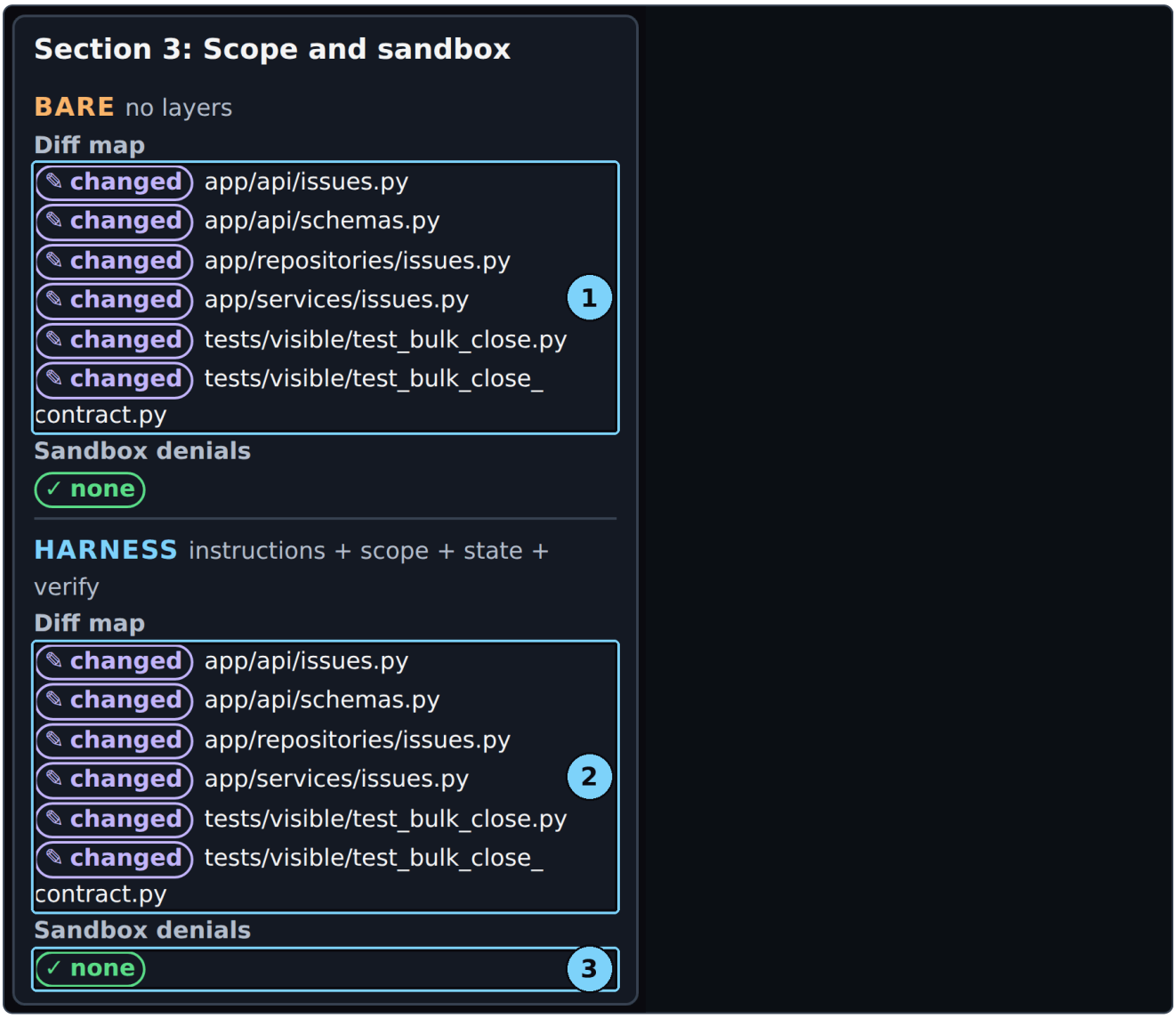
Say: "Instructions and state: the agent used the map and the plan the harness left for it."

Step 12 Screen 3, scope: diff maps and sandbox denials

WHERE: Browser, screen 3, middle column

TYPE: Press 3

YOU SHOULD SEE:



Both arms changed the same six files, all inside the allowed paths.

1. Diff map (bare)
2. Diff map (harness)
3. Sandbox denials (harness)

LOOK AT: "Section 3: Scope and sandbox". Both arms show "✎ changed" for `app/api/issues.py`, `app/api/schemas.py`, `app/repositories/issues.py`, `app/services/issues.py`, `tests/visible/test_bulk_close.py` and `tests/visible/test_bulk_close_contract.py`. "Sandbox denials: none" for both.

WHAT IT MEANS: No file outside the allowed paths in either arm, so the scope layer had nothing to block. A "⊘ blocked" row would show an edit the scope layer refused.

✓ CHECKPOINT

Both diff maps show only "✎ changed" rows, and both arms say "none" under sandbox denials.

IF IT LOOKS DIFFERENT

- A "⊘ blocked" row in the harness arm → the scope layer refused an edit outside the allowed paths; screen 1 shows the reason.

◆ INSTRUCTOR NOTE

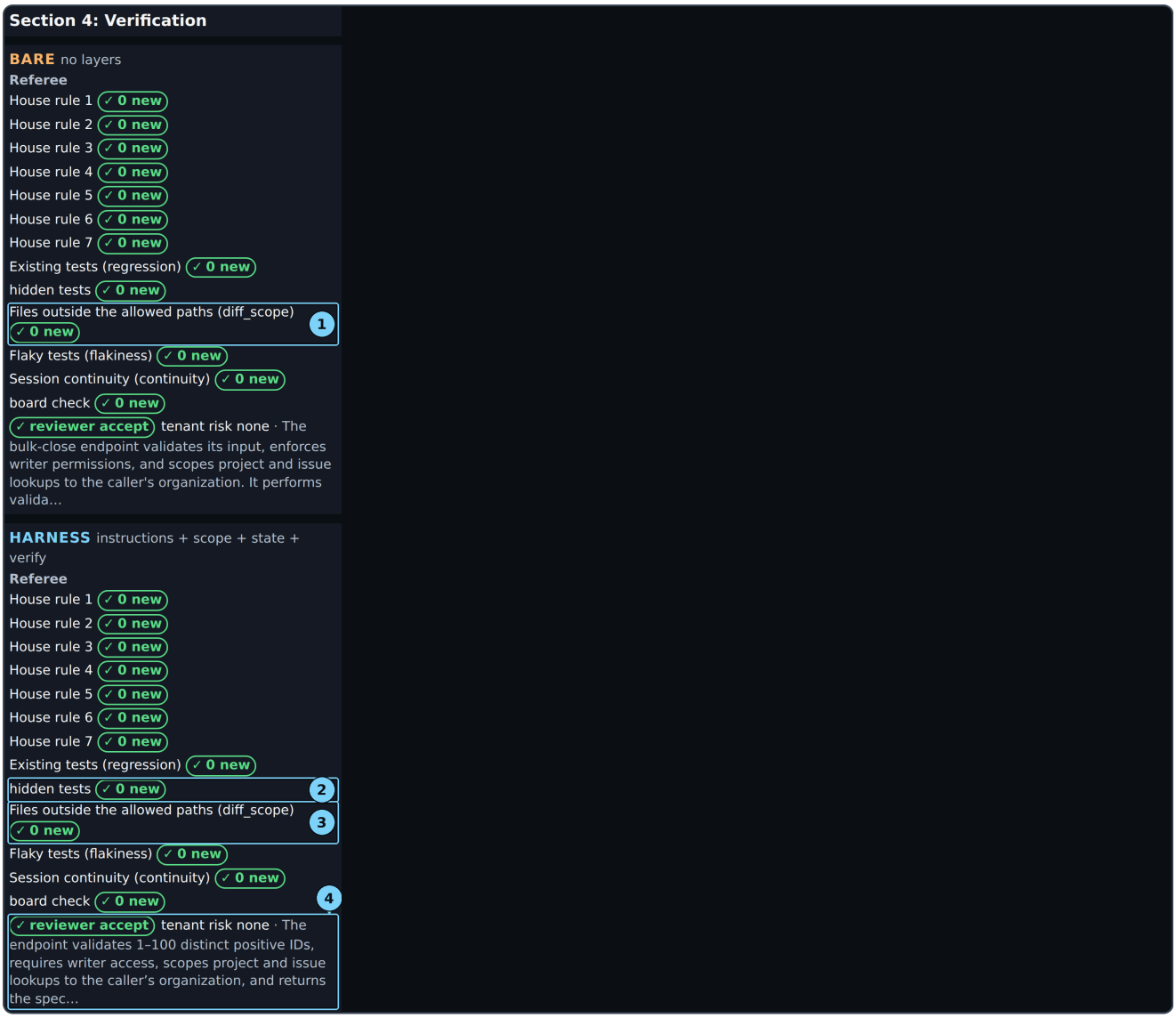
Say: "Scope: on this ticket both arms stayed inside the allowed paths."

Step 13 Screen 3, verify: referee checks, mutants, filter table

WHERE: Browser, screen 3, right column; scroll down for the mutants and the filter table

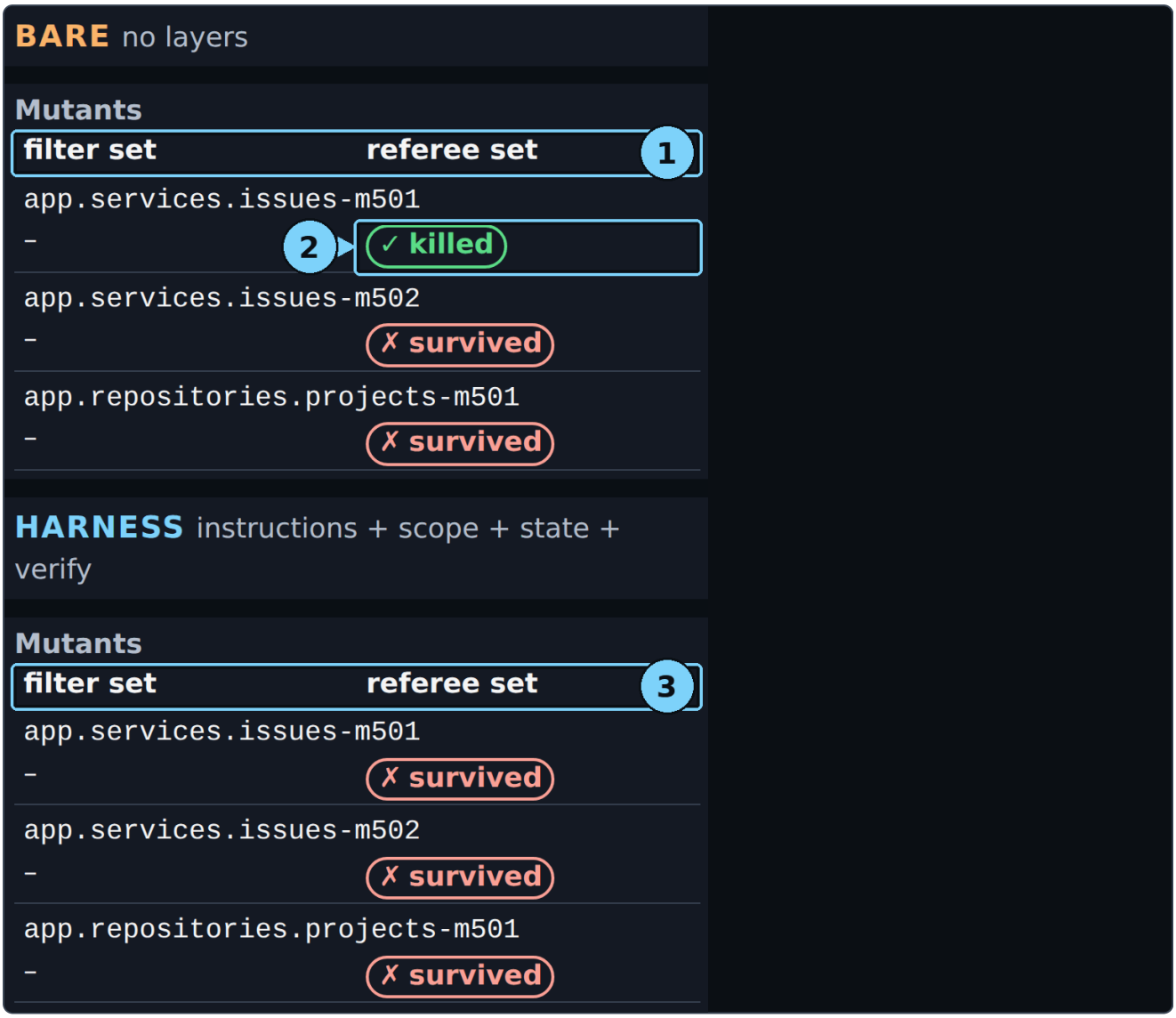
TYPE: Press 3, then scroll the right column.

YOU SHOULD SEE:



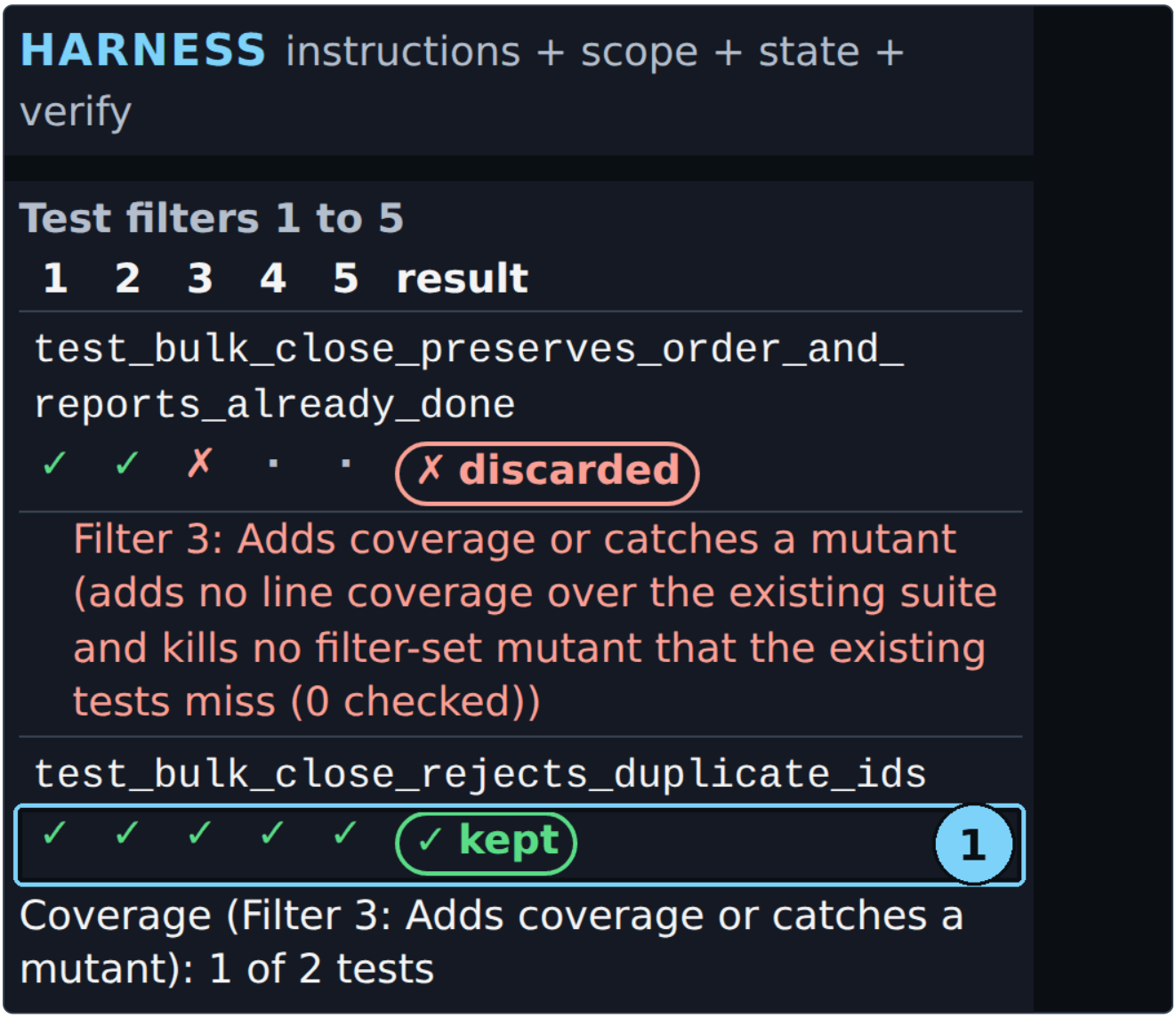
> The referee rows: every check "0 new" in both arms.

1. Files outside the allowed paths (diff_scope), bare
2. hidden tests (harness)
3. Files outside the allowed paths (diff_scope), harness
4. Blinded reviewer's reason (harness)



The mutant table: which planted bugs each arm's tests caught.

1. Mutants (bare): filter set and referee set
2. Bare: a mutant the referee set killed
3. Mutants (harness): filter set and referee set



The harness arm's filter table: one row per test, one column per filter.

1 A test kept by the filters

LOOK AT: The referee rows per arm: House rule 1 to House rule 7, "Existing tests (regression)", "hidden tests", "Files outside the allowed paths (diff_scope)", "Flaky tests (flakiness)", "Session continuity (continuity)", "board check", each "0 new". Then "reviewer accept · tenant risk none" with the reviewer's one-line reason. Then the mutants: the bare arm killed `app.services.issues-m501`; `app.services.issues-m502` and `app.repositories.projects-m501` survived. In the harness arm all three survived. Last, the filter table: `test_bulk_close_preserves_order_and_reports_already_done` "✗ discarded" at Filter 3, `test_bulk_close_rejects_duplicate_ids` "✓ kept".

WHAT IT MEANS: No check found a new violation in either arm, and the reviewer accepted both. The mutants are the one count that separates the arms in this run, by one planted bug.

✓ CHECKPOINT

Both arms show "reviewer accept"; the mutant table shows one "killed" for bare and none for harness.

IF IT LOOKS DIFFERENT

- A filter row says "fails on the current code; this may be a real bug" → a Filter 2 discard that matched the ticket's value; read it, it may point to a real bug.

◆ INSTRUCTOR NOTE

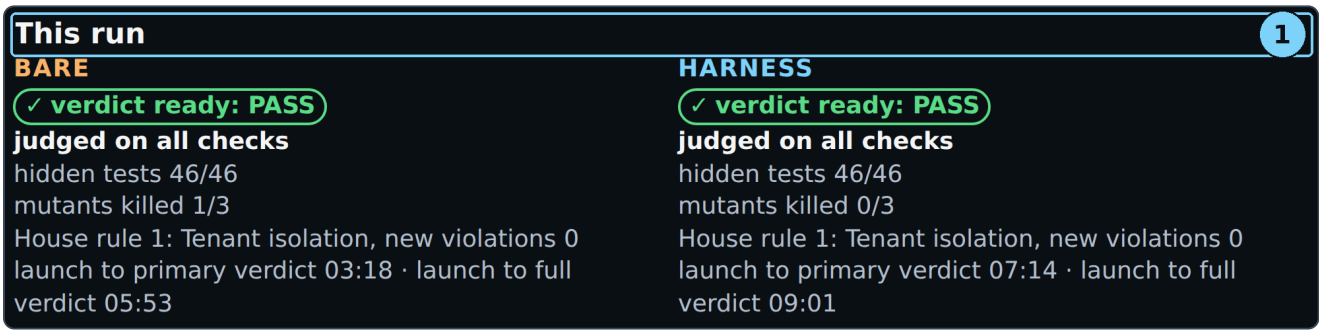
On a reviewer reject: "A separate model read the diff without knowing which arm wrote it, and objected. Read its one-line reason before you judge the arm."

Step 14 Screen 4, this run

WHERE: Browser, screen 4, top

TYPE: Press 4

YOU SHOULD SEE:



This run, both arms, at the top of the A/B Report.

1 This run

LOOK AT: "This run": BARE "verdict ready: PASS · judged on all checks", hidden tests 46/46, mutants killed 1/3; HARNESS the same pill, hidden tests 46/46, mutants killed 0/3. Both: House rule 1, new violations 0.

WHAT IT MEANS: Screen 4 repeats this run's verdicts so you can hold them next to the earlier runs below.

✓ CHECKPOINT

"This run" shows both arms with their counts.

IF IT LOOKS DIFFERENT

- "No results to compare yet" → the run is not finished; come back after both full verdicts.

◆ INSTRUCTOR NOTE

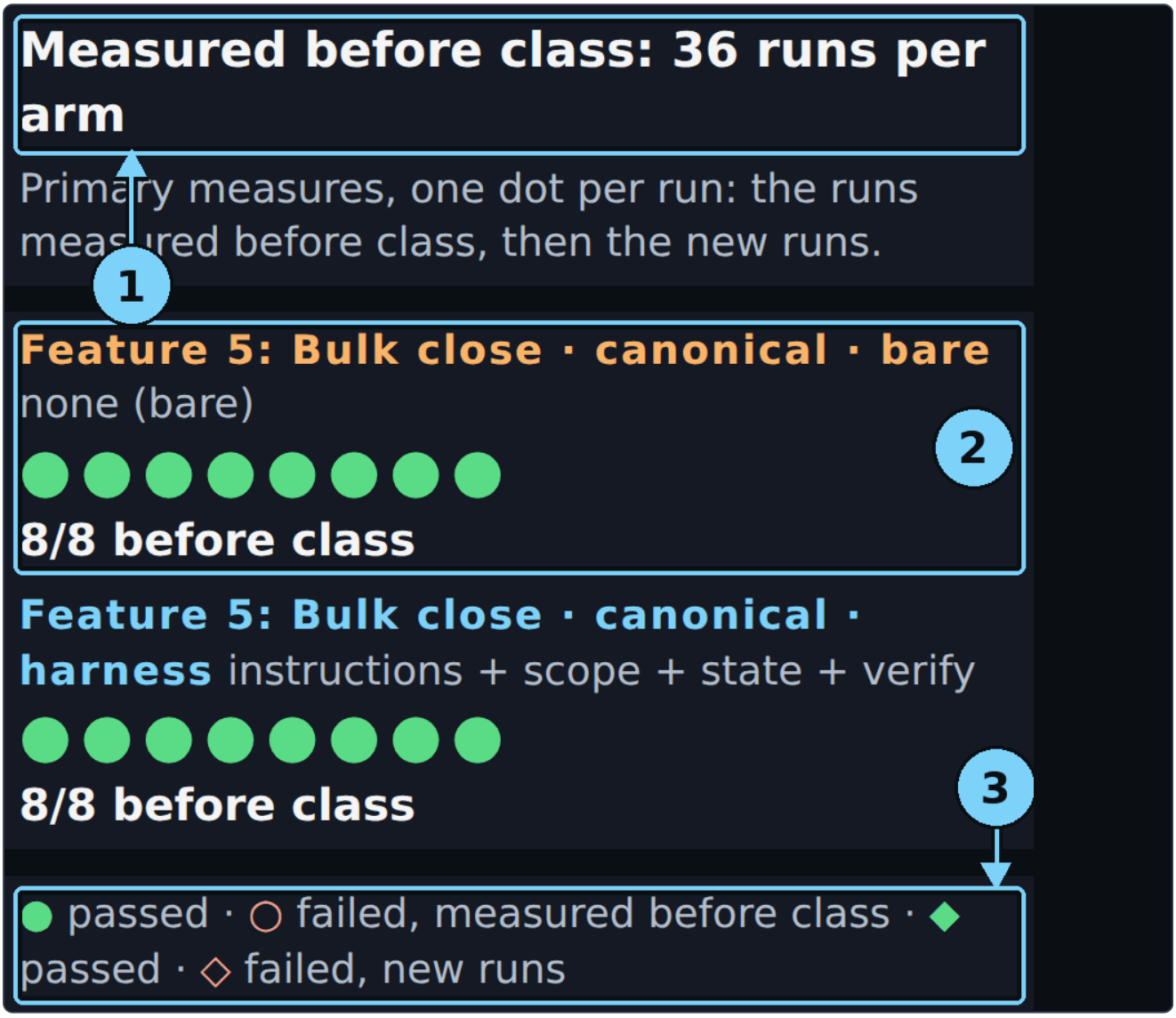
Say: "Here is today's result. Now look at what came before it."

Step 15 Screen 4, the runs measured before class

WHERE: Browser, screen 4, left panel; scroll to the Feature 5 rows

TYPE: Press 4, then scroll the left panel to the end.

YOU SHOULD SEE:



One dot per run: filled passed, hollow failed; diamonds are new runs.

1. Measured before class
2. Dots per run, Feature 5: Bulk close
3. Legend

LOOK AT: The two Feature 5 rows. "Feature 5: Bulk close · canonical · bare": "8/8 before class · 1/1 new". "Feature 5: Bulk close · canonical · harness": "8/8 before class · 1/1 new". Today's run is the diamond after the bar.

WHAT IT MEANS: One A/B is one dot. With no primary measure, every Feature 5 run before class passed in both arms, and today's did too. The dots alone cannot separate the arms on this feature; the counts in steps 9 and 13 do, run by run.

✓ CHECKPOINT

Each Feature 5 row ends with one new diamond.

IF IT LOOKS DIFFERENT

- No diamond → the score record is not in `exports/` yet; reload after the terminal prints its `Cost:` line.

◆ INSTRUCTOR NOTE

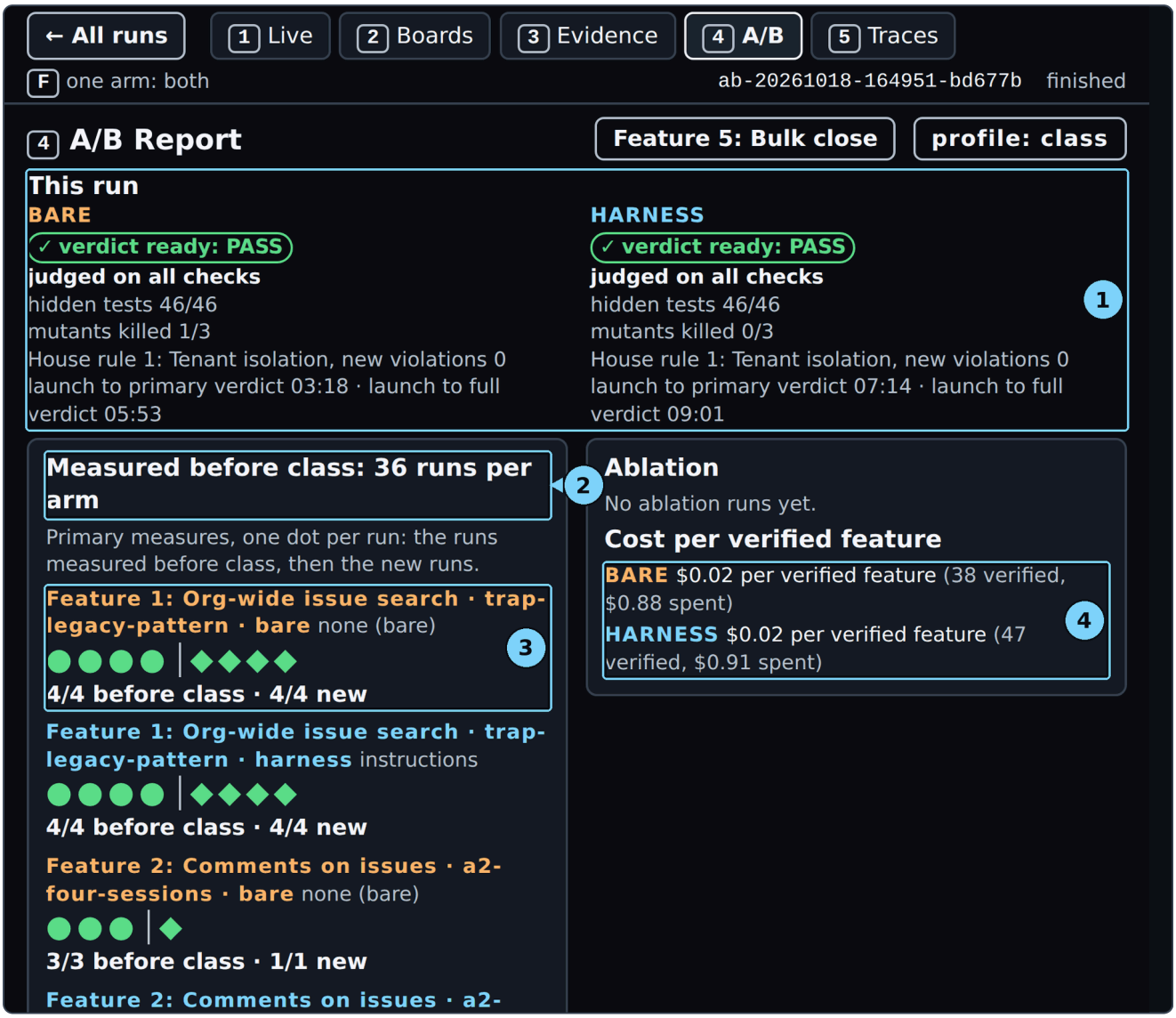
Say: "One A/B is one dot. Read it next to the dots from before class." Point at the rows of Feature 4: Version checks on issue edits for contrast: there the dots do separate the arms on files outside the allowed paths.

Step 16 Screen 4, cost per verified feature and reviewer agreement

WHERE: Browser, screen 4, middle and right panels

TYPE: Press 4

YOU SHOULD SEE:



The top of the A/B Report.

1. This run
2. Measured before class
3. Dots per run
4. Cost per verified feature

LOOK AT: "Cost per verified feature": BARE "$0.03 per verified feature (27 verified, $0.68 spent)", HARNESS "$0.02 per verified feature (36 verified, $0.76 spent)". "Reviewer agreement": BARE "reviewer agrees with the referee in 27/35 runs", HARNESS "29/35 runs".

WHAT IT MEANS: Cost per verified feature divides what an arm spent across all scored runs by the features the referee verified. Reviewer agreement counts how often the blinded reviewer and the referee agreed. Both pool every run in `exports/`, not only today's.

✓ CHECKPOINT

Both panels show one line per arm.

IF IT LOOKS DIFFERENT

- "No costs yet" → no scored record is in `exports/` yet; wait for the full verdicts.

◆ INSTRUCTOR NOTE

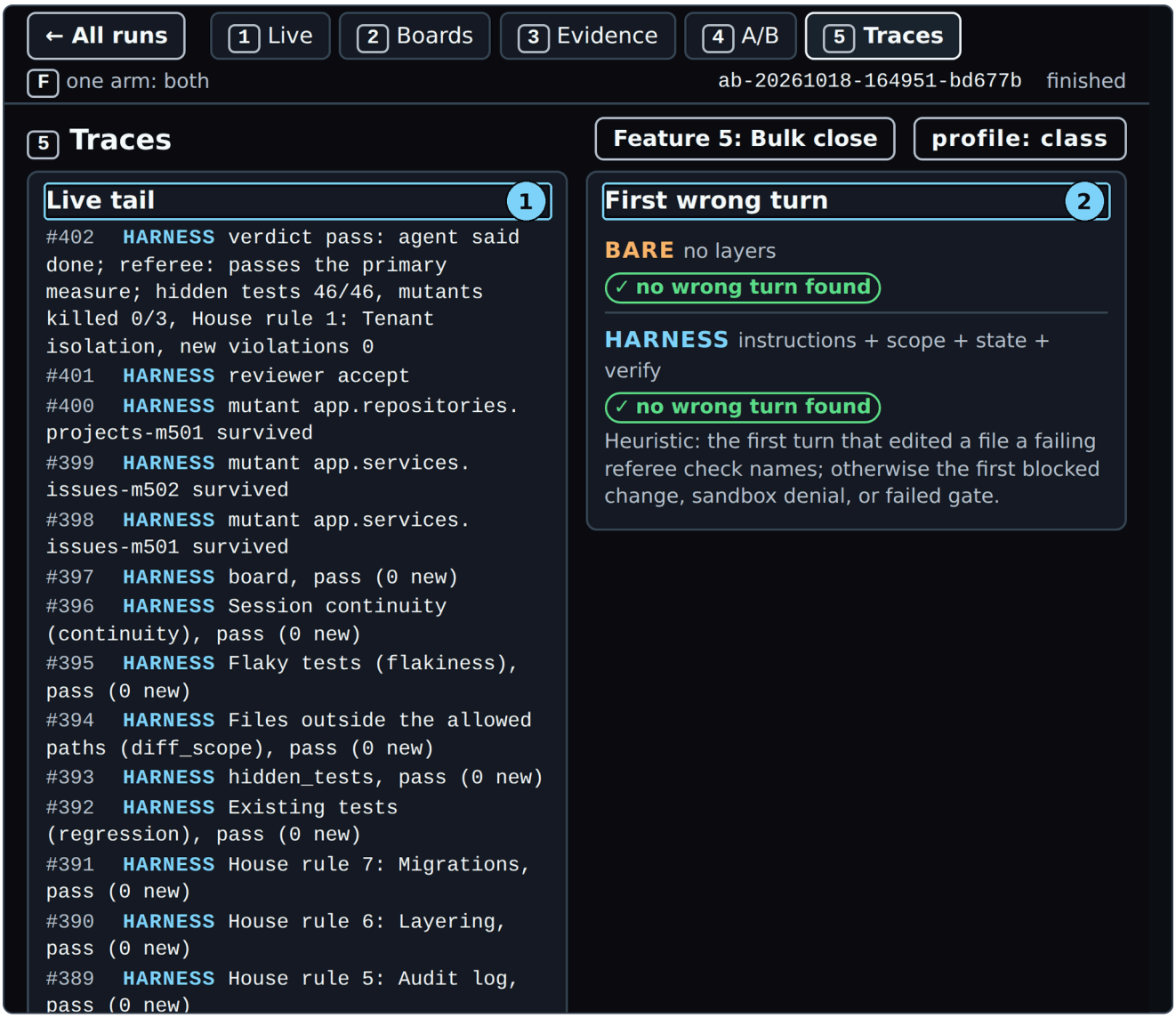
Say: "Across all runs, the harness arm verified more features for a little more money in total. In this run it cost $0.0166 against the bare arm's $0.0173."

Step 17 Screen 5, traces

WHERE: Browser, screen 5

TYPE: Press 5

YOU SHOULD SEE:



Traces: the event tail and the first wrong turn per arm.

1. Live tail
2. First wrong turn

LOOK AT: "First wrong turn": "no wrong turn found" for both arms. The tail ends with the harness arm's mutants "survived", "reviewer accept" and its verdict line.

WHAT IT MEANS: The console looks for the first turn that edited a file a failing referee check names, else the first blocked change, sandbox denial or failed gate. With every check passing, it finds none. Surviving mutants do not count as a wrong turn: they show weak tests, not a wrong edit.

✓ CHECKPOINT

Each arm shows a "First wrong turn" result and LangSmith links.

IF IT LOOKS DIFFERENT

- No LangSmith links → tracing is off; the run is still valid.

◆ INSTRUCTOR NOTE
Open one LangSmith link only if a question needs it.

Step 18 Read the terminal's final lines

WHERE: Terminal 2

TYPE: No command.

YOU SHOULD SEE:

Terminal 2

```
$ make final FEATURES=bulk-close PURPOSE=pilot
… 2 lines above
  bare     completed         agent said done · no primary measure · hidden tests 46 of 46 · mutants
killed 1 of 3 · reviewer accept · cost $0.0173
  harness  completed         agent said done · no primary measure · hidden tests 46 of 46 · mutants
killed 0 of 3 · reviewer accept · cost $0.0166
Cost: $0.037714 (this command: gpt-5.6-luna $0.003787, gpt-6-luna $0.033927; session $0.183679 of
$2.50; build $0.183679 of $15.00 ($14.816321 left))
```

Terminal 2 when the command ends.

OUTPUT

```
Final A/B: Feature 5: Bulk close
A/B ab-20261018-164951-bd677b (Feature 5: Bulk close, layers=['instructions', 'scope', 'state',
'verify'], profile=class)
  bare     completed         agent said done · no primary measure · hidden tests 46 of 46 · mutants
killed 1 of 3 · reviewer accept · cost $0.0173
  harness  completed         agent said done · no primary measure · hidden tests 46 of 46 · mutants
killed 0 of 3 · reviewer accept · cost $0.0166
Cost: $0.037714 (this command: gpt-5.6-luna $0.003787, gpt-6-luna $0.033927; session $0.183679 of
$2.50; build $0.183679 of $15.00 ($14.816321 left))
```

LOOK AT: Per arm: status, "agent said: done", "no primary measure", hidden tests, mutants killed, reviewer, cost. Then the `Cost:` line.

WHAT IT MEANS: The arm lines repeat screen 1, one line each. The `Cost:` line gives this command's spend, split into reviews (gpt-5.6-luna) and agents (gpt-6-luna), then the session and build totals against their caps. The session figure here includes the four section runs before it.

✓ CHECKPOINT
Both arm lines and the `Cost:` line are printed, and the prompt returns (exit code 0).

IF IT LOOKS DIFFERENT

- `make: *** … Error 2` after the lines → an arm was aborted on time or cost. Both arms are still judged: a result, not a failure.

◆ INSTRUCTOR NOTE
Read the session figure on the `Cost:` line aloud. In class it shows what the whole session has spent against $2.50.

Step 19 Say what the result means

WHERE: Browser, screen 1 or screen 4

TYPE: No command.

YOU SHOULD SEE: The table in step 9 and the Feature 5 dots in step 15.

LOOK AT: Mutants killed, the gate, and files outside the allowed paths, per arm.

WHAT IT MEANS: Read the outcome as one of three:

- Harness ahead: better counts, and the runs before class show the same gap.
- No measurable difference: equal counts, or a difference one run can produce.
- Bare ahead: the same as harness ahead, the other way round.

In this run both arms passed every check. The bare arm killed one more mutant (1 of 3 against 0 of 3). As one dot that is a difference one run can produce. Earlier Feature 5 runs split 2 of 3 against 3 of 3 and 3 of 3 against 1 of 3, so no arm leads on mutants across runs.

✓ CHECKPOINT
You can say the result in one sentence, with its counts.

IF IT LOOKS DIFFERENT
- One arm failed → name the failed count and its cause (the layers, or machine time).

◆ INSTRUCTOR NOTE
Both pass: "The difference is in how, not whether: mutants killed, scope, gate." One fails: "This is one dot. Here is why it failed, and here is what the earlier dots say."

Result summary

	Bare arm	Harness arm
Status	completed, 38 turns	completed, 35 turns
Sessions	build 26, test phase 12	build 23, test phase 12
Agent said	done	done
Gate	none (verify layer off)	passed at attempt 1
Hidden tests	46 of 46	46 of 46
Mutants killed	1 of 3	0 of 3

	Bare arm	Harness arm
Files outside the allowed paths	0	0
Sandbox denials	none	none
Test filters	none	kept 1, discarded 1
Blinded reviewer	accept	accept
Launch to primary verdict (from make)	3:27	7:23
Launch to full verdict (from make)	6:02	9:10
Arm cost	$0.0173	$0.0166

Time and cost

The run ran alone on the class machine. From the make command to the `Cost:` line: 9 min 11 s, exit code 0. The console's own "launch to" times are about 9 seconds shorter, because its clock starts at the run's first event. The command cost $0.037714: $0.033927 for the two agents on gpt-6-luna and $0.003787 for the two blinded reviews on gpt-5.6-luna. The cap is $0.58 per feature: two arms at $0.25 and two reviews at $0.04. Before class, three runs of the same command, two at a time, took up to 15 min 22 s and cost $0.044 to $0.048 each.

Moment	Range, two runs, 2026-10-18
Launch to primary verdict	bare 2:48 to 3:27, harness 5:31 to 7:23
Launch to full verdict	bare 5:07 to 6:02, harness 7:25 to 9:10
Cost of the command	$0.038 to $0.042

The ranges cover this run and a second run of the same command by a reviewer on a fresh clone (`ab-20261018-182151-a4bd6f`), each alone on the machine. That run has no recorded launch time, so its times count from its first event plus this run's 7 s. Running two demos at once makes both slower.

What to say in class

- "One A/B is one dot. Read it next to the dots from before class, not on its own."
- "Feature 5 has no primary measure, so 'verdict ready: PASS' is not news. The counts are."
- "Both arms passed 46 of 46 hidden tests. The difference is in how, not whether: mutants killed, scope, gate."
- "Today the bare arm's tests caught one more planted bug. Earlier runs split the other way, so this is no gap yet."
- "To learn which layers earn their place, remove one layer at a time and compare many runs, not one."

APPENDIX

Cheat sheet, troubleshooting, glossary and fallback

Command cheat sheet

Every command this guide uses, in the order the guide uses it. A demo's time is a range over two runs of the command on 2026-10-18, with the class profile, each run alone on the machine: this guide's run and a second run by a reviewer on a fresh clone. It runs from launch to the last arm's full verdict; the command ends a few seconds later. The reviewer's runs have no recorded launch time, so their times count from the run's first event plus the 6 to 8 s that the same command took to write its first event in this guide's run. Two demos running at once make both slower. "At most" is the cap from `.env.example`.

Command	Where	What it does	Time and cost
`wsl -d Ubuntu`, then `cd ~/shipharness`	PowerShell	Opens Ubuntu in your copy	No cost
`uv run shipharness verify --tag class-v1`	Terminal 2	Checks your copy against the class release	Seconds, no cost
`make doctor PURPOSE=pilot`	Terminal 2	Checks tools, Docker, keys, models, prices	6 s, no cost
`make warm PURPOSE=pilot`	Terminal 2	Builds changed images, caches baselines	431 s the first time, then about 2 s; no cost
`make console`	Terminal 1	Serves the console on port 8765	Runs until Ctrl+C; no cost
`make console VERBOSE=1`	Terminal 1	The same, with one line per request	No cost
`make console CONSOLE_PORT=8766`	Terminal 1	The console on another port	No cost
`make show-1 PURPOSE=pilot`	Terminal 2	Chapter 1 demo: harness arm alone	3:37 to 4:45; $0.020 to $0.021; at most $0.24
`make show-2 SESSION_BUDGET=12,38 PURPOSE=pilot`	Terminal 2	Chapter 2 demo: two sessions	5:52 to 7:11; $0.036 to $0.044; at most $0.24
`make demo-3 FEATURE=version-checks PURPOSE=pilot`	Terminal 2	Chapter 3 demo: an A/B	5:14 to 8:24; $0.050 to $0.056; at most $0.48
`make show-4 PURPOSE=pilot`	Terminal 2	Chapter 4 demo: harness arm alone	5:32 to 8:02; $0.026 to $0.028; at most $0.24
`make final FEATURES=bulk-close PURPOSE=pilot`	Terminal 2	Chapter 5 demo: the final A/B	7:25 to 9:10; $0.038 to $0.042; at most $0.58
`make replay RUN=<run id> PURPOSE=pilot`	Terminal 2	Shows a recorded run again	2:06 (one run); $0, no model calls

Console keys: `1` Live, `2` Boards, `3` Evidence, `4` A/B report, `5` Traces; `F` one arm full width; "← All runs", `H` or Esc back to the run list. Reload the run list to see a new run.

Troubleshooting

Find what you see in the first column. Every live command prints a `Cost:` line at the end; an exit code of 2 after the results means an arm hit a cap or a time limit, and the run is still complete.

Symptom	Cause	Fix
Console text or panels look cut off or too large	Browser zoom	Press `Ctrl+0`, then reload with `Ctrl+F5`
Console looks squeezed on a laptop screen	Windows display scaling	Nothing should be cut: press `Ctrl+0`, reload with `Ctrl+F5`
Console shows an older design	The browser cached the old style	Hard reload with `Ctrl+F5`
`address already in use` from `make console`	Another server holds port 8765	`make console CONSOLE_PORT=8766`, then open port 8766
The console header shows `reconnecting`	The console server stopped	Run `make console` again in Terminal 1, then reload
A new run is not in the run list	The list does not refresh by itself	Reload the page; the newest run is first
`No runs yet. Start one with make demo-1.`	The console started in another folder	Start `make console` in the repository folder
`permission denied` on `/var/run/docker.sock`	Your user is not in the `docker` group yet	`sudo usermod -aG docker $USER`, then `wsl.exe --shutdown` in PowerShell and reopen Ubuntu
`x SHIPHARNESS_CLASS_DATE is not set`	Class commands need the class day	Add `SHIPHARNESS_CLASS_DATE=YYYY-MM-DD` to `.env`, or add `PURPOSE=pilot`
`x missing agent model` from `make doctor`	The key cannot use the course model	Use the key from the program's email, or your own on the Build tier
`x OpenAI refused the key: ...; no arm was started`	Key invalid, model missing, or OpenAI unreachable	`make doctor` names the problem
`ABORTED: timeout` in a result line	The arm used its agent time (420 s) or harness time (900 s)	Nothing: the arm is scored and shown; read its verdict
`ABORTED: cap` in a result line	A model call would have crossed a cost cap	Nothing: scored and shown; the caps are in `.env.example`
`ERROR` with `docker run failed`	Docker stopped, or an image is missing	`sudo systemctl start docker`, then `make warm PURPOSE=pilot`
`make: command not found`	You are in PowerShell, not Ubuntu	Open the Ubuntu app (Part 0, Step 1)
`uv: command not found`	The terminal started before uv was installed	Open a new Ubuntu terminal
Screen 2 says `No board screenshot yet` all run long	The preview container did not start	Display only: the referee's board checks still run

Symptom	Cause	Fix
Screen 4 does not show a `make show-N` run	A single-arm run has nothing to compare	Expected
`make replay` asks for `SHIPHARNESS_CLASS_DATE`	`make replay` uses the class purpose by default	Add `PURPOSE=pilot`
A replay's screen 2 shows broken images	Replay does not copy the board screenshots	Copy the recorded run's `screenshots/` folder into the replay's run folder, then reload

Every other message, with the file that prints it, is in `docs/troubleshooting.md`.

Glossary

- **A/B**: bare arm and harness arm on the same ticket, scored the same way.
- **Arm**: one agent run in an A/B. The bare arm has no harness; the harness arm has it.
- **Bare arm**: an agent run with the ticket and the repository, and no harness.
- **Baseline**: the results on the unchanged app, recorded once, so later checks count only new problems.
- **Blinded reviewer**: a separate model that reads a diff without knowing which arm wrote it.
- **Cap**: the most money a run, a review, or a session may spend on model calls.
- **Checkpoint tag**: a git tag such as `m1-s1-start` that marks the code a section starts from.
- **Class date**: the class day in `.env`; it names the LangSmith project and keys the session cap.
- **Class profile**: the settings of the live class runs, the same for both arms: low reasoning effort, the Fast tier, 2 CPUs, a 420 s agent time limit, a 900 s harness work limit.
- **Console**: the browser page that shows a run live, on five screens.
- **Cost line**: the `Cost:` line every live command prints at the end.
- **Full verdict**: the primary verdict plus every other check: hidden tests, mutants and review.
- **Gate**: the checks that run when the agent says done; a failure sends the claim back.
- **Harness**: the files and code around the agent that decide what it sees and may do.
- **Harness arm**: the same run with the harness turned on.
- **Harness in action**: the harness arm alone, live, scored by the referee (`make show-N`); not an A/B.
- **Harness layer**: one part of the harness: instructions, state, scope, or verification.
- **Hidden tests**: tests the agent never sees; the referee runs them after the agent stops.
- **LangSmith**: the tracing service that records every model call of a run.
- **Map**: a short `AGENTS.md` that says where each rule lives, instead of holding every rule.
- **Mutant**: a copy of the code with one small bug planted on purpose; a good test fails on it.
- **Pill**: a status badge on console screen 1, such as the primary verdict.
- **Primary measure**: the one question a section's run answers, fixed in writing before any run.
- **Primary verdict**: the referee's answer to the primary measure for one arm: PASS or FAIL.
- **Purpose**: the make variable that picks the LangSmith project and the cost session.
- **Recording**: a run shown again by `make replay`; every screen carries a `RECORDING` badge.

- **Referee**: the program that scores each run after the agent stops.
- **Replay**: showing a recorded run again and judging it again, with no model calls.
- **Sandbox**: the container the agent's commands run in, with no network, the same for both arms.
- **Session**: one stretch of agent work that starts with a fresh context.
- **Session turn budget**: the number of turns a session gets before the runner stops it.
- **Test filters**: the checks a test from the test phase must pass to be kept.
- **Ticket**: the task the agent gets, written like a work ticket.
- **Trace**: the recorded steps of a run, each model call and tool call, in LangSmith.
- **Turn**: one model reply, with the tool calls it makes.

Fallback: replay a recorded run

Use this when a live run cannot start in class: no network, a key problem, or no time left. A replay shows a run recorded earlier on every screen, and the referee judges it again. It makes no model calls and costs nothing; the blinded reviewer does not run. Here you replay the run of Section 1: Instructions that Chapter 1 follows.

Step 1 Find the run id of the recorded run

WHERE: browser, the console's run list

TYPE: Press `H` to go to the run list, and find the card of the Section 1 run: type "harness in action", Feature 1: Org-wide issue search.

YOU SHOULD SEE: the card with its run id and start time. Every copy has its own ids: in this guide's runs it was `ab-20261018-162244-860638`, started 16:22 UTC, and on your copy it is the id of your own Chapter 1 run. The same id is on the `A/B ab-…` line that Terminal 2 printed at the end of Chapter 1.

LOOK AT: the run id: it starts with `ab-`, then the date and time of the run. Write it down; the next two steps use it as the recorded id.

WHAT IT MEANS: every run keeps its events, diff and scores in its own folder under `.shipharness/runs/`, named by its run id, so it can be shown again.

✓ **CHECKPOINT**
You have the id of a finished run.

IF IT LOOKS DIFFERENT
No Section 1 run in the list → use any finished run; the replay works the same on every run.

◆ **INSTRUCTOR NOTE**
Pick the fallback run before class and write its id in your notes, so you never search for it live.

Step 2 Start the replay

WHERE: Terminal 2

TYPE: put your recorded id from Step 1 in a shell variable, then start the replay. Replace `<recorded id>` with that id, angle brackets included:

TYPE THIS — COPY

```
REC=<recorded id>
make replay RUN=$REC PURPOSE=pilot
```

YOU SHOULD SEE: a `RECORDING:` line with the new run id, then, when the referee is done, the arm's result line and the cost line. It took 2:06 in this guide's run, where the recorded id was `ab-20261018-162244-860638`:

> Your output will differ: run times, ids, and the agent's wording change on every run.

OUTPUT

```
uv run shipharness replay ab-20261018-162244-860638 --purpose pilot
RECORDING: replay ab-20261018-165923-1d87bd of ab-20261018-162244-860638 (195 events)
  harness  completed         agent said done · House rule 1: Tenant isolation, new violations: 0 ·
primary measure passed · hidden tests 52 of 52 · mutants killed 3 of 4 · reviewer not run · cost
$0.0178
Cost: $0.000000 (this command: no calls; session $0.183679 of $2.50; build $0.183679 of $15.00
($14.816321 left))
```

LOOK AT: `reviewer not run` in the result line, and `this command: no calls` in the cost line. Then the `RECORDING:` line: the id after `replay` is the replay id, which Step 3 needs (`ab-20261018-165923-1d87bd` in this guide's run).

WHAT IT MEANS: the replay plays the 195 recorded events into a new run folder, marked as a recording, and the referee judges the same diff again. The `cost $0.0178` in the result line is what the agent cost when it was recorded; this command spent $0.

✓ **CHECKPOINT**
The cost line says `this command: no calls`, and the `RECORDING:` line names the new run id.

IF IT LOOKS DIFFERENT
`x no recorded A/B for` that id → the id is not in your `.shipharness/runs/`, for example an id copied from this guide. Copy the id from your own run list, set `REC` again, and rerun.
`RUN=` is empty → `REC` is not set in this terminal; run the `REC=` line again.
It asks for `SHIPHARNESS_CLASS_DATE` → add `PURPOSE=pilot` as above.

◆ **INSTRUCTOR NOTE**
Say it plainly: "This is a recording of a real run, not a live one." The console says so too. The referee's verdict matches the recorded one: 52 of 52 hidden tests, 3 of 4 mutants killed.

Step 3 Copy the board screenshots into the replay

WHERE: Terminal 2

TYPE: in the same terminal, so `REC` is still set, put the replay id from the `RECORDING:` line in a second variable, then copy. Replace `<replay id>` with that id, angle brackets included:

TYPE THIS — COPY

```
NEW=<replay id>
cp -a .shipharness/runs/$REC/screenshots .shipharness/runs/$NEW/
```

YOU SHOULD SEE: no output; the prompt comes back.

LOOK AT: the two ids: `REC` is the recorded run (Step 1, the run list card), `NEW` is the replay (the `RECORDING:` line, or the top card of the run list after a reload).

WHAT IT MEANS: a replay does not copy the board screenshots that screen 2 shows. This command copies them from the recorded run.

✓ **CHECKPOINT**
The command prints nothing and returns at once.

IF IT LOOKS DIFFERENT
`No such file or directory` → check both ids with `echo $REC $NEW`; the second one is the id after `replay` on the `RECORDING:` line.

◆ **INSTRUCTOR NOTE**
Do this right after the replay ends, before you open screen 2.

Step 4 Open the replay from the run list

WHERE: browser, the console's run list

TYPE: Reload the page with `F5`, then click the run id on the top card.

YOU SHOULD SEE: the replay as the newest card:

> The replay is a new run at the top of the list; the recorded Section 1 run stays at the bottom.

1. Run id: open the run
2. Type: harness in action or A/B
3. Status
4. Start time

LOOK AT: the type, "harness in action", and the status, "finished".

WHAT IT MEANS: the replay is a run of its own, with its own id, next to the run it plays.

✓ CHECKPOINT

The top card has the id from the `RECORDING:` line.

IF IT LOOKS DIFFERENT

The card's section reads "unknown" → this is how the run list labels a replay; the feature and type are right.

No new card → reload again; the list does not refresh by itself.

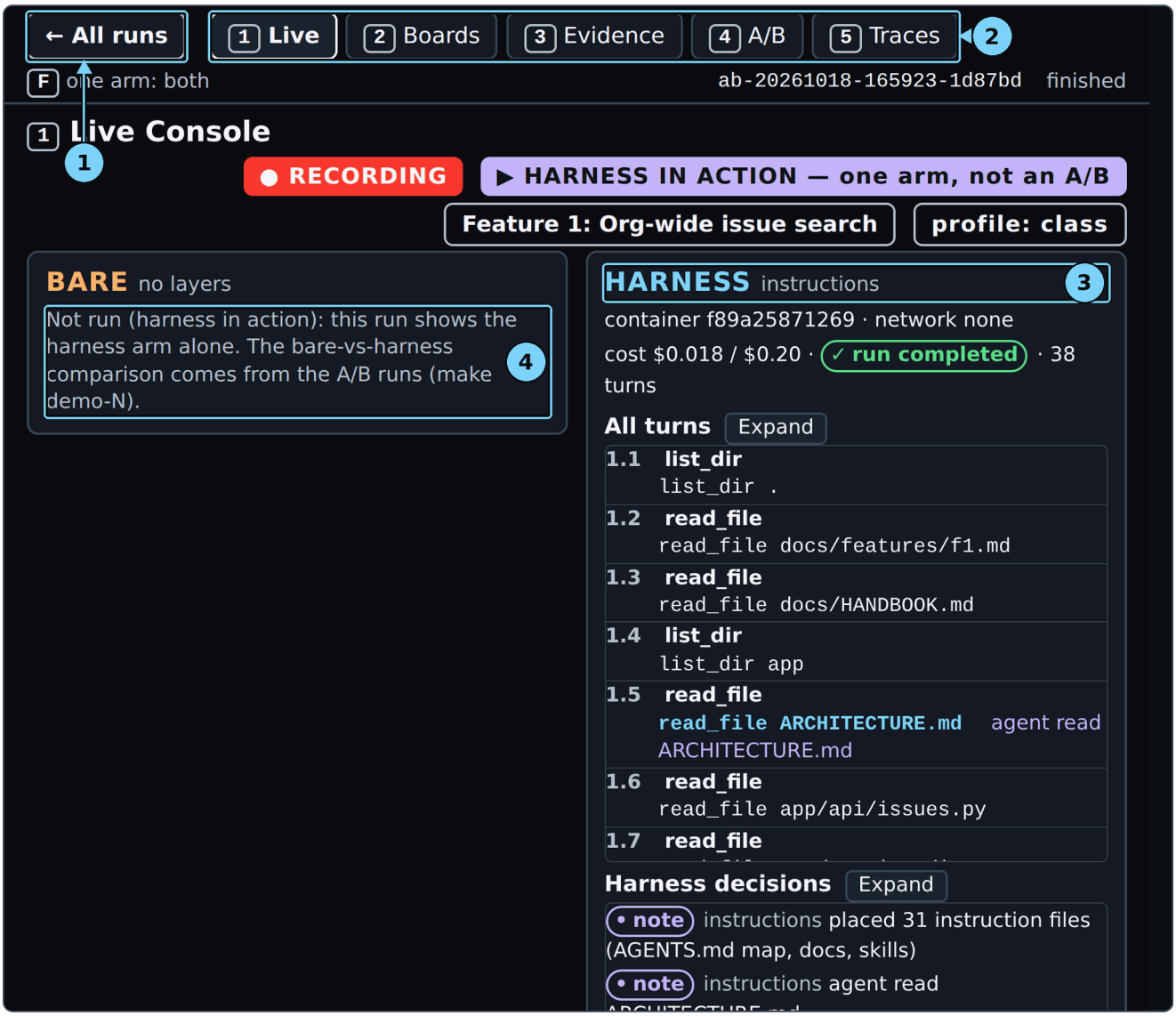
> **◆ INSTRUCTOR NOTE**
> Point at the top card and the bottom card: the replay and the run it plays are the same agent run.

Step 5 Walk the screens of the recording

WHERE: browser, the console

TYPE: Press `1`, then `2` to `5` as in Chapter 1.

YOU SHOULD SEE: the same screens as a live run, each with the red `● RECORDING` badge in its header:



Screen 1 of the replay: the badge says RECORDING, and the panels show the finished run.

1. ← All runs: back to the run list
2. Screen tabs 1 to 5
3. Harness arm line: its layers
4. Bare arm: Not run (harness in action)

LOOK AT: the `● RECORDING` badge next to the screen title, on every screen.

WHAT IT MEANS: every screen reads the recorded events, so you can teach the chapter as written, screen by screen.

✓ CHECKPOINT
Every screen shows the `● RECORDING` badge, and screen 2 shows the board screenshots.

IF IT LOOKS DIFFERENT
Screen 2 shows broken images → do Step 3, then reload.

◆ INSTRUCTOR NOTE
Follow Chapter 1 from "Open the run" onwards. Skip the waiting steps: a recording shows the finished state at once.